

libuv

libuv 中文使用手册

官方文档全译 · 实战演练 13 讲

基于 libuv 1.52.2 官方文档 (commit b05973e)

教程篇 + API 参考篇 + 实战导学

全部示例经 Linux + GCC 编译运行验证

forwaard <forwaard@163.com>

github.com/forwaard/libuv-book

目录

导读

前言

实战导学

第一部分 • 教程篇（官方 Guide 全译）

- 1 libuv 基础
- 2 高级事件循环
- 3 实用工具库
- 4 文件系统
- 5 网络编程
- 6 进程
- 7 线程与同步原语

第二部分 • 参考篇（官方 API 全译）

- 8 API：事件循环
- 9 API：句柄与请求
- 10 API：定时器与异步唤醒
- 11 API：文件系统
- 12 API：文件监控
- 13 API：DNS、线程池与指标
- 14 API：TCP 与 UDP
- 15 API：流、管道、轮询与 TTY
- 16 API：信号、进程与线程
- 17 API：杂项、错误码与设计概览

前言

前言

本书基于 libuv 1.52.2 官方文档（源码 commit b05973e）翻译整理，全部 13 个实战示例均在 Linux x86_64 + GCC 环境下编译运行验证通过。

这本书讲什么

libuv 是一个专注于异步 I/O 的跨平台支持库。它是 Node.js 的事件循环底座，也被 Luvit、Julia、uvloop 等项目采用。一句话概括：**只要你的程序需要"一边等 I/O、一边还能干别的"，libuv 就是干这个的。**

本书由两部分组成：

1. **教程篇（第 1-7 章）**：官方 Guide 的完整中文翻译，从事件循环的直觉讲起，覆盖文件系统、网络、进程、线程，穿插大量可运行的代码。
2. **参考篇（第 8-17 章）**：官方 API 文档的完整中文翻译，按主题重新编排，每个函数的签名、参数、返回值、注意事项一应俱全。日常开发把它当词典查。

翻译原则

- **术语统一**：event loop→事件循环，handle→句柄，request→请求，callback→回调，socket→套接字，thread pool→线程池；poll、ref/unref 等在中文语境更清晰的词保留英文。
- **代码不译**：所有 C 代码、函数签名、参数名、错误码保留英文原样，保证可以直接复制运行。
- **信达雅**：叙述文字按中文技术写作习惯重写，不逐词硬译；原文的警告、注意事项、版本标注一个不漏。

实战篇使用说明

每个实战示例都是独立完整的 C 文件，放在 `demos/` 目录。编译方式统一：

```
gcc demoXX_*.c -o demoXX -I$HOME/.local/include \
-L$HOME/.local/lib -luv -Wl,-rpath,$HOME/.local/lib
```

提示：如果你的系统已用包管理器安装了 libuv 开发头文件（如 `apt install libuv1-dev`），直接 `-luv` 即可，不需要 `-I / -L`。

实战示例的难度阶梯：

阶段	示例	主题
入门	demo01	第一个事件循环：idle + timer
入门	demo02	实用工具函数：时钟/内存/CPU
入门	demo03	异步文件读取（理解线程池）
进阶	demo04	prepare/check 钩子与循环生命周期
进阶	demo05	TCP echo 服务器（每连接一句柄模式）
进阶	demo06	UDP 数据报与组播
进阶	demo07	异步 DNS 解析
实战	demo08	Unix domain socket 管道
实战	demo09	子进程执行命令并捕获输出
实战	demo10	多线程 + async 跨线程唤醒（铁律！）
实战	demo11	信号处理与优雅退出
实战	demo12	文件系统事件监控（inotify 封装）
高阶	demo13	uv_queue_work 显式使用线程池

排版说明

本书 PDF 版为平板阅读优化：大字号、宽行距、代码块独立配色、双页对开。建议搭配平板的横屏双页模式阅读。

译于 2026 年 9 月。译文与代码如有疏漏，欢迎指正。

实战导学

实战导学：把翻译读"活"

本章是全书实战部分的地图。每个示例标注了对应的书内章节、核心 API 和"读后你应该能回答的问题"。建议按顺序做，每个示例动手编译运行后再往下走。

准备工作

先确认 libuv 开发环境可用（本书环境为源码编译到 `~/.local`）：

```
## 从 libuv 源码构建（如果系统没有 libuv1-dev）
git clone --depth 1 https://github.com/libuv/libuv.git
cd libuv && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$HOME/.local
make -j$(nproc) && make install

## 写个三行程序验证
cat > /tmp/t.c <<'EOF'
#include <uv.h>
#include <stdio.h>
int main(){ printf("%s\n", uv_version_string()); return 0; }
EOF
gcc /tmp/t.c -o /tmp/t -I$HOME/.local/include -L$HOME/.local/lib \
-luv -Wl,-rpath,$HOME/.local/lib && /tmp/t
```

输出 `1.52.2` 或类似版本号即就绪。

阶段一：建立直觉（demo01-03）

demo01 hello_loop —— 对应第 1、2 章。

最小可用事件循环：一个 idle 句柄空转 10 万次，一个 100ms 定时器把它叫停。运行后观察输出顺序，回答：为什么 idle 停了之后循环还能继续跑？（答：定时器还活着，句柄引用计数未归零。）

核心 API： `uv_loop_init` `uv_idle_start` `uv_timer_start` `uv_run`

demo02 utilities —— 对应第 3 章。

不写事件循环，先摸一遍工具函数：纳秒时钟打点、内存信息、CPU 列表。注意 `uv_cpu_info` 用完要 `uv_free_cpu_info`，这类"谁分配谁释放"的约定贯穿整个库。

核心 API： `uv_hrttime` `uv_uptime` `uv_cpu_info` `uv_available_parallelism`

demo03 async_fs —— 对应第 4 章。

回调式读文件。重点体会两件事：

1. 文件 I/O **不走 epoll**，是线程池里的工作线程在执行阻塞调用；
2. 回调最终在**主线程**执行，所以回调里可以安全地继续起下一个 `uv_fs_read`。

核心 API： `uv_fs_open` `uv_fs_read` `uv_fs_close` `uv_buf_init` `uv_fs_req_cleanup`

阶段二：看清循环 (demo04)

demo04 queue_watch —— 对应第 2 章循环章节。

用 `uv_prepare_t`（每圈前）和 `uv_check_t`（每圈后）给事件循环装上"行车记录仪"，打印每一圈。运行输出会清晰展示：**一圈 = timers → poll → check**，以及周期定时器如何驱动多圈循环。

核心 API： `uv_prepare_start` `uv_check_start` `uv_timer_stop` `uv_stop` `uv_close`

实战坑：demo 里所有句柄 close 之后要再 `uv_run(loop, UV_RUN_NOWAIT)` 跑一圈让 close 回调落地，否则 `uv_loop_close` 会返回 `UV_EBUSY`。

阶段三：网络三件套 (demo05-07)

demo05 tcp_echo —— 对应第 5 章和第 14 章。

单文件同时含服务器和客户端（命令行参数区分）。必须掌握的"每连接一句柄"模式：

```
uv_listen(监听句柄) --新连接--> malloc(uv_tcp_t) --> uv_accept
--> uv_read_start --数据--> 回调里 uv_write 回写 --EOF/错误--> uv_close + free
```

运行验证（两个终端）：

```
./demo05 server    # 终端 A
./demo05 client    # 终端 B, 会收到 "你好, libuv TCP!" 回显
```

核心 API: `uv_tcp_bind` `uv_listen` `uv_accept` `uv_read_start` `uv_write`

实战坑: `alloc_buffer` 里 `malloc` 的缓冲区, 在 `nread > 0` 且转交给 `uv_write` 时所有权移到 `write` 回调, 其余路径必须在 `read` 回调里 `free`——漏掉就是内存泄漏。客户端主动断开时服务器可能收到 `ECONNRESET` 而不是 `UV_EOF`, 两种都要处理。

demo06 udp —— 对应第 5 章和第 14 章。

组播发送/接收。对比 TCP 的三个不同: 没有 `accept` (`recv_start` 后直接收包)、没有连接状态 (一个句柄可以对多个对端收发)、回调带发送方地址。

核心 API: `uv_udp_bind` (注意 `UV_UDP_REUSEADDR`) `uv_udp_recv_start` `uv_udp_send` `uv_udp_set_membership`

demo07 dns —— 对应第 13 章。

`uv_getaddrinfo` 是阻塞函数 `getaddrinfo` 的线程池包装。注意 `hints.ai_family = AF_INET` 只查 IPv4, 以及 `uv_freeaddrinfo` 必须调用。

核心 API: `uv_getaddrinfo` `uv_freeaddrinfo`

阶段四：系统编程 (demo08-12)

demo08 pipe —— 对应第 15 章。

Unix domain socket 版 echo。体会 stream 抽象: 把 TCP 换成管道, `uv_read_start` / `uv_write` 一个字不用改。

核心 API: `uv_pipe_init` `uv_pipe_bind` `uv_pipe_connect` (bind 前记得 `unlink` 残留 socket 文件)

demo09 spawn —— 对应第 6 章和第 16 章。

把 `ls -l` 的 `stdout` 用管道接到父进程的流上。子进程输出"像网络流一样"异步到达——这就是 Node.js `child_process` 的原型。

核心 API: `uv_spawn` `uv_stdio_container_t` (`UV_CREATE_PIPE` | `UV_WRITABLE_PIPE`)
`uv_process_t`

实战坑：退出回调命名别用 `on_exit`，会和 `<stdlib.h>` 里的 `on_exit()` 冲突。

demo10 threads —— 对应第 7 章和第 16 章。**全书最重要的一课。**

工作线程算完 1 加到 100 万，用 `uv_async_send` 唤醒事件循环，回调在主线程读出结果。铁律：

- libuv 的循环和回调只属于创建它的线程；
- 跨线程通信只有 `uv_async_send` 这一条安全通道；
- 共享数据必须配 `uv_mutex_t`。

核心 API: `uv_thread_create` `uv_mutex_lock` `uv_async_init` `uv_async_send`

demo11 signal —— 对应第 16 章。

SIGINT 触发优雅退出。生产服务的标配姿势：收到信号 → 停止 accept → 清理 → 退出，而不是被 kill 打断。

核心 API: `uv_signal_init` `uv_signal_start` `uv_signal_stop`

demo12 fsevent —— 对应第 12 章。

监控目录变更（创建/追加/重命名各触发一次），底层 Linux inotify、macOS FSEvents 被 libuv 抹平成同一接口。适合配置热更新、目录同步等场景。

核心 API: `uv_fs_event_start` `UV_RENAME` `UV_CHANGE`

阶段五：压轴（demo13）

demo13 queue_work —— 对应第 7 章"线程池"一节。

`uv_queue_work` 是"我不想阻塞循环"的标准答案，也是文件 I/O、DNS 在内部的实现机制。运行时用 `UV_THREADPOOL_SIZE=2 ./demo13` 试试调线程池大小——两个任务在 2 线程池里真并行。

核心 API: `uv_queue_work` `uv_work_t` (`do_work` 在池线程，`after_work` 在主线程)

学完之后

1. 读 **Node.js** 源码的 **deps/uv** 目录，你会发现每个 API 都眼熟了；
2. 把 demo05 扩展成带协议帧（长度前缀）的聊天服务器，练 stream 的分包问题；
3. 用 demo10 + demo13 组合写一个"批量压缩文件"工具：主线程扫目录（`uv_fs_scandir`），线程池压缩，async 汇报进度——这就是一个生产级 libuv 程序的骨架。

第 1 章 • libuv 基础

本章译自 libuv 官方文档 `guide/basics.rst` (libuv 1.52)

libuv 基础

libuv 强制采用**异步、事件驱动**的编程风格。它的核心工作是提供一个事件循环 (event loop)，并基于回调 (callback) 对 I/O 及其他活动发出通知。libuv 还提供了一系列核心工具：定时器、非阻塞网络支持、异步文件系统访问、子进程等等。

事件循环

在事件驱动的编程模型中，应用程序对某些事件表达出兴趣，并在这些事件发生时作出响应。从操作系统收集事件、或监视其他事件来源的工作由 libuv 负责，用户只需注册回调，即可在事件发生时被调用。事件循环通常会**永远**运行下去。用伪代码表示：

```
while there are still events to process:
    e = get the next event
    if there is a callback associated with e:
        call the callback
```

事件的例子有：

- 文件已可写入
- 某个套接字 (socket) 有数据可读
- 某个定时器已超时

这个事件循环被封装在 `uv_run()` 中——它是使用 libuv 时最核心的函数。

系统程序最常见的活动是处理输入和输出，而不是大量的数值计算。使用传统输入/输出函数 (`read`、`fprintf` 等) 的问题在于它们是**阻塞**的。真正写入硬盘或从网络读取数据所耗费的时间，与处理器的速度相比长得不成比例。这些函数要等到任务完成才返回，

期间你的程序什么事都做不了。对于追求高性能的程序来说，这是一大障碍，因为其他活动和其他 I/O 操作都只能干等。

标准解决方案之一是使用线程（thread）。每个阻塞的 I/O 操作都在单独的线程（或线程池）中启动。当阻塞函数在某个线程中被调用时，操作系统可以调度另一个真正需要 CPU 的线程去运行。

libuv 采用的是另一种风格，即**异步、非阻塞**（asynchronous, non-blocking）风格。大多数现代操作系统都提供事件通知子系统。举例来说，对一个套接字发起普通的 `read` 调用会一直阻塞，直到发送方真的发来数据；替代做法是，应用程序请求操作系统替它监视这个套接字，并把事件通知放进队列。应用程序可以在自己方便的时候查看这些事件（也许在此之前先做一些数值计算，把处理器能力用满），然后取出数据。说它是**异步**的，是因为应用程序在一个时间点表达兴趣，而在另一个时间点（时间与空间上的另一点）使用数据；说它是**非阻塞**的，是因为应用程序进程可以自由地去做其他任务。这与 libuv 的事件循环方式契合得很好，因为操作系统事件可以被当作又一种 libuv 事件来处理。非阻塞保证了其他事件能在到达时被尽快继续处理¹。

注意：

I/O 在后台如何运行并不是我们需要关心的，但受限于计算机硬件的工作方式——线程是处理器的基本调度单元——libuv 和操作系统通常会运行后台/工作线程，和/或采用轮询（polling）的方式，以非阻塞的方式完成任务。

libuv 核心开发者之一 Bert Belder 有一个小视频，讲解了 libuv 的架构及其来龙去脉。如果你此前没有使用过 libuv，这个视频短小而值得一看。

libuv 的事件循环在[官方文档](#)中有更详细的说明。

[视频链接（Bert Belder 讲 libuv 架构）](#)

Hello World

基础铺垫完毕，现在来写第一个 libuv 程序。它什么也不做，只是启动一个会立即退出的循环。

helloworld/main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <uv.h>

int main() {
    uv_loop_t *loop = malloc(sizeof(uv_loop_t));
    uv_loop_init(loop);

    printf("Now quitting.\n");
    uv_run(loop, UV_RUN_DEFAULT);

    uv_loop_close(loop);
    free(loop);
    return 0;
}

```

这个程序会立即退出，因为它没有任何事件需要处理。必须通过各种 API 函数告诉 libuv 的事件循环要监视哪些事件。

从 libuv v1.0 开始，用户应当先用 `uv_loop_init(uv_loop_t *)` 初始化之前自行分配循环所需的内存。这样你就可以接入自定义的内存管理。记得用

`uv_loop_close(uv_loop_t *)` 反初始化循环，然后再释放存储空间。本书的示例从不关闭循环，因为程序在循环结束后就退出了，系统会回收内存。生产级项目，尤其是长期运行的系统程序，应当注意正确地释放资源。

默认循环

libuv 提供了一个默认循环，可以通过 `uv_default_loop()` 获取。如果你只需要一个循环，就应当使用它。

default-loop/main.c

```
#include <stdio.h>
#include <uv.h>

int main() {
    uv_loop_t *loop = uv_default_loop();

    printf("Default loop.\n");
    uv_run(loop, UV_RUN_DEFAULT);

    uv_loop_close(loop);
    return 0;
}
```

注意：

node.js 使用默认循环作为它的主循环。如果你在编写绑定（binding），应当留意这一点。

错误处理

可能失败的初始化函数或同步函数在出错时返回一个负数。可能失败的异步函数则通过回调传入一个状态参数。错误消息被定义为 `UV_E*` 常量。

你可以使用 `uv_strerror(int)` 和 `uv_err_name(int)` 函数分别获得描述错误的 `const char *` 或错误名称。

I/O 读取回调（例如文件和套接字的回调）会收到一个参数 `nread`。如果 `nread` 小于 0，说明发生了错误（`UV_EOF` 表示读到文件末尾，你可能希望对它做区别处理）。

句柄与请求

libuv 的工作方式是让用户对特定事件表达兴趣。这通常通过创建一个指向 I/O 设备、定时器或进程的**句柄**（handle）来完成。句柄是不透明的结构体，命名为 `uv_TYPE_t`，其中 type 表示该句柄的用途。

libuv 的各类 watcher

```

/* Handle types. */
typedef struct uv_loop_s uv_loop_t;
typedef struct uv_handle_s uv_handle_t;
typedef struct uv_dir_s uv_dir_t;
typedef struct uv_stream_s uv_stream_t;
typedef struct uv_tcp_s uv_tcp_t;
typedef struct uv_udp_s uv_udp_t;
typedef struct uv_pipe_s uv_pipe_t;
typedef struct uv_tty_s uv_tty_t;
typedef struct uv_poll_s uv_poll_t;
typedef struct uv_timer_s uv_timer_t;
typedef struct uv_prepare_s uv_prepare_t;
typedef struct uv_check_s uv_check_t;
typedef struct uv_idle_s uv_idle_t;
typedef struct uv_async_s uv_async_t;
typedef struct uv_process_s uv_process_t;
typedef struct uv_fs_event_s uv_fs_event_t;
typedef struct uv_fs_poll_s uv_fs_poll_t;
typedef struct uv_signal_s uv_signal_t;

/* Request types. */
typedef struct uv_req_s uv_req_t;
typedef struct uv_getaddrinfo_s uv_getaddrinfo_t;
typedef struct uv_getnameinfo_s uv_getnameinfo_t;
typedef struct uv_shutdown_s uv_shutdown_t;
typedef struct uv_write_s uv_write_t;
typedef struct uv_connect_s uv_connect_t;
typedef struct uv_udp_send_s uv_udp_send_t;
typedef struct uv_fs_s uv_fs_t;
typedef struct uv_work_s uv_work_t;
typedef struct uv_random_s uv_random_t;

/* None of the above. */
typedef struct uv_env_item_s uv_env_item_t;
typedef struct uv_cpu_info_s uv_cpu_info_t;
typedef struct uv_interface_address_s uv_interface_address_t;
typedef struct uv_dirent_s uv_dirent_t;
typedef struct uv_passwd_s uv_passwd_t;
typedef struct uv_utsname_s uv_utsname_t;
typedef struct uv_statfs_s uv_statfs_t;

```

句柄代表长生命周期的对象。对这些句柄的异步操作使用**请求**（request）来标识。请求是短生命周期的（通常只在一次回调中使用），通常对应句柄上的一次 I/O 操作。请求用于在单个动作的发起与回调之间保存上下文。举例来说，一个 UDP 套接字由 `uv_udp_t`

表示，而对该套接字的每次单独写入则使用 `uv_udp_send_t` 结构，写入完成后该结构会被传递给回调。

句柄通过对应的

```
uv_TYPE_init(uv_loop_t *, uv_TYPE_t *)
```

函数来建立。

回调是 libuv 在 watcher 感兴趣的事件发生时调用的函数。应用程序特有的逻辑通常就实现在回调里。例如，I/O watcher 的回调会收到从文件中读到的数据，定时器回调在超时时被触发，等等。

Idling

下面是一个使用 idle 句柄的例子。其回调在事件循环的每一轮都会被调用一次。idle 句柄的一个用例将在 utilities 一章中讨论。让我们用 idle watcher 来观察 watcher 的生命周期，并看看当存在 watcher 时 `uv_run()` 会如何阻塞。计数达到目标后 idle watcher 被停止，由于不再有任何处于活动状态的事件 watcher，`uv_run()` 随之退出。

idle-basic/main.c

```

#include <stdio.h>
#include <uv.h>

int64_t counter = 0;

void wait_for_a_while(uv_idle_t* handle) {
    counter++;

    if (counter >= 10e6)
        uv_idle_stop(handle);
}

int main() {
    uv_idle_t idler;

    uv_idle_init(uv_default_loop(), &idler);
    uv_idle_start(&idler, wait_for_a_while);

    printf("Idling...\n");
    uv_run(uv_default_loop(), UV_RUN_DEFAULT);

    uv_loop_close(uv_default_loop());
    return 0;
}

```

保存上下文

在基于回调的编程风格中，你常常需要在调用点与回调之间传递一些"上下文"——即应用特有的信息。所有句柄和请求都有一个 `void* data` 成员，你可以把它设置为上下文，并在回调中转换回原来的类型。这是 C 库生态中随处可见的常用模式。此外，`uv_loop_t` 也有一个类似的 `data` 成员。

1. 当然，这取决于硬件的性能。 ↔

第 2 章 · 高级事件循环

高级事件循环

本章译自 libuv 官方文档 `guide/eventloops.rst` (libuv 1.52)

libuv 为用户提供了相当多的事件循环控制手段，通过巧妙地组合多个循环，你可以实现很多有趣的效果。你甚至可以把 libuv 的事件循环嵌入到另一个基于事件循环的库中——想象一个基于 Qt 的图形界面，由 Qt 的事件循环驱动 libuv 后端来完成繁重的系统级任务。

停止事件循环

可以使用 `uv_stop()` 来停止事件循环。循环最早也要到下一次迭代才会真正停止，甚至可能更晚。这意味着本轮迭代中已经就绪、等待处理的事件仍然会被处理完，所以 `uv_stop()` 不能当作“急停开关”来用。调用 `uv_stop()` 时，循环在本轮迭代中**不会再阻塞**等待 I/O。这些语义理解起来有点费劲，我们直接看看 `uv_run()` 吧——所有控制流程都发生在那里。

`src/unix/core.c - uv_run`

```

int uv_run(uv_loop_t* loop, uv_run_mode mode) {
    int timeout;
    int r;
    int can_sleep;

    r = uv__loop_alive(loop);
    if (!r)
        uv__update_time(loop);

    /* Maintain backwards compatibility by processing timers before entering the
     * while loop for UV_RUN_DEFAULT. Otherwise timers only need to be executed
     * once, which should be done after polling in order to maintain proper
     * execution order of the conceptual event loop. */
    if (mode == UV_RUN_DEFAULT && r != 0 && loop->stop_flag == 0) {
        uv__update_time(loop);
        uv__run_timers(loop);
    }

    while (r != 0 && loop->stop_flag == 0) {
        can_sleep =
            uv__queue_empty(&loop->pending_queue) &&
            uv__queue_empty(&loop->idle_handles);

        uv__run_pending(loop);
        uv__run_idle(loop);
        uv__run_prepare(loop);

        timeout = 0;
        if ((mode == UV_RUN_ONCE && can_sleep) || mode == UV_RUN_DEFAULT)
            timeout = uv__backend_timeout(loop);

        uv__metrics_inc_loop_count(loop);

        uv__io_poll(loop, timeout);

        /* Process immediate callbacks (e.g. write_cb) a small fixed number of
         * times to avoid loop starvation.*/
        for (r = 0; r < 8 && !uv__queue_empty(&loop->pending_queue); r++)
            uv__run_pending(loop);

        /* Run one final update on the provider_idle_time in case uv__io_poll
         * returned because the timeout expired, but no events were received. This
         * call will be ignored if the provider_entry_time was either never set (if
         * the timeout == 0) or was already updated b/c an event was received.
         */
        uv__metrics_update_idle_time(loop);
    }
}

```

```

    uv__run_check(loop);
    uv__run_closing_handles(loop);

    uv__update_time(loop);
    uv__run_timers(loop);

    r = uv__loop_alive(loop);
}

return r;
}

```

`stop_flag` 由 `uv_stop()` 置位。libuv 的所有回调都在事件循环内部被调用，因此即便在回调里调用 `uv_stop()`，本轮循环迭代仍会继续执行完毕。libuv 先更新定时器，然后依次执行待处理的定时器回调、idle 回调和 prepare 回调，并调用所有待处理的 I/O 回调。如果在其中任何一个回调里调用了 `uv_stop()`，`stop_flag` 就会被置位。这会使 `uv_backend_timeout()` 返回 0，循环因此不再阻塞在 I/O 上。反过来，如果你是在某个 check 回调里调用的 `uv_stop()`，那时 I/O 已经完成，自然不受影响。

`uv_stop()` 的用武之地在于：当已经算出结果或发生错误时，可以直接关停整个循环，而无需逐一停掉所有回调。

下面的简单示例会停止循环，同时演示本轮循环迭代依然会执行完毕这一事实。

uvstop/main.c

```

#include <stdio.h>
#include <uv.h>

int64_t counter = 0;

void idle_cb(uv_idle_t *handle) {
    printf("Idle callback\n");
    counter++;

    if (counter >= 5) {
        uv_stop(uv_default_loop());
        printf("uv_stop() called\n");
    }
}

void prep_cb(uv_prepare_t *handle) {
    printf("Prep callback\n");
}

int main() {
    uv_idle_t idler;
    uv_prepare_t prep;

    uv_idle_init(uv_default_loop(), &idler);
    uv_idle_start(&idler, idle_cb);

    uv_prepare_init(uv_default_loop(), &prep);
    uv_prepare_start(&prep, prep_cb);

    uv_run(uv_default_loop(), UV_RUN_DEFAULT);

    return 0;
}

```

第 3 章 · 实用工具库

实用工具库

本章译自 libuv 官方文档 `guide/utilities.rst` (libuv 1.52)

本章汇总了处理常见任务的实用工具和技巧。`libev man page` 中已经介绍过一些惯用法，只需稍作 API 调整即可移植到 libuv 上。本章还会讲到 libuv API 中那些不足以单独成章的部分。

定时器

定时器 (timer) 会在启动后经过指定时间时调用回调。libuv 的定时器还可以设置成按固定间隔反复触发，而不只是触发一次。

基本用法是初始化一个观察器，然后用 `timeout` 和可选的 `repeat` 启动它。定时器随时都可以停止。

```
uv_timer_t timer_req;

uv_timer_init(loop, &timer_req);
uv_timer_start(&timer_req, callback, 5000, 2000);
```

这段代码会启动一个循环定时器：它在 `uv_timer_start` 执行 5 秒后 (`timeout`) 首次触发，之后每 2 秒 (`repeat`) 重复一次。调用：

```
uv_timer_stop(&timer_req);
```

即可停止定时器。在回调内部这样调用也是安全的。

重复间隔可以用下面的函数随时修改：

```
uv_timer_set_repeat(uv_timer_t *timer, int64_t repeat);
```

修改会**尽早生效**。如果这个函数是在定时器回调里调用的，具体来说：

- 如果该定时器是非循环的，它此时已经被停止了，需要再次调用 `uv_timer_start`。
- 如果该定时器是循环的，下一次超时已经排定，因此旧的重复间隔还会生效一次，之后定时器才会切换到新闻隔。

工具函数：

```
int uv_timer_again(uv_timer_t *)
```

只适用于循环定时器，效果等价于停止定时器，然后以旧的 `repeat` 值同时作为初始 `timeout` 和 `repeat` 重新启动它。如果定时器尚未启动，则调用失败（错误码 `UV_EINVAL`）并返回 `-1`。

一个实际的定时器示例见事件循环引用计数一节。

事件循环引用计数

只要还有活跃的句柄（handle），事件循环就会一直运行。这套机制的原理是：每个句柄在启动时递增事件循环的引用计数，在停止时递减。此外，也可以用下面的函数手动调整句柄的引用计数：

```
void uv_ref(uv_handle_t*);
void uv_unref(uv_handle_t*);
```

这两个函数可以用来让循环在观察器仍然活跃时退出，或者用自定义对象来维持循环存活。

后一种用法常与循环定时器配合。比如你有一个每 `X` 秒运行一次的垃圾回收器，或者网络服务要定期向其他节点发送心跳，但你不想在每一条正常退出路径或错误场景下都得记得把它们停掉；又或者你希望程序在其余所有观察器都结束后就退出。这时只需在创建定时器后立刻 `unref` 它，这样当它成为唯一还在运行的观察器时，`uv_run` 依然会退出。

`node.js` 中也用到了这个机制：某些 `libuv` 方法被上抛为 JS API。每个 JS 对象都会创建一个对应的 `uv_handle_t`（所有观察器的超类），并可以对它进行 `ref/unref`。

ref-timer/main.c

```

uv_loop_t *loop;
uv_timer_t gc_req;
uv_timer_t fake_job_req;

void gc(uv_timer_t *handle) {
    fprintf(stderr, "Freeing unused objects\n");
}

void fake_job(uv_timer_t *handle) {
    fprintf(stdout, "Fake job done\n");
}

int main() {
    loop = uv_default_loop();

    uv_timer_init(loop, &gc_req);
    uv_unref((uv_handle_t*) &gc_req);

    uv_timer_start(&gc_req, gc, 0, 2000);

    // could actually be a TCP download or something
    uv_timer_init(loop, &fake_job_req);
    uv_timer_start(&fake_job_req, fake_job, 9000, 0);
    return uv_run(loop, UV_RUN_DEFAULT);
}

```

我们初始化了垃圾回收定时器，然后立即 `unref` 它。注意观察：9 秒后假任务完成时，程序自动退出了，尽管垃圾回收器仍在运行。

Idle 模式

idle 句柄的回调在每轮事件循环中都会被调用一次。idle 回调适合用来执行优先级极低的任务。例如，你可以在系统空闲期间向开发人员分发一份当日应用性能汇总供分析，或者用应用的 CPU 时间跑 SETI 计算 :) 在 GUI 应用中，idle 观察器同样好用。假设你用事件循环下载文件，如果 TCP 套接字还在建立中、又没有其他事件，事件循环就会暂停（阻塞），这意味着进度条会卡住不动，用户面对的是一个失去响应的应用。这种情况下，挂载一个 idle 观察器就能让 UI 保持运转。

idle-compute/main.c

```

uv_loop_t *loop;
uv_fs_t stdin_watcher;
uv_idle_t idler;
char buffer[1024];

void crunch_away(uv_idle_t* handle) {
    // Compute extra-terrestrial life
    // fold proteins
    // computer another digit of PI
    // or similar
    fprintf(stderr, "Computing PI...\n");
    // just to avoid overwhelming your terminal emulator
    uv_idle_stop(handle);
}

void on_type(uv_fs_t *req) {
    if (stdin_watcher.result > 0) {
        buffer[stdin_watcher.result] = '\0';
        printf("Typed %s\n", buffer);

        uv_buf_t buf = uv_buf_init(buffer, 1024);
        uv_fs_read(loop, &stdin_watcher, 0, &buf, 1, -1, on_type);
        uv_idle_start(&idler, crunch_away);
    }
    else if (stdin_watcher.result < 0) {
        fprintf(stderr, "error opening file: %s\n", uv_strerror(req->result));
    }
}

int main() {
    loop = uv_default_loop();

    uv_idle_init(loop, &idler);

    uv_buf_t buf = uv_buf_init(buffer, 1024);
    uv_fs_read(loop, &stdin_watcher, 0, &buf, 1, -1, on_type);
    uv_idle_start(&idler, crunch_away);
    return uv_run(loop, UV_RUN_DEFAULT);
}

```

这里我们初始化 `idle` 观察器，并把它和真正关心的 I/O 事件一起挂进循环。此后 `crunch_away` 会被反复调用，直到用户输入内容并按下回车。处理输入数据期间它会被短暂打断，随后循环又继续调用 `idle` 回调。

idle-compute/main.c

```
void crunch_away(uv_idle_t* handle) {
    // Compute extra-terrestrial life
    // fold proteins
    // computer another digit of PI
    // or similar
    fprintf(stderr, "Computing PI...\n");
    // just to avoid overwhelming your terminal emulator
    uv_idle_stop(handle);
}
```

向工作线程传递数据

使用 `uv_queue_work` 时，通常需把复杂的数据传给工作线程。解决办法是使用一个 `struct`，并让 `uv_work_t.data` 指向它。一个略有变化的写法是把 `uv_work_t` 本身作为该结构体的第一个成员（这种结构体称为 `baton[1]`）。这样可以只调用一次 `free` 就同时释放工作请求和所有数据。

```
struct ftp_baton {
    uv_work_t req;
    char *host;
    int port;
    char *username;
    char *password;
}
```

```
ftp_baton *baton = (ftp_baton*) malloc(sizeof(ftp_baton));
baton->req.data = (void*) baton;
baton->host = strdup("my.webhost.com");
baton->port = 21;
// ...

uv_queue_work(loop, &baton->req, ftp_session, ftp_cleanup);
```

这里我们创建了 `baton` 并把任务入队。

现在任务函数就可以取出自己需要的数据了：

```

void ftp_session(uv_work_t *req) {
    ftp_baton *baton = (ftp_baton*) req->data;

    fprintf(stderr, "Connecting to %s\n", baton->host);
}

void ftp_cleanup(uv_work_t *req) {
    ftp_baton *baton = (ftp_baton*) req->data;

    free(baton->host);
    // ...
    free(baton);
}

```

随后我们释放 baton，观察器也随之一起被释放。

使用轮询 (poll) 处理外部 I/O

通常，第三方库会自己处理 I/O，并在内部维护自己的套接字和其他文件。这种情况下虽然无法使用标准的流式 I/O 操作，但仍然可以把这个库整合进 libuv 的事件循环——前提是该库允许你访问底层的文件描述符，并且提供能把任务拆成小步推进的函数，由你的应用决定何时推进。不过有些库不允许这种访问，只提供一个标准的阻塞函数，把整个 I/O 事务做完才返回。把这类库用在事件循环线程里是不明智的，请改用[线程池](#) (threadpool)。当然，这也意味着失去对库的细粒度控制。

libuv 的 `uv_poll` 部分所做的，就是借助操作系统的通知机制监视文件描述符。从某种意义上说，libuv 自己实现的所有 I/O 操作，底层也是由类似 `uv_poll` 的代码支撑的。每当操作系统检测到被轮询的文件描述符状态变化时，libuv 就会调用相应的回调。

下面我们来写一个简单的下载管理器，它使用 `libcurl_` 下载文件。我们不把控制权完全交给 `libcurl`，而是使用 libuv 事件循环，配合非阻塞的异步 `multi_` 接口：每当 libuv 通知 I/O 就绪时，就让下载往前推进一步。

uvwget/main.c - 整体初始化

```

uv_loop_t *loop;
CURLM *curl_handle;
uv_timer_t timeout;

typedef struct curl_context_s {
    uv_poll_t poll_handle;
    curl_socket_t sockfd;
} curl_context_t;

curl_context_t *create_curl_context(curl_socket_t sockfd) {
    curl_context_t *context;

    context = (curl_context_t*) malloc(sizeof *context);

    context->sockfd = sockfd;

    int r = uv_poll_init_socket(loop, &context->poll_handle, sockfd);
    assert(r == 0);
    context->poll_handle.data = context;

    return context;
}

void curl_close_cb(uv_handle_t *handle) {
    curl_context_t *context = (curl_context_t*) handle->data;
    free(context);
}

void destroy_curl_context(curl_context_t *context) {
    uv_close((uv_handle_t*) &context->poll_handle, curl_close_cb);
}

void add_download(const char *url, int num) {
    char filename[50];
    sprintf(filename, "%d.download", num);
    FILE *file;

    file = fopen(filename, "w");
    if (file == NULL) {
        fprintf(stderr, "Error opening %s\n", filename);
        return;
    }

    CURL *handle = curl_easy_init();
    curl_easy_setopt(handle, CURLOPT_WRITEDATA, file);
    curl_easy_setopt(handle, CURLOPT_URL, url);

```

```

    curl_multi_add_handle(curl_handle, handle);
    fprintf(stderr, "Added download %s -> %s\n", url, filename);
}

void check_multi_info(void) {
    char *done_url;
    CURLMsg *message;
    int pending;

    while ((message = curl_multi_info_read(curl_handle, &pending))) {
        switch (message->msg) {
            case CURLMSG_DONE:
                curl_easy_getinfo(message->easy_handle, CURLINFO_EFFECTIVE_URL,
                                &done_url);
                printf("%s DONE\n", done_url);

                curl_multi_remove_handle(curl_handle, message->easy_handle);
                curl_easy_cleanup(message->easy_handle);
                break;

            default:
                fprintf(stderr, "CURLMSG default\n");
                abort();
        }
    }
}

void curl_perform(uv_poll_t *req, int status, int events) {
    uv_timer_stop(&timeout);
    int running_handles;
    int flags = 0;
    if (status < 0) flags = CURL_CSELECT_ERR;
    if (!status && events & UV_READABLE) flags |= CURL_CSELECT_IN;
    if (!status && events & UV_WRITABLE) flags |= CURL_CSELECT_OUT;

    curl_context_t *context;

    context = (curl_context_t*)req;

    curl_multi_socket_action(curl_handle, context->sockfd, flags, &running_handles);
    check_multi_info();
}

void on_timeout(uv_timer_t *req) {
    int running_handles;
    curl_multi_socket_action(curl_handle, CURL_SOCKET_TIMEOUT, 0, &running_handles);
}

```

```

es);
    check_multi_info();
}

void start_timeout(CURLM *multi, long timeout_ms, void *userp) {
    if (timeout_ms <= 0)
        timeout_ms = 1; /* 0 means directly call socket_action, but we'll do it
in a bit */
    uv_timer_start(&timeout, on_timeout, timeout_ms, 0);
}

int handle_socket(CURL *easy, curl_socket_t s, int action, void *userp, void *socketp) {
    curl_context_t *curl_context;
    if (action == CURL_POLL_IN || action == CURL_POLL_OUT) {
        if (socketp) {
            curl_context = (curl_context_t*) socketp;
        }
        else {
            curl_context = create_curl_context(s);
            curl_multi_assign(curl_handle, s, (void *) curl_context);
        }
    }

    switch (action) {
        case CURL_POLL_IN:
            uv_poll_start(&curl_context->poll_handle, UV_READABLE, curl_perform)
;
            break;
        case CURL_POLL_OUT:
            uv_poll_start(&curl_context->poll_handle, UV_WRITABLE, curl_perform)
;
            break;
        case CURL_POLL_REMOVE:
            if (socketp) {
                uv_poll_stop(&((curl_context_t*)socketp)->poll_handle);
                destroy_curl_context((curl_context_t*) socketp);
                curl_multi_assign(curl_handle, s, NULL);
            }
            break;
        default:
            abort();
    }

    return 0;
}

```

```

int main(int argc, char **argv) {
    loop = uv_default_loop();

    if (argc <= 1)
        return 0;

    if (curl_global_init(CURL_GLOBAL_ALL)) {
        fprintf(stderr, "Could not init cURL\n");
        return 1;
    }

    uv_timer_init(loop, &timeout);

    curl_handle = curl_multi_init();
    curl_multi_setopt(curl_handle, CURLMOPT_SOCKETFUNCTION, handle_socket);
    curl_multi_setopt(curl_handle, CURLMOPT_TIMERFUNCTION, start_timeout);

    while (argc-- > 1) {
        add_download(argv[argc], argc);
    }

    uv_run(loop, UV_RUN_DEFAULT);
    curl_multi_cleanup(curl_handle);
    return 0;
}

```

每个库与 libuv 的集成方式各不相同。就 libcurl 而言，我们可以注册两个回调。套接字回调 `handle_socket` 会在套接字状态变化、需要开始轮询它的时候被调用。`start_timeout` 由 libcurl 调用，通知我们下一个超时间隔——超时后无论 I/O 状态如何，都应当驱动 libcurl 继续前进，以便它能处理错误或做其他推动下载所必需的事情。我们的下载器按如下方式调用：

```
$ ./uvwget [url1] [url2] ...
```

因此我们把每个参数作为一个 URL 添加进去。

uvwget/main.c - 添加 URL

```

void add_download(const char *url, int num) {
    char filename[50];
    sprintf(filename, "%d.download", num);
    FILE *file;

    file = fopen(filename, "w");
    if (file == NULL) {
        fprintf(stderr, "Error opening %s\n", filename);
        return;
    }

    CURL *handle = curl_easy_init();
    curl_easy_setopt(handle, CURLOPT_WRITEDATA, file);
    curl_easy_setopt(handle, CURLOPT_URL, url);
    curl_multi_add_handle(curl_handle, handle);
    fprintf(stderr, "Added download %s -> %s\n", url, filename);
}

```

我们让 libcurl 直接把数据写入文件，但只要你想，还能做得更多。

libcurl 首次会立即调用 `start_timeout`，于是整个流程开始运转。它只是启动了一个 libuv 定时器（见[定时器](#)一节），每当定时器超时，就带着 `CURL_SOCKET_TIMEOUT` 驱动一次 `curl_multi_socket_action`。`curl_multi_socket_action` 是驱动 libcurl 的核心，套接字状态变化时我们也会调用它。不过在此之前，先来看看 `handle_socket` 被调用时如何轮询套接字。

uvwget/main.c - 设置轮询

```

int handle_socket(CURL *easy, curl_socket_t s, int action, void *userp, void
*socketp) {
    curl_context_t *curl_context;
    if (action == CURL_POLL_IN || action == CURL_POLL_OUT) {
        if (socketp) {
            curl_context = (curl_context_t*) socketp;
        }
        else {
            curl_context = create_curl_context(s);
            curl_multi_assign(curl_handle, s, (void *) curl_context);
        }
    }

    switch (action) {
        case CURL_POLL_IN:
            uv_poll_start(&curl_context->poll_handle, UV_READABLE, curl_perform)
;
            break;
        case CURL_POLL_OUT:
            uv_poll_start(&curl_context->poll_handle, UV_WRITABLE, curl_perform)
;
            break;
        case CURL_POLL_REMOVE:
            if (socketp) {
                uv_poll_stop(&((curl_context_t*)socketp)->poll_handle);
                destroy_curl_context((curl_context_t*) socketp);
                curl_multi_assign(curl_handle, s, NULL);
            }
            break;
        default:
            abort();
    }

    return 0;
}

```

我们关心的是套接字描述符 `s` 和 `action`。对每个套接字，如果尚无对应的 `uv_poll_t` 句柄，就创建一个，并用 `curl_multi_assign` 把它和套接字关联起来。这样每当回调被调用时，`socketp` 就指向它。

当下载完成或失败时，libcurl 会请求移除轮询，于是我们停止并释放对应的 poll 句柄。

根据 libcurl 想监视的事件，我们分别以 `UV_READABLE` 或 `UV_WRITABLE` 启动轮询。此后只要套接字可读或可写，libuv 就会调用 poll 回调。对同一个句柄多次调用 `uv_poll_start` 是允许的，它只会用新值更新事件掩码。`curl_perform` 是这个程序的关键所在。

uvwget/main.c - 驱动 libcurl

```
void curl_perform(uv_poll_t *req, int status, int events) {
    uv_timer_stop(&timeout);
    int running_handles;
    int flags = 0;
    if (status < 0)                flags = CURL_CSELECT_ERR;
    if (!status && events & UV_READABLE) flags |= CURL_CSELECT_IN;
    if (!status && events & UV_WRITABLE) flags |= CURL_CSELECT_OUT;

    curl_context_t *context;

    context = (curl_context_t*)req;

    curl_multi_socket_action(curl_handle, context->sockfd, flags, &running_handles);
    check_multi_info();
}
```

我们要做的第一件事是停掉定时器，因为这段时间内下载已有进展。然后根据触发回调的事件设置正确的标志，再带上取得进展的套接字和描述所发生事件的标志调用 `curl_multi_socket_action`。此时 libcurl 会在内部以小步推进的方式完成它的全部工作，并尽力尽快返回——这正是事件驱动型程序对主线程的期望。libcurl 会不断把传输进度消息排进自己的队列。在本例中我们只关心已完成的传输，所以把这些消息取出来，并清理传输已完成的句柄。

uvwget/main.c - 读取传输状态

```

void check_multi_info(void) {
    char *done_url;
    CURLMsg *message;
    int pending;

    while ((message = curl_multi_info_read(curl_handle, &pending))) {
        switch (message->msg) {
            case CURLMSG_DONE:
                curl_easy_getinfo(message->easy_handle, CURLINFO_EFFECTIVE_URL,
                                &done_url);
                printf("%s DONE\n", done_url);

                curl_multi_remove_handle(curl_handle, message->easy_handle);
                curl_easy_cleanup(message->easy_handle);
                break;

            default:
                fprintf(stderr, "CURLMSG default\n");
                abort();
        }
    }
}

```

Check 与 Prepare 观察器

TODO

加载动态库

libuv 提供了一套跨平台 API 来动态加载共享库。它可以用来实现你自己的插件/扩展/模块系统，node.js 就是用它来实现 `require()` 对绑定的支持的。只要你的库导出了正确的符号，用法相当简单。加载第三方代码时务必做好健壮性和安全性检查，否则程序的行为将不可预测。下面的例子实现了一个非常简单的插件系统，除了打印插件名字之外什么也不做。

先来看提供给插件作者使用的接口：

plugin/plugin.h

```
#ifndef UVBOOK_PLUGIN_SYSTEM
#define UVBOOK_PLUGIN_SYSTEM

// Plugin authors should use this to register their plugins with mfp.
void mfp_register(const char *name);

#endif
```

你可以照此添加更多函数，让插件作者能在你的应用里做更多有用的事情[2]。一个使用该 API 的示例插件如下：

plugin/hello.c

```
#include "plugin.h"

void initialize() {
    mfp_register("Hello World!");
}
```

我们的接口规定，所有插件都应提供一个 `initialize` 函数，由应用程序调用。该插件被编译成共享库，运行我们的应用即可加载它：

```
$ ./plugin libhello.dylib
Loading libhello.dylib
Registered plugin "Hello World!"
```

注意：共享库的文件名因平台而异。在 Linux 上是 `libhello.so`。

具体过程是：先用 `uv_dlopen` 加载共享库 `libhello.dylib`，然后用 `uv_dlsym` 拿到 `initialize` 函数并调用它。

plugin/main.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#include <uv.h>

#include "plugin.h"

typedef void (*init_plugin_function)();

void mfp_register(const char *name) {
    fprintf(stderr, "Registered plugin \"%s\"\n", name);
}

int main(int argc, char **argv) {
    if (argc == 1) {
        fprintf(stderr, "Usage: %s [plugin1] [plugin2] ...\n", argv[0]);
        return 0;
    }

    uv_lib_t lib;
    while (--argc) {
        fprintf(stderr, "Loading %s\n", argv[argc]);
        if (uv_dlopen(argv[argc], &lib)) {
            fprintf(stderr, "Error: %s\n", uv_dlerror(&lib));
            continue;
        }

        init_plugin_function init_plugin;
        if (uv_dlsym(&lib, "initialize", (void **) &init_plugin)) {
            fprintf(stderr, "dlsym error: %s\n", uv_dlerror(&lib));
            continue;
        }

        init_plugin();
    }
    return 0;
}

```

`uv_dlopen` 接收共享库的路径，并设置不透明的 `uv_lib_t` 指针。成功返回 0，出错返回 -1。可用 `uv_dlerror` 获取错误信息。

`uv_dlsym` 把符号的指针存入第三个参数指向的位置（第二个参数为库名）。

`init_plugin_function` 是一个函数指针，指向我们在应用插件中所寻找的那类函数。

终端 (TTY)

文本终端很早就支持基本的格式化功能，命令集也**相当标准化**。程序经常利用这些格式化手段来提升终端输出的可读性，比如 `grep --colour`。libuv 提供了 `uv_tty_t` 抽象（一种流）及相关函数，可在所有平台上实现 ANSI 转义码——也就是说，libuv 会把 ANSI 代码转换为 Windows 上的等价物，并提供获取终端信息的函数。

首先要用读写所依据的文件描述符来初始化一个 `uv_tty_t`，通过下面的函数完成：

```
int uv_tty_init(uv_loop_t*, uv_tty_t*, uv_file fd, int unused)
```

`unused` 参数如今已被自动检测并忽略；它以前必须正确设置，才能在该流上使用 `uv_read_start()`。

接下来最好用 `uv_tty_set_mode` 把模式设为 `normal`，这样可以启用大多数 TTY 格式化、流控制等设置。此外还有**其他模式**可用。

记住在程序退出时调用 `uv_tty_reset_mode` 来恢复终端状态——这只是基本的礼貌。另一件有礼貌的事是留意输出重定向：如果用户把你的命令输出重定向到了文件，就不应写入控制序列，否则会妨碍阅读和 `grep`。要判断文件描述符是否真的是 TTY，可以调用 `uv_guess_handle` 并把返回值与 `UV_TTY` 比较。

下面这个简单的例子在红色背景上打印白色文字：

tty/main.c

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <uv.h>

uv_loop_t *loop;
uv_tty_t tty;

int main() {
    uv_write_t req;
    uv_buf_t buf;
    uv_write_t req1;
    uv_buf_t buf1;

    loop = uv_default_loop();
    uv_tty_init(loop, &tty, STDOUT_FILENO, 0);
    uv_tty_set_mode(&tty, UV_TTY_MODE_NORMAL);

    if (uv_guess_handle(1) == UV_TTY) {
        buf1.base = "\033[41;37m";
        buf1.len = strlen(buf1.base);
        uv_write(&req1, (uv_stream_t*) &tty, &buf1, 1, NULL);
    }

    buf.base = "Hello TTY\n";
    buf.len = strlen(buf.base);
    uv_write(&req, (uv_stream_t*) &tty, &buf, 1, NULL);

    uv_tty_reset_mode();
    return uv_run(loop, UV_RUN_DEFAULT);
}

```

最后一个 TTY 辅助函数是 `uv_tty_get_winsize()`，用于获取终端的宽和高，成功时返回 0。下面这个小程序利用该函数以及字符定位转义码做了一点动画。

tty-gravity/main.c

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <uv.h>

uv_loop_t *loop;
uv_tty_t tty;
uv_timer_t tick;
uv_write_t write_req;
int width, height;
int pos = 0;
char *message = " Hello TTY ";

void update(uv_timer_t *req) {
    char data[500];

    uv_buf_t buf;
    buf.base = data;
    buf.len = sprintf(data, "\033[2J\033[H\033[%dB\033[%luC\033[42;37m%s",
                        pos,
                        (unsigned long) (width-strlen(message))/2,
                        message);
    uv_write(&write_req, (uv_stream_t*) &tty, &buf, 1, NULL);

    pos++;
    if (pos > height) {
        uv_tty_reset_mode();
        uv_timer_stop(&tick);
    }
}

int main() {
    loop = uv_default_loop();

    uv_tty_init(loop, &tty, STDOUT_FILENO, 0);
    uv_tty_set_mode(&tty, 0);

    if (uv_tty_get_winsize(&tty, &width, &height)) {
        fprintf(stderr, "Could not get TTY information\n");
        uv_tty_reset_mode();
        return 1;
    }

    fprintf(stderr, "Width %d, height %d\n", width, height);
    uv_timer_init(loop, &tick);
    uv_timer_start(&tick, update, 200, 200);
}

```

```
return uv_run(loop, UV_RUN_DEFAULT);
}
```

用到的转义码如下：

转义码	含义
2 J	清除部分屏幕，2 表示整屏
H	将光标移动到指定位置，默认为左上角
n B	光标下移 n 行
n C	光标右移 n 列
m	应用一组显示设置，本例中为绿色背景（40+2）、白色文字（30+7）

可以看出，这对输出漂亮的格式化文本非常有用——如果你有兴致，甚至可以拿来做基于控制台的街机游戏。想要更花哨的控制，可以试试 `ncurses`。

自 1.23.1 版本起变更：`uv_tty_init` 的 `readable` 参数现已废弃并被忽略，适当的取值会由内核自动检测。

[1] 我最早是在 Konstantin Käfer 关于编写 `node.js` 绑定的精彩幻灯片中见到 `baton` 这个术语的：<https://kkaefer.github.io/node-cpp-modules/#baton>（视频）

[2] `mfp` 的意思是 My Fancy Plugin。

第 4 章 · 文件系统

文件系统

本章译自 libuv 官方文档 `guide/filesystem.rst` (libuv 1.52)

简单的文件系统读写操作通过 `uv_fs_*` 系列函数和 `uv_fs_t` 结构体来实现。

注意：libuv 的文件系统操作与套接字操作不同。套接字操作使用操作系统提供的非阻塞操作，而文件系统操作在内部使用阻塞函数，但这些函数是在线程池（`thread pool`）中调用的，并在需要与应用程序交互时通知注册到事件循环的监视器。

所有文件系统函数都有两种形式——同步和异步。

同步形式在回调为 `null` 时被自动调用（并且会阻塞）。函数的返回值是 libuv 错误码，这通常只对同步调用有用。异步形式在传入回调时被调用，此时返回值为 0。

读写文件

通过下面的函数可以获得一个文件描述符：

```
int uv_fs_open(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, int mode, uv_fs_cb cb)
```

`flags` 和 `mode` 是标准的 Unix 标志。libuv 会负责把它们转换成 Windows 上相应的标志。

文件描述符通过下面的函数关闭：

```
int uv_fs_close(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

文件系统操作回调的签名为：

```
void callback(uv_fs_t* req);
```

下面来看一个简单的 `cat` 实现。首先我们为文件打开事件注册一个回调：

uvcat/main.c - 打开文件

```
void on_open(uv_fs_t *req) {
    // The request passed to the callback is the same as the one the call setup
    // function was passed.
    assert(req == &open_req);
    if (req->result >= 0) {
        iov = uv_buf_init(buffer, sizeof(buffer));
        uv_fs_read(uv_default_loop(), &read_req, req->result,
                  &iov, 1, -1, on_read);
    }
    else {
        fprintf(stderr, "error opening file: %s\n", uv_strerror((int)req-
>result));
    }
}
```

在 `uv_fs_open` 的回调中，`uv_fs_t` 的 `result` 字段就是文件描述符。如果文件成功打开，我们就开始读取它。

uvcat/main.c - 读回调

```
void on_read(uv_fs_t *req) {
    if (req->result < 0) {
        fprintf(stderr, "Read error: %s\n", uv_strerror(req->result));
    }
    else if (req->result == 0) {
        uv_fs_t close_req;
        // synchronous
        uv_fs_close(uv_default_loop(), &close_req, open_req.result, NULL);
    }
    else if (req->result > 0) {
        iov.len = req->result;
        uv_fs_write(uv_default_loop(), &write_req, 1, &iov, 1, -1, on_write);
    }
}
```

在读调用中，你需要传入一个已初始化的缓冲区，它会在读回调被触发之前填充数据。

`uv_fs_*` 系列操作几乎直接对应于某些 POSIX 函数，因此在这种情况下，EOF 以 `result` 为 0 来表示。而对于流或管道，则会以 `UV_EOF` 常量作为状态传入。

这里可以看到编写异步程序时的一个常见模式：`uv_fs_close()` 调用是同步执行的。通常，一次性的任务，或者属于启动/关闭阶段的任务，都会同步执行，因为我们关心的是程序执行主要任务、处理多个 I/O 源时的 I/O 速度。对于单个任务来说，性能差异通常可以忽略不计，而且同步写法还能让代码更简单。

使用 `uv_fs_write()` 进行文件系统写操作同样简单。你的回调会在写入完成后被触发。在我们的例子里，回调只是简单地驱动下一次读取。于是读和写通过回调的方式交替进行、步调一致。

uvcat/main.c - 写回调

```
void on_write(uv_fs_t *req) {
    if (req->result < 0) {
        fprintf(stderr, "Write error: %s\n", uv_strerror((int)req->result));
    }
    else {
        uv_fs_read(uv_default_loop(), &read_req, open_req.result, &iiov, 1, -1, 0,
n_read);
    }
}
```

警告： 由于文件系统和磁盘驱动出于性能考虑的配置方式，一次"成功"的写入可能尚未真正提交到磁盘。

我们在 `main()` 中推动第一块多米诺骨牌：

uvcat/main.c

```
int main(int argc, char **argv) {
    uv_fs_open(uv_default_loop(), &open_req, argv[1], 0_RDONLY, 0, on_open);
    uv_run(uv_default_loop(), UV_RUN_DEFAULT);

    uv_fs_req_cleanup(&open_req);
    uv_fs_req_cleanup(&read_req);
    uv_fs_req_cleanup(&write_req);
    return 0;
}
```

警告：对文件系统请求必须始终调用 `uv_fs_req_cleanup()` 函数，以释放 libuv 内部的内存分配。

文件系统操作

所有标准的文件系统操作，如 `unlink`、`rmdir`、`stat` 等，都支持异步方式，并且参数顺序直观易记。它们遵循与读/写/打开调用相同的模式，将结果返回在 `uv_fs_t.result` 字段中。完整列表如下：

文件系统操作

```

int uv_fs_close(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb);
int uv_fs_open(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, int mode, uv_fs_cb cb);
int uv_fs_read(uv_loop_t* loop, uv_fs_t* req, uv_file file, const uv_buf_t bufs[], unsigned int nbufs, int64_t offset, uv_fs_cb cb);
int uv_fs_unlink(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb);
int uv_fs_write(uv_loop_t* loop, uv_fs_t* req, uv_file file, const uv_buf_t bufs[], unsigned int nbufs, int64_t offset, uv_fs_cb cb);
int uv_fs_copyfile(uv_loop_t* loop, uv_fs_t* req, const char* path, const char* new_path, int flags, uv_fs_cb cb);
int uv_fs_mkdir(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode, uv_fs_cb cb);
int uv_fs_mkdtemp(uv_loop_t* loop, uv_fs_t* req, const char* tpl, uv_fs_cb cb);
int uv_fs_mkstemp(uv_loop_t* loop, uv_fs_t* req, const char* tpl, uv_fs_cb cb);
int uv_fs_rmdir(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb);
int uv_fs_scandir(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, uv_fs_cb cb);
int uv_fs_scandir_next(uv_fs_t* req, uv_dirent_t* ent);
int uv_fs_opendir(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb);
int uv_fs_readdir(uv_loop_t* loop, uv_fs_t* req, uv_dir_t* dir, uv_fs_cb cb);
int uv_fs_closedir(uv_loop_t* loop, uv_fs_t* req, uv_dir_t* dir, uv_fs_cb cb);
int uv_fs_stat(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb);
int uv_fs_fstat(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb);
int uv_fs_rename(uv_loop_t* loop, uv_fs_t* req, const char* path, const char* new_path, uv_fs_cb cb);
int uv_fs_fsync(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb);
int uv_fs_fdatasync(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb);
int uv_fs_ftruncate(uv_loop_t* loop, uv_fs_t* req, uv_file file, int64_t offset, uv_fs_cb cb);
int uv_fs_sendfile(uv_loop_t* loop, uv_fs_t* req, uv_file out_fd, uv_file in_fd, int64_t in_offset, size_t length, uv_fs_cb cb);
int uv_fs_access(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode, uv_fs_cb cb);
int uv_fs_chmod(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode, uv_fs_cb cb);
int uv_fs_utime(uv_loop_t* loop, uv_fs_t* req, const char* path, double atime, double mtime, uv_fs_cb cb);
int uv_fs_futime(uv_loop_t* loop, uv_fs_t* req, uv_file file, double atime, double mtime, uv_fs_cb cb);
int uv_fs_lutime(uv_loop_t* loop, uv_fs_t* req, const char* path, double atime, double mtime, uv_fs_cb cb);
int uv_fs_lstat(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb);
int uv_fs_link(uv_loop_t* loop, uv_fs_t* req, const char* path, const char* new_path, uv_fs_cb cb);
int uv_fs_symlink(uv_loop_t* loop, uv_fs_t* req, const char* path, const char* new_path, int flags, uv_fs_cb cb);
int uv_fs_readlink(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)

```

```

;
int uv_fs_realpath(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
;
int uv_fs_fchmod(uv_loop_t* loop, uv_fs_t* req, uv_file file, int mode,
uv_fs_cb cb);
int uv_fs_chown(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_uid_t uid, u
v_gid_t gid, uv_fs_cb cb);
int uv_fs_fchown(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_uid_t uid, uv_g
id_t gid, uv_fs_cb cb);
int uv_fs_lchown(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_uid_t uid,
uv_gid_t gid, uv_fs_cb cb);
int uv_fs_statfs(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb);

```

缓冲区与流

libuv 中基本的 I/O 句柄是流（`uv_stream_t`）。TCP 套接字、TTY 终端，以及用于文件 I/O 和 IPC 的管道，都被视为流的子类。

流先通过各子类自定义的函数初始化，然后使用下列函数进行操作：

```

int uv_read_start(uv_stream_t*, uv_alloc_cb alloc_cb, uv_read_cb read_cb);
int uv_read_stop(uv_stream_t*);
int uv_write(uv_write_t* req, uv_stream_t* handle,
const uv_buf_t bufs[], unsigned int nbufs, uv_write_cb cb);

```

基于流的函数比文件系统函数更易使用：只要调用一次 `uv_read_start()`，libuv 就会自动持续地从流中读取数据，直到调用 `uv_read_stop()` 为止。

数据的离散单元是缓冲区——`uv_buf_t`。它只是一个简单的结构，包含一个指向字节的指针（`uv_buf_t.base`）和长度（`uv_buf_t.len`）。`uv_buf_t` 非常轻量，按值传递。真正需要管理的是实际的字节数据，它们必须由应用程序自己分配和释放。

错误：本程序并不总是能正常工作，需要更好的实现（原文如此）

为了演示流，我们需要使用 `uv_pipe_t`。它允许对流式地读取本地文件²。下面是一个使用 libuv 实现的简单 tee 工具。把所有操作都做成异步的，正好展示了事件驱动 I/O 的威力：两个写入操作不会互相阻塞，但我们必须小心地复制缓冲区数据，以确保在缓冲区写入完成之前不会把它释放掉。

程序的运行方式为：

```
./uvtee <output_file>
```

我们首先在所需的文件上打开管道。libuv 中指向文件的管道默认以双向方式打开。

uvtee/main.c - 管道上的读取

```
int main(int argc, char **argv) {
    loop = uv_default_loop();

    uv_pipe_init(loop, &stdin_pipe, 0);
    uv_pipe_open(&stdin_pipe, 0);

    uv_pipe_init(loop, &stdout_pipe, 0);
    uv_pipe_open(&stdout_pipe, 1);

    uv_fs_t file_req;
    int fd = uv_fs_open(loop, &file_req, argv[1], O_CREAT | O_RDWR, 0644, NULL);
    uv_pipe_init(loop, &file_pipe, 0);
    uv_pipe_open(&file_pipe, fd);

    uv_read_start((uv_stream_t*)&stdin_pipe, alloc_buffer, read_stdin);

    uv_run(loop, UV_RUN_DEFAULT);
    return 0;
}
```

`uv_pipe_init()` 的第三个参数在使用命名管道进行 IPC 时应设为 1，这部分内容在进程一章中介绍。`uv_pipe_open()` 调用将管道与文件描述符关联起来，这里是 0（标准输入）。

我们开始监视 `stdin`。每当需要新的缓冲区来容纳传入数据时，`alloc_buffer` 回调就会被调用。随后 `read_stdin` 会带着这些缓冲区被调用。

uvtee/main.c - 读取缓冲区

```
void alloc_buffer(uv_handle_t *handle, size_t suggested_size, uv_buf_t *buf) {
    *buf = uv_buf_init((char*) malloc(suggested_size), suggested_size);
}
```

```

void read_stdin(uv_stream_t *stream, ssize_t nread, const uv_buf_t *buf) {
    if (nread < 0){
        if (nread == UV_EOF){
            // end of file
            uv_close((uv_handle_t *)&stdin_pipe, NULL);
            uv_close((uv_handle_t *)&stdout_pipe, NULL);
            uv_close((uv_handle_t *)&file_pipe, NULL);
        }
    } else if (nread > 0) {
        write_data((uv_stream_t *)&stdout_pipe, nread, *buf, on_stdout_write);
        write_data((uv_stream_t *)&file_pipe, nread, *buf, on_file_write);
    }

    // OK to free buffer as write_data copies it.
    if (buf->base)
        free(buf->base);
}

```

标准的 `malloc` 在这里就足够了，但你也可以使用任何内存分配方案。例如，node.js 使用自己的 slab 分配器，把缓冲区与 V8 对象关联起来。

读回调的 `nread` 参数在任何出错情况下都小于 0。这个错误可能是 EOF，此时我们使用通用的关闭函数 `uv_close()` 关闭所有流——它会根据句柄的内部类型进行相应处理。否则 `nread` 是一个非负数，我们可以尝试把相应字节数写入输出流。最后别忘了，缓冲区的分配和释放是应用程序的责任，所以我们要释放数据。

分配回调在分配内存失败时可以返回一个长度为零的缓冲区。在这种情况下，读回调会以错误 `UV_ENOBUFS` 被调用。不过 libuv 仍会继续尝试读取该流，因此如果希望在分配失败时停止，你必须显式调用 `uv_close()`。

读回调也可能以 `nread = 0` 被调用，表示此刻没有可读的数据。大多数应用程序会直接忽略这种情况。

uvtee/main.c - 写入管道

```

typedef struct {
    uv_write_t req;
    uv_buf_t buf;
} write_req_t;

```

```

void free_write_req(uv_write_t *req) {
    write_req_t *wr = (write_req_t*) req;
    free(wr->buf.base);
    free(wr);
}

void on_stdout_write(uv_write_t *req, int status) {
    free_write_req(req);
}

void on_file_write(uv_write_t *req, int status) {
    free_write_req(req);
}

void write_data(uv_stream_t *dest, size_t size, uv_buf_t buf, uv_write_cb cb) {
    write_req_t *req = (write_req_t*) malloc(sizeof(write_req_t));
    req->buf = uv_buf_init((char*) malloc(size), size);
    memcpy(req->buf.base, buf.base, size);
    uv_write((uv_write_t*) req, (uv_stream_t*)dest, &req->buf, 1, cb);
}

```

`write_data()` 会复制从读取中得到的缓冲区。这个缓冲区不会传递到写入完成时触发的写回调中。为了解决这个问题，我们把写请求和缓冲区一起包装在 `write_req_t` 中，并在回调中再解包出来。我们制作副本，是为了能把两次调用 `write_data` 所产生的两个缓冲区彼此独立地释放。对于这样的演示程序来说这样做没问题，但在任何正式的应用程序中，你可能都需要更聪明的内存管理方式，比如引用计数的缓冲区或缓冲区池。

警告：如果你的程序要与其他程序配合使用，它可能有意或无意地向管道写入数据。这使其容易因收到 `SIGPIPE` 而终止（参见关于 `SIGPIPE` 的特殊问题）。建议在应用程序的初始化阶段插入：

```
signal(SIGPIPE, SIG_IGN)
```

文件变化事件

所有现代操作系统都提供了针对单个文件或目录设置监视、并在文件被修改时获得通知的 API。libuv 封装了常见的文件变化通知库¹。这是 libuv 中一致性较差的部分之一：文件变化通知机制本身在各平台之间差异极大，因此要让所有功能在所有平台上都正常工

作非常困难。作为演示，我将构建一个简单的工具：当任何被监视的文件发生变化时，就运行一条命令：

```
./onchange <command> <file1> [file2] ...
```

注意：目前这个例子只能在 OSX 和 Windows 上工作。请参阅 `uv_fs_event_start` 函数的说明。

文件变化通知通过 `uv_fs_event_init()` 启动：

onchange/main.c - 初始化设置

```
int main(int argc, char **argv) {
    if (argc <= 2) {
        fprintf(stderr, "Usage: %s <command> <file1> [file2 ...]\n", argv[0]);
        return 1;
    }

    loop = uv_default_loop();
    command = argv[1];

    while (argc-- > 2) {
        fprintf(stderr, "Adding watch on %s\n", argv[argc]);
        uv_fs_event_t *fs_event_req = malloc(sizeof(uv_fs_event_t));
        uv_fs_event_init(loop, fs_event_req);
        // The recursive flag watches subdirectories too.
        uv_fs_event_start(fs_event_req, run_command, argv[argc], UV_FS_EVENT_RECURSIVE);
    }

    return uv_run(loop, UV_RUN_DEFAULT);
}
```

第三个参数是要监视的实际文件或目录。最后一个参数 `flags` 可以是：

```

/*
 * Flags to be passed to uv_fs_event_start().
 */
enum uv_fs_event_flags {
    UV_FS_EVENT_WATCH_ENTRY = 1,
    UV_FS_EVENT_STAT = 2,
    UV_FS_EVENT_RECURSIVE = 4
};

```

`UV_FS_EVENT_WATCH_ENTRY` 和 `UV_FS_EVENT_STAT` 目前什么也不做（尚未实现）。
`UV_FS_EVENT_RECURSIVE` 会在支持的平台上同时监视子目录。

回调会接收以下参数：

1. `uv_fs_event_t *handle` —— 句柄。句柄的 `path` 字段是设置监视的文件。
2. `const char *filename` —— 如果监视的是目录，这是发生变化的文件名。仅在 Linux 和 Windows 上非 `null`；即使在这两个平台上也可能是 `null`。
3. `int events` —— `UV_RENAME` 或 `UV_CHANGE` 之一，或两者的按位或。
4. `int status` —— 如果 `status < 0`，则存在 libuv 错误。

在我们的例子中，我们只是简单地打印这些参数，并使用 `system()` 运行命令。

onchange/main.c - 文件变化通知回调

```

void run_command(uv_fs_event_t *handle, const char *filename, int events, int
status) {
    char path[1024];
    size_t size = 1023;
    // Does not handle error if path is longer than 1023.
    uv_fs_event_getpath(handle, path, &size);
    path[size] = '\0';

    fprintf(stderr, "Change detected in %s: ", path);
    if (events & UV_RENAME)
        fprintf(stderr, "renamed");
    if (events & UV_CHANGE)
        fprintf(stderr, "changed");

    fprintf(stderr, " %s\n", filename ? filename : "");
    system(command);
}

```

1. Linux 上是 inotify, Darwin 上是 FSEvents, BSD 上是 kqueue, Windows 上是 ReadDirectoryChangesW, Solaris 上是 event ports, Cygwin 上不支持 ↩
2. 参见管道一节 ↩

第 5 章 · 网络编程

网络

本章译自 libuv 官方文档 `guide/networking.rst` (libuv 1.52)

libuv 中的网络编程与直接使用 BSD 套接字接口没有太大区别：有些事情变得更简单了，一切都是非阻塞的，但概念保持不变。此外，libuv 还提供了一些实用函数，用来抽象那些繁琐、重复且底层的工作，比如用 BSD 套接字结构设置套接字、DNS 查询，以及调整各种套接字参数。

`uv_tcp_t` 和 `uv_udp_t` 结构体用于网络 I/O。

注意：本章的代码示例只是为了演示某些 libuv API，并不是高质量代码的范例。它们会泄漏内存，也不总是正确地关闭连接。

TCP

TCP 是面向连接的流协议，因此建立在 libuv 的流基础设施之上。

服务器

服务器套接字的流程如下：

1. 用 `uv_tcp_init` 初始化 TCP 句柄。
2. 用 `uv_tcp_bind` 绑定它。
3. 对该句柄调用 `uv_listen`，每当有客户端建立新连接时就会触发一个回调。
4. 用 `uv_accept` 接受连接。
5. 使用流操作与客户端通信。

下面是一个简单的回显（echo）服务器：

tcp-echo-server/main.c - 监听套接字

```

int main() {
    loop = uv_default_loop();

    uv_tcp_t server;
    uv_tcp_init(loop, &server);

    uv_ip4_addr("0.0.0.0", DEFAULT_PORT, &addr);

    uv_tcp_bind(&server, (const struct sockaddr*)&addr, 0);
    int r = uv_listen((uv_stream_t*) &server, DEFAULT_BACKLOG,
on_new_connection);
    if (r) {
        fprintf(stderr, "Listen error %s\n", uv_strerror(r));
        return 1;
    }
    return uv_run(loop, UV_RUN_DEFAULT);
}

```

这里可以看到实用函数 `uv_ip4_addr` 的用法：它把人类可读的 IP 地址和端口对转换为 BSD 套接字 API 所需的 `sockaddr_in` 结构。反向转换可以用 `uv_ip4_name` 完成。

注意：ip4 系列函数都有对应的 `uv_ip6_*` 版本。

大多数初始化设置函数都是同步的，因为它们是 CPU 密集型的。`uv_listen` 是我们回到 libuv 回调风格的地方。它的第二个参数是 backlog 队列——排队连接的最大长度。

当客户端发起连接时，回调需要为客户端套接字创建一个句柄，并用 `uv_accept` 把句柄关联起来。在这个例子中，我们还注册了对该流读取的兴趣。

tcp-echo-server/main.c - 接受客户端

```

void on_new_connection(uv_stream_t *server, int status) {
    if (status < 0) {
        fprintf(stderr, "New connection error %s\n", uv_strerror(status));
        // error!
        return;
    }

    uv_tcp_t *client = (uv_tcp_t*) malloc(sizeof(uv_tcp_t));
    uv_tcp_init(loop, client);
    if (uv_accept(server, (uv_stream_t*) client) == 0) {
        uv_read_start((uv_stream_t*) client, alloc_buffer, echo_read);
    }
    else {
        uv_close((uv_handle_t*) client, on_close);
    }
}

```

其余的函数与流示例非常相似，可以直接在代码中查看。只需记住，当套接字不再需要时调用 `uv_close`。即使你对连接不感兴趣、不想接受它，也可以直接在 `uv_listen` 的回调中完成这一操作。

客户端

服务器端需要 `bind/listen/accept`，而客户端只需调用 `uv_tcp_connect` 即可。
`uv_tcp_connect` 使用与 `uv_listen` 相同的 `uv_connect_cb` 风格回调。试试：

```

uv_tcp_t* socket = (uv_tcp_t*)malloc(sizeof(uv_tcp_t));
uv_tcp_init(loop, socket);

uv_connect_t* connect = (uv_connect_t*)malloc(sizeof(uv_connect_t));

struct sockaddr_in dest;
uv_ip4_addr("127.0.0.1", 80, &dest);

uv_tcp_connect(connect, socket, (const struct sockaddr*)&dest, on_connect);

```

其中 `on_connect` 会在连接建立后被调用。回调接收 `uv_connect_t` 结构体，其成员 `.handle` 指向该套接字。

UDP

用户数据报协议（User Datagram Protocol）提供无连接、不可靠的网络通信。因此 libuv 不为它提供流。libuv 通过 `uv_udp_t` 句柄（用于接收）和 `uv_udp_send_t` 请求（用于发送）以及相关函数提供非阻塞的 UDP 支持。不过，实际的读写 API 与普通的流读取非常相似。为了展示 UDP 的用法，示例展示了从 DHCP 服务器获取 IP 地址的第一阶段——DHCP Discover。

注意：你必须以 **root** 身份运行 `udp-dhcp`，因为它使用了 1024 以下的知名端口号。

udp-dhcp/main.c - 初始化并发送 UDP 数据包

```
int main() {
    loop = uv_default_loop();

    uv_udp_init(loop, &recv_socket);
    struct sockaddr_in recv_addr;
    uv_ip4_addr("0.0.0.0", 68, &recv_addr);
    uv_udp_bind(&recv_socket, (const struct sockaddr *)&recv_addr, UV_UDP_REUSEADDR);
    uv_udp_recv_start(&recv_socket, alloc_buffer, on_read);

    uv_udp_init(loop, &send_socket);
    struct sockaddr_in broadcast_addr;
    uv_ip4_addr("0.0.0.0", 0, &broadcast_addr);
    uv_udp_bind(&send_socket, (const struct sockaddr *)&broadcast_addr, 0);
    uv_udp_set_broadcast(&send_socket, 1);

    uv_udp_send_t send_req;
    uv_buf_t discover_msg = make_discover_msg();

    struct sockaddr_in send_addr;
    uv_ip4_addr("255.255.255.255", 67, &send_addr);
    uv_udp_send(&send_req, &send_socket, &discover_msg, 1, (const struct sockaddr *)&send_addr, on_send);

    return uv_run(loop, UV_RUN_DEFAULT);
}
```

注意：IP 地址 `0.0.0.0` 用于绑定所有网络接口。IP 地址 `255.255.255.255` 是广播地址，意味着数据包会被发送到子网上的所有接口。端口 `0` 表示由操作系统随机分配端口。

首先，我们设置接收套接字，将其绑定到所有接口的 68 端口（DHCP 客户端端口），并在其上启动读取。这样就能读取到任何应答的 DHCP 服务器的响应。我们使用 `UV_UDP_REUSEADDR` 标志，以便与这台计算机上运行在同一端口的其他系统 DHCP 客户端和平共处。然后我们设置一个类似的发送套接字，并使用 `uv_udp_send` 向 67 端口（DHCP 服务器端口）发送一条广播消息。

必须设置广播标志，否则你会得到 `EACCES` 错误¹。所发送的具体消息内容与本书无关，有兴趣可以自行研究代码。和往常一样，如果出了问题，读写回调会收到一个 `< 0` 的状态码。

由于 UDP 套接字不与特定的对端建立连接，读回调会多收到一个参数，用以说明数据包的发送方。

如果没有更多数据可读，`nread` 可能为零。若 `addr` 为 `NULL`，表示没有数据可读（回调不应做任何事情）；若不为 `NULL`，则表示收到了来自 `addr` 主机的一个空数据报。如果分配器提供的缓冲区不足以容纳数据，`flags` 参数可能是 `UV_UDP_PARTIAL`。在这种情况下，操作系统会丢弃装不下的数据（这就是 UDP！）

udp-dhcp/main.c - 读取数据包

```

void on_read(uv_udp_t *req, ssize_t nread, const uv_buf_t *buf, const struct
sockaddr *addr, unsigned flags) {
    if (nread < 0) {
        fprintf(stderr, "Read error %s\n", uv_err_name(nread));
        uv_close((uv_handle_t*) req, NULL);
        free(buf->base);
        return;
    }

    char sender[17] = { 0 };
    uv_ip4_name((const struct sockaddr_in*) addr, sender, 16);
    fprintf(stderr, "Recv from %s\n", sender);

    // ... DHCP specific code
    unsigned int *as_integer = (unsigned int*)buf->base;
    unsigned int ipbin = ntohl(as_integer[4]);
    unsigned char ip[4] = {0};
    int i;
    for (i = 0; i < 4; i++)
        ip[i] = (ipbin >> i*8) & 0xff;
    fprintf(stderr, "Offered IP %d.%d.%d.%d\n", ip[3], ip[2], ip[1], ip[0]);

    free(buf->base);
    uv_udp_recv_stop(req);
}

```

UDP 选项

生存时间 (TTL)

套接字发出数据包的 TTL 可以用 `uv_udp_set_ttl` 修改。

仅 IPv6 协议栈

IPv6 套接字可以同时用于 IPv4 和 IPv6 通信。如果想把套接字限制为仅使用 IPv6，请把 `UV_UDP_IPV6ONLY` 标志传给 `uv_udp_bind`。

组播

套接字可以使用以下函数加入/退出组播组：

```
int uv_udp_set_membership(uv_udp_t* handle, const char* multicast_addr, const
char* interface_addr, uv_membership membership);
```

其中 `membership` 是 `UV_JOIN_GROUP` 或 `UV_LEAVE_GROUP`。

组播的概念在这篇指南中讲解得很好。

组播数据包的本地回环默认是开启的²，可以使用 `uv_udp_set_multicast_loop` 关闭它。

组播数据包的生存时间 (TTL) 可以用 `uv_udp_set_multicast_ttl` 修改。

查询 DNS

libuv 提供异步 DNS 解析。为此，它提供了自己的 `getaddrinfo` 替代实现³。在回调中，你可以对获得的地址执行普通的套接字操作。让我们连接 Libera.chat，看一个 DNS 解析的例子。

dns/main.c

```
int main() {
    loop = uv_default_loop();

    struct addrinfo hints;
    hints.ai_family = PF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_protocol = IPPROTO_TCP;
    hints.ai_flags = 0;

    uv_getaddrinfo_t resolver;
    fprintf(stderr, "irc.libera.chat is... ");
    int r = uv_getaddrinfo(loop, &resolver, on_resolved, "irc.libera.chat", "6667", &hints);

    if (r) {
        fprintf(stderr, "getaddrinfo call error %s\n", uv_err_name(r));
        return 1;
    }
    return uv_run(loop, UV_RUN_DEFAULT);
}
```

如果 `uv_getaddrinfo` 返回非零值，说明设置阶段出了问题，你的回调根本不会被调用。

`uv_getaddrinfo` 返回之后，所有参数都可以立即释放。`hostname`、`servname` 和 `hints` 结构在 `getaddrinfo` 手册页中有说明。回调可以是 `NULL`，此时该函数将同步运行。

在解析回调中，你可以从 `struct addrinfo(s)` 链表中任选一个 IP。这里也演示了 `uv_tcp_connect` 的用法。在回调中必须调用 `uv_freeaddrinfo`。

dns/main.c

```
void on_resolved(uv_getaddrinfo_t *resolver, int status, struct addrinfo *res) {
    if (status < 0) {
        fprintf(stderr, "getaddrinfo callback error %s\n", uv_err_name(status));
        return;
    }

    char addr[17] = {'\0'};
    uv_ip4_name((struct sockaddr_in*) res->ai_addr, addr, 16);
    fprintf(stderr, "%s\n", addr);

    uv_connect_t *connect_req = (uv_connect_t*) malloc(sizeof(uv_connect_t));
    uv_tcp_t *socket = (uv_tcp_t*) malloc(sizeof(uv_tcp_t));
    uv_tcp_init(loop, socket);

    uv_tcp_connect(connect_req, socket, (const struct sockaddr*) res->ai_addr, on_connect);

    uv_freeaddrinfo(res);
}
```

libuv 还提供反向操作 `uv_getnameinfo`。

网络接口

通过 libuv 的 `uv_interface_addresses` 可以获取系统网络接口的信息。下面这个简单的程序只是打印出所有接口的详细信息，让你了解有哪些可用字段。这可用于让服务在启动时绑定到 IP 地址。

interfaces/main.c

```

int main() {
    char buf[512];
    uv_interface_address_t *info;
    int count, i;

    uv_interface_addresses(&info, &count);
    i = count;

    printf("Number of interfaces: %d\n", count);
    while (i--) {
        uv_interface_address_t interface_a = info[i];

        printf("Name: %s\n", interface_a.name);
        printf("Internal? %s\n", interface_a.is_internal ? "Yes" : "No");

        if (interface_a.address.address4.sin_family == AF_INET) {
            uv_ip4_name(&interface_a.address.address4, buf, sizeof(buf));
            printf("IPv4 address: %s\n", buf);
        }
        else if (interface_a.address.address4.sin_family == AF_INET6) {
            uv_ip6_name(&interface_a.address.address6, buf, sizeof(buf));
            printf("IPv6 address: %s\n", buf);
        }

        printf("\n");
    }

    uv_free_interface_addresses(info, count);
    return 0;
}

```

`is_internal` 对回环 (loopback) 接口为 true。注意，如果一个物理接口有多个 IPv4/IPv6 地址，该名称会被报告多次，每个地址报告一次。

1. <https://beej.us/guide/bgnet/html/#broadcast-packetshello-world> ↩
2. <https://tldp.org/HOWTO/Multicast-HOWTO-6.html#ss6.1> ↩
3. libuv 在 libuv 线程池中使用系统的 `getaddrinfo`。libuv v0.8.0 及更早版本还包含 `c-ares` 作为备选方案，但在 v0.9.0 中已移除。 ↩

第 6 章 · 进程

本章译自 libuv 官方文档 `guide/processes.rst` (libuv 1.52)

进程

libuv 提供了相当完善的子进程管理能力，抽象掉了平台之间的差异，并允许通过流 (stream) 或命名管道与子进程通信。

在 Unix 中有一个惯例：让每个进程只做一件事，并把它做好。在这种思路下，一个进程常常借助多个子进程来完成任务（类似于在 shell 中使用管道）。与使用线程和共享内存的模型相比，基于消息传递的多进程模型通常也更容易推理和理解。

针对事件驱动程序，一种常见的质疑是它们无法利用现代计算机的多核。在多线程程序中，内核可以进行调度，把不同的线程分配到不同的核心上，从而提升性能；但事件循环只有一个线程。解决办法是改为启动多个进程，每个进程运行一个事件循环，并被分配到单独的 CPU 核心上。

创建子进程

最简单的情形是你只想启动一个进程，并知道它何时退出。这可以通过 `uv_spawn` 来实现。

```
spawn/main.c
```

```

uv_loop_t *loop;
uv_process_t child_req;
uv_process_options_t options;

int main() {
    loop = uv_default_loop();

    char* args[3];
    args[0] = "mkdir";
    args[1] = "test-dir";
    args[2] = NULL;

    options.exit_cb = on_exit;
    options.file = "mkdir";
    options.args = args;

    int r;
    if ((r = uv_spawn(loop, &child_req, &options))) {
        fprintf(stderr, "%s\n", uv_strerror(r));
        return 1;
    } else {
        fprintf(stderr, "Launched process with ID %d\n", child_req.pid);
    }

    return uv_run(loop, UV_RUN_DEFAULT);
}

```

注意：由于 `options` 是全局变量，它会被隐式地初始化为零值。如果你把 `options` 改成局部变量，请记得将它初始化，把所有未使用的字段都清空：

```
c uv_process_options_t options = {0};
```

`uv_process_t` 结构体只充当句柄，所有选项都通过 `uv_process_options_t` 来设置。若只是想简单地启动一个进程，你只需要设置 `file` 和 `args` 字段。`file` 是要执行的程序。由于 `uv_spawn` 内部使用 `execvp(3)`，因此无需提供完整路径。最后，按照底层约定，**参数数组必须比参数个数多一，且最后一个元素为 NULL。**

调用 `uv_spawn` 之后，`uv_process_t.pid` 中会保存子进程的进程 ID。

退出回调会带着退出状态（exit status）以及导致退出的信号类型被调用。

注意，**不要在退出回调被调用之前调用 `uv_close`**，这一点很重要。

spawn/main.c

```
void on_exit(uv_process_t *req, int64_t exit_status, int term_signal) {
    fprintf(stderr, "Process exited with status %" PRIu64 " ", signal %d\n", exit_
status, term_signal);
    uv_close((uv_handle_t*) req, NULL);
}
```

进程退出之后，**必须**关闭这个进程监视器。

修改进程参数

在子进程启动之前，你可以利用 `uv_process_options_t` 中的字段来控制它的执行环境。

修改执行目录

把 `uv_process_options_t.cwd` 设置为相应的目录即可。

设置环境变量

`uv_process_options_t.env` 是一个以 `NULL` 结尾的字符串数组，其中每个字符串形如 `VAR=VALUE`，用于为进程设置环境变量。将其设为 `NULL` 则继承父（当前）进程的环境变量。

选项标志

把 `uv_process_options_t.flags` 设置为下列标志的按位或，可以改变子进程的行为：

- `UV_PROCESS_SETUID` — 将子进程的执行用户 ID 设置为 `uv_process_options_t.uid`。
- `UV_PROCESS_SETGID` — 将子进程的执行组 ID 设置为 `uv_process_options_t.gid`。

修改 `UID/GID` 只在 `Unix` 上受支持，在 `Windows` 上 `uv_spawn` 会失败并返回 `UV_ENOTSUP`。

- `UV_PROCESS_WINDOWS_VERBATIM_ARGUMENTS` — 在 `Windows` 上不对 `uv_process_options_t.args` 做任何引号包裹或转义处理。在 `Unix` 上被忽略。
- `UV_PROCESS_DETACHED` — 在新的会话（session）中启动子进程，这样即使父进程退出，子进程也会继续运行。参见下文示例。

分离进程

传入 `UV_PROCESS_DETACHED` 标志可以用来启动守护进程，或者启动独立于父进程的子进程，使父进程的退出不影响它们。

detach/main.c

```
int main() {
    loop = uv_default_loop();

    char* args[3];
    args[0] = "sleep";
    args[1] = "100";
    args[2] = NULL;

    options.exit_cb = NULL;
    options.file = "sleep";
    options.args = args;
    options.flags = UV_PROCESS_DETACHED;

    int r;
    if ((r = uv_spawn(loop, &child_req, &options))) {
        fprintf(stderr, "%s\n", uv_strerror(r));
        return 1;
    }
    fprintf(stderr, "Launched sleep with PID %d\n", child_req.pid);
    uv_unref((uv_handle_t*) &child_req);

    return uv_run(loop, UV_RUN_DEFAULT);
}
```

请记住，句柄仍然在监视着子进程，所以你的程序不会退出。如果你想要更发射后不管（fire-and-forget）的效果，可以使用 `uv_unref()`。

向进程发送信号

libuv 封装了 Unix 上的标准 `kill(2)` 系统调用，并在 Windows 上实现了语义类似的版本，但有一个**注意点**：`SIGTERM`、`SIGINT` 和 `SIGKILL` 都会导致进程被终止。`uv_kill` 的签名为：

```
uv_err_t uv_kill(int pid, int signum);
```

对于用 libuv 启动的进程，你可以改用 `uv_process_kill`，它接受 `uv_process_t` 监视器（而不是 `pid`）作为第一个参数。在这种情况下，**记得要在退出回调被调用之后对监视器调用 `uv_close`**。

信号

libuv 对 Unix 信号做了封装，同时也提供部分 Windows 支持。

使用 `uv_signal_init()` 初始化一个句柄，并将它与一个事件循环关联起来。要在该句柄上监听特定的信号，可使用 `uv_signal_start()` 并传入处理函数。每个处理函数只能关联一个信号编号，后续对 `uv_signal_start()` 的调用会覆盖之前的关联。使用 `uv_signal_stop()` 停止监听。下面是一个演示各种用法的小例子：

```
signal/main.c
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <uv.h>

uv_loop_t* create_loop()
{
    uv_loop_t *loop = malloc(sizeof(uv_loop_t));
    if (loop) {
        uv_loop_init(loop);
    }
    return loop;
}

void signal_handler(uv_signal_t *handle, int signum)
{
    printf("Signal received: %d\n", signum);
    uv_signal_stop(handle);
}

// two signal handlers in one loop
void thread1_worker(void *userp)
{
    uv_loop_t *loop1 = create_loop();

    uv_signal_t sig1a, sig1b;
    uv_signal_init(loop1, &sig1a);
    uv_signal_start(&sig1a, signal_handler, SIGUSR1);

    uv_signal_init(loop1, &sig1b);
    uv_signal_start(&sig1b, signal_handler, SIGUSR1);

    uv_run(loop1, UV_RUN_DEFAULT);
}

// two signal handlers, each in its own loop
void thread2_worker(void *userp)
{
    uv_loop_t *loop2 = create_loop();
    uv_loop_t *loop3 = create_loop();

    uv_signal_t sig2;
    uv_signal_init(loop2, &sig2);
    uv_signal_start(&sig2, signal_handler, SIGUSR1);

    uv_signal_t sig3;
    uv_signal_init(loop3, &sig3);

```

```

    uv_signal_start(&sig3, signal_handler, SIGUSR1);

    while (uv_run(loop2, UV_RUN_NOWAIT) || uv_run(loop3, UV_RUN_NOWAIT)) {
    }

}

int main()
{
    printf("PID %d\n", getpid());

    uv_thread_t thread1, thread2;

    uv_thread_create(&thread1, thread1_worker, 0);
    uv_thread_create(&thread2, thread2_worker, 0);

    uv_thread_join(&thread1);
    uv_thread_join(&thread2);
    return 0;
}

```

注意： `uv_run(loop, UV_RUN_NOWAIT)` 与 `uv_run(loop, UV_RUN_ONCE)` 的相似之处在于二者都只处理一个事件。区别在于：没有待处理事件时，`UV_RUN_ONCE` 会阻塞，而 `UV_RUN_NOWAIT` 会立即返回。我们使用 `NOWAIT`，是为了避免其中一个循环因为另一个循环没有待处理的活动而得不到运行机会（被“饿死”）。

给该进程发送 `SIGUSR1`，你会发现处理函数被调用了 4 次——每个 `uv_signal_t` 各一次。处理函数只是把每个句柄停掉，程序随之退出。这种向所有处理函数广播式的分发非常有用：一个使用多个事件循环的服务器只需让每个循环都为 `SIGINT` 添加一个监视器，就能确保在终止之前所有数据都被安全地保存。

子进程 I/O

一个普通的新建进程拥有自己的一组文件描述符，其中 0、1、2 分别是 `stdin`、`stdout` 和 `stderr`。有时你可能想与子进程共享文件描述符。举例来说，也许你的应用会启动一个子命令，你希望错误信息写入日志文件，同时忽略 `stdout`。这时你希望子进程的 `stderr` 与父进程的 `stderr` 相同。为此，libuv 支持继承文件描述符。在本例中，我们调用这个测试程序：

```
proc-streams/test.c
```

```
#include <stdio.h>

int main()
{
    fprintf(stderr, "This is stderr\n");
    printf("This is stdout\n");
    return 0;
}
```

实际运行的程序 `proc-streams` 在执行它的同时只共享 `stderr`。子进程的文件描述符通过 `uv_process_options_t` 中的 `stdio` 字段来设置。首先把 `stdio_count` 字段设置为要设置的文件描述符的数量。`uv_process_options_t.stdio` 是一个 `uv_stdio_container_t` 数组，该结构定义为：

```
typedef struct uv_stdio_container_s {
    uv_stdio_flags flags;

    union {
        uv_stream_t* stream;
        int fd;
    } data;
} uv_stdio_container_t;
```

其中 `flags` 可以取若干个值。如果某个描述符不会被用到，就使用 `UV_IGNORE`。如果前三个 `stdio` 项都被标记为 `UV_IGNORE`，它们会被重定向到 `/dev/null`。

由于我们想传入一个已有的描述符，所以使用 `UV_INHERIT_FD`，然后把 `fd` 设置为 `stderr`。

`proc-streams/main.c`

```

int main() {
    loop = uv_default_loop();

    /* ... */

    options.stdio_count = 3;
    uv_stdio_container_t child_stdio[3];
    child_stdio[0].flags = UV_IGNORE;
    child_stdio[1].flags = UV_IGNORE;
    child_stdio[2].flags = UV_INHERIT_FD;
    child_stdio[2].data.fd = 2;
    options.stdio = child_stdio;

    options.exit_cb = on_exit;
    options.file = args[0];
    options.args = args;

    int r;
    if ((r = uv_spawn(loop, &child_req, &options))) {
        fprintf(stderr, "%s\n", uv_strerror(r));
        return 1;
    }

    return uv_run(loop, UV_RUN_DEFAULT);
}

```

运行 `proc-stream`，你会看到只有 "This is stderr" 这一行被显示出来。试着把 `stdout` 也标记为继承，再观察输出。

把这种重定向应用到流（stream）上也非常简单。将 `flags` 设为 `UV_INHERIT_STREAM`，并把 `data.stream` 设置为父进程中的某个流，子进程就可以把这个流当作标准 I/O 来使用。这可以用来实现类似 CGI 的功能。

CGI 脚本/可执行程序的一个示例如下：

`cgi/tick.c`

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int i;
    for (i = 0; i < 10; i++) {
        printf("tick\n");
        fflush(stdout);
        sleep(1);
    }
    printf("BOOM!\n");
    return 0;
}
```

这个 CGI 服务器把本章的概念与网络编程一章的概念结合起来：每个客户端会被发送十个 tick，之后连接被关闭。

cgi/main.c

```
void on_new_connection(uv_stream_t *server, int status) {
    if (status == -1) {
        // error!
        return;
    }

    uv_tcp_t *client = (uv_tcp_t*) malloc(sizeof(uv_tcp_t));
    uv_tcp_init(loop, client);
    if (uv_accept(server, (uv_stream_t*) client) == 0) {
        invoke_cgi_script(client);
    }
    else {
        uv_close((uv_handle_t*) client, NULL);
    }
}
```

这里我们只是简单地接受 TCP 连接，并把套接字（流）传递给 `invoke_cgi_script`。

cgi/main.c

```

args[1] = NULL;

/* ... finding the executable path and setting up arguments ... */

options.stdio_count = 3;
uv_stdio_container_t child_stdio[3];
child_stdio[0].flags = UV_IGNORE;
child_stdio[1].flags = UV_INHERIT_STREAM;
child_stdio[1].data.stream = (uv_stream_t*) client;
child_stdio[2].flags = UV_IGNORE;
options.stdio = child_stdio;

options.exit_cb = cleanup_handles;
options.file = args[0];
options.args = args;

// Set this so we can close the socket after the child process exits.
child_req.data = (void*) client;
int r;
if ((r = uv_spawn(loop, &child_req, &options))) {
    fprintf(stderr, "%s\n", uv_strerror(r));
}

```

CGI 脚本的 `stdout` 被设置为该套接字，这样 tick 脚本打印的任何内容都会被发送给客户端。借助进程，我们可以把读写的缓冲工作交给操作系统来完成，就便利性而言非常出色。只是要当心：创建进程是一笔开销不小的操作。

父子进程 IPC

通过把 `uv_stdio_container_t.flags` 设置为 `UV_CREATE_PIPE` 与 `UV_READABLE_PIPE` 或 `UV_WRITABLE_PIPE` 的按位组合，父子进程可以在一条管道上进行单向或双向通信。读/写标志是从子进程的视角来看的。在这种情况下，`uv_stream_t* stream` 字段必须指向一个已初始化、尚未打开的 `uv_pipe_t` 实例。

新建 stdio 管道

`uv_pipe_t` 结构所代表的不只是 `pipe(7)`（也就是 `|`），它支持任何流式的、类文件的对象。在 Windows 上，符合这一描述的对象只有命名管道（Named Pipe）。在 Unix 上，它可以是 Unix 域套接字，可以派生自 `mkfifo(1)`，也可以真的就是一条 `pipe(7)` 管道。当 `uv_spawn` 因为 `UV_CREATE_PIPE` 标志而初始化一个 `uv_pipe_t` 时，它会选择创建一个 `socketpair(2)`。

这样做的目的是让多个 libuv 进程能够通过 IPC 相互通信，下文会讨论这一点。

任意进程间的 IPC

由于域套接字（domain socket，在 Windows 上这一节的内容同样适用于命名管道）^[^*] 可以拥有一个众所周知的名字和文件系统中的位置，它们可以用于不相关的进程之间的 IPC。开源桌面环境所使用的 D-Bus 系统就用域套接字来做事件通知：当联系人上线或检测到新硬件时，各种应用程序都可以做出响应。MySQL 服务器同样运行着一个域套接字，客户端可以通过它与服务器交互。

使用域套接字时，通常遵循客户端-服务器模式，套接字的创建者/拥有者充当服务器。在完成初始设置之后，消息传递与 TCP 没有任何区别，因此我们将复用回显服务器的例子。

pipe-echo-server/main.c

```
void remove_sock(int sig) {
    uv_fs_t req;
    uv_fs_unlink(loop, &req, PIPENAME, NULL);
    exit(0);
}

int main() {
    loop = uv_default_loop();

    uv_pipe_t server;
    uv_pipe_init(loop, &server, 0);

    signal(SIGINT, remove_sock);

    int r;
    if ((r = uv_pipe_bind(&server, PIPENAME))) {
        fprintf(stderr, "Bind error %s\n", uv_err_name(r));
        return 1;
    }
    if ((r = uv_listen((uv_stream_t*) &server, 128, on_new_connection))) {
        fprintf(stderr, "Listen error %s\n", uv_err_name(r));
        return 2;
    }
    return uv_run(loop, UV_RUN_DEFAULT);
}
```

我们把套接字命名为 `echo.sock`，也就是说它会被创建在当前目录下。就流 API 而言，这个套接字的行为与 TCP 套接字并无二致。你可以用 `socat` 来测试这个服务器：

```
$ socat - /path/to/socket
```

想要连接域套接字的客户端会使用：

```
void uv_pipe_connect(uv_connect_t *req, uv_pipe_t *handle, const char *name,
uv_connect_cb cb);
```

其中 `name` 就是 `echo.sock` 之类的名字。在 Unix 系统上，`name` 必须指向一个有效的文件（例如 `/tmp/echo.sock`）；在 Windows 上，`name` 遵循 `\\?\\pipe\\echo.sock` 这样的格式。

通过管道发送文件描述符

域套接字的一个妙处在于：文件描述符可以通过域套接字在进程之间传递。这使得进程可以把 I/O 移交给其他进程。应用场景包括负载均衡服务器、工作进程（worker process），以及其他充分利用 CPU 的方式。目前 libuv 只支持通过管道发送 **TCP 套接字或其他管道**。

为了演示，我们来看一个把客户端以轮询（round-robin）方式移交给工作进程的回显服务器实现。这个程序稍微复杂一些，虽然本书只收录了片段，但建议通读完整代码以真正理解它。

工作进程相当简单，因为文件描述符是由主进程（master）移交给它的。

`multi-echo-server/worker.c`

```
uv_loop_t *loop;
uv_pipe_t queue;
int main() {
    loop = uv_default_loop();

    uv_pipe_init(loop, &queue, 1 /* ipc */);
    uv_pipe_open(&queue, 0);
    uv_read_start((uv_stream_t*)&queue, alloc_buffer, on_new_connection);
    return uv_run(loop, UV_RUN_DEFAULT);
}
```

`queue` 是连接到主进程的那条管道，新的文件描述符沿着它被发送过来。重要的是，要把 `uv_pipe_init` 的 `ipc` 参数设为 1，以表明这条管道将用于进程间通信！由于主进程会把文件句柄写入工作进程的标准输入，我们用 `uv_pipe_open` 把管道连接到 `stdin`。

`multi-echo-server/worker.c`

```
void on_new_connection(uv_stream_t *q, ssize_t nread, const uv_buf_t *buf) {
    if (nread < 0) {
        if (nread != UV_EOF)
            fprintf(stderr, "Read error %s\n", uv_err_name(nread));
        uv_close((uv_handle_t*) q, NULL);
        return;
    }

    uv_pipe_t *pipe = (uv_pipe_t*) q;
    if (!uv_pipe_pending_count(pipe)) {
        fprintf(stderr, "No pending count\n");
        return;
    }

    uv_handle_type pending = uv_pipe_pending_type(pipe);
    assert(pending == UV_TCP);

    uv_tcp_t *client = (uv_tcp_t*) malloc(sizeof(uv_tcp_t));
    uv_tcp_init(loop, client);
    if (uv_accept(q, (uv_stream_t*) client) == 0) {
        uv_os_fd_t fd;
        uv_fileno((const uv_handle_t*) client, &fd);
        fprintf(stderr, "Worker %d: Accepted fd %d\n", getpid(), fd);
        uv_read_start((uv_stream_t*) client, alloc_buffer, echo_read);
    }
    else {
        uv_close((uv_handle_t*) client, NULL);
    }
}
```

我们先调用 `uv_pipe_pending_count()`，确保确实有句柄可以读出。如果你的程序可能处理多种类型的句柄，可以用 `uv_pipe_pending_type()` 来判断类型。虽然 `accept` 出现在这段代码里看起来有些奇怪，但实际上是说得通的：`accept` 传统上做的事情就是从另一个文件描述符（监听套接字）那里取得一个文件描述符（客户端），而这正是我们在这里所做的——从 `queue` 中取出文件描述符（`client`）。从这一刻起，工作进程做的就是标准的回显服务器那套事情。

接下来看主进程，了解一下工作进程是如何被启动以实现负载均衡的。

multi-echo-server/main.c

```
struct child_worker {  
    uv_process_t req;  
    uv_process_options_t options;  
    uv_pipe_t pipe;  
} *workers;
```

`child_worker` 结构体把进程本身以及主进程与各个进程之间的管道包装在一起。

multi-echo-server/main.c

```

void setup_workers() {
    round_robin_counter = 0;

    // ...

    // launch same number of workers as number of CPUs
    uv_cpu_info_t *info;
    int cpu_count;
    uv_cpu_info(&info, &cpu_count);
    uv_free_cpu_info(info, cpu_count);

    child_worker_count = cpu_count;

    workers = calloc(cpu_count, sizeof(struct child_worker));
    while (cpu_count--) {
        struct child_worker *worker = &workers[cpu_count];
        uv_pipe_init(loop, &worker->pipe, 1);

        uv_stdio_container_t child_stdio[3];
        child_stdio[0].flags = UV_CREATE_PIPE | UV_READABLE_PIPE;
        child_stdio[0].data.stream = (uv_stream_t*) &worker->pipe;
        child_stdio[1].flags = UV_IGNORE;
        child_stdio[2].flags = UV_INHERIT_FD;
        child_stdio[2].data.fd = 2;

        worker->options.stdio = child_stdio;
        worker->options.stdio_count = 3;

        worker->options.exit_cb = close_process_handle;
        worker->options.file = args[0];
        worker->options.args = args;

        uv_spawn(loop, &worker->req, &worker->options);
        fprintf(stderr, "Started worker %d\n", worker->req.pid);
    }
}

```

在设置工作进程时，我们用到了 libuv 一个很方便的函数 `uv_cpu_info` 来获取 CPU 数量，从而启动数量相等的工作进程。同样重要的是，要把充当 IPC 通道的管道的第三个参数初始化为 1。然后我们声明子进程的 `stdin` 将是一条可读管道（从子进程的视角看）。到这里为止一切都直截了当：工作进程被启动，等待文件描述符写入它们的标准输入。

真正接收客户端套接字并把它传递给轮询中下一个工作进程的地方是 `on_new_connection` (TCP 基础设施在 `main()` 中初始化)。

multi-echo-server/main.c

```
void on_new_connection(uv_stream_t *server, int status) {
    if (status == -1) {
        // error!
        return;
    }

    uv_tcp_t *client = (uv_tcp_t*) malloc(sizeof(uv_tcp_t));
    uv_tcp_init(loop, client);
    if (uv_accept(server, (uv_stream_t*) client) == 0) {
        uv_write_t *write_req = (uv_write_t*) malloc(sizeof(uv_write_t));
        dummy_buf = uv_buf_init("a", 1);
        struct child_worker *worker = &workers[round_robin_counter];
        uv_write2(write_req, (uv_stream_t*) &worker->pipe, &dummy_buf, 1, (uv_stream_t*) client, NULL);
        round_robin_counter = (round_robin_counter + 1) % child_worker_count;
    }
    else {
        uv_close((uv_handle_t*) client, NULL);
    }
}
```

`uv_write2` 调用处理了所有的抽象细节，我们只需把句柄 (`client`) 作为相应参数传入即可。至此，我们的多进程回显服务器就可以运转了。

感谢 Kyle 指出 `uv_write2()` 在发送句柄时也需要一个非空的缓冲区。

1. 在这一节中，域套接字在 Windows 上同样代指命名管道。↩

第 7 章 · 线程与同步原语

本章译自 libuv 官方文档 `guide/threads.rst` (libuv 1.52)

线程与同步原语

等等，我们为什么突然讲起线程了？事件循环不是号称做 web 规模编程 的 **正道** 吗？
呃……并不全是。处理器终究是在线程中执行工作的，因此线程有时依然大有用处——
尽管你可能得趟过各种同步原语的泥潭。

libuv 在内部使用线程来模拟所有系统调用的异步特性。libuv 也借助线程让你（应用程序）能够异步地执行一个实际上是阻塞的任务：派生一个线程，并在任务完成时收集结果。

如今主流的线程库有两个：Windows 线程实现和 POSIX 的 `pthread(7)`。libuv 的线程 API 与 `pthread` API 类似，语义上也常常相近。

libuv 线程设施有一个值得注意的特点：它是 libuv 中一个完全自包含的部分。libuv 的其他功能都与事件循环和回调机制紧密耦合，而线程对这一切完全无感——它们可以按需阻塞，通过返回值直接报告错误，而且正如下面第一个例子所示，甚至不需要一个正在运行的事件循环。

由于各个平台上线程的语义和语法都各不相同、完善程度不一，libuv 的线程 API 也相当精简。

本章基于如下假设：**只有一个事件循环，它运行在一个线程（主线程）中**。其他任何线程都不与事件循环交互（除非通过 `uv_async_send`）。

核心线程操作

这里没什么复杂的东西：用 `uv_thread_create()` 启动一个线程，用 `uv_thread_join()` 等待它结束即可。

```
thread-create/main.c
```

```

int tracklen = 10;
uv_thread_t hare_id;
uv_thread_t tortoise_id;
uv_thread_create(&hare_id, hare, &tracklen);
uv_thread_create(&tortoise_id, tortoise, &tracklen);

uv_thread_join(&hare_id);
uv_thread_join(&tortoise_id);
return 0;
}

```

提示： `uv_thread_t` 在 Unix 上只是 `pthread_t` 的别名，但这属于实现细节，不要依赖它永远成立。

第二个参数是充当线程入口点的函数，最后一个参数是一个 `void *`，可以用来向线程传递自定义参数。现在函数 `hare` 将在一个单独的线程中运行，由操作系统抢占式地调度：

thread-create/main.c

```

int tracklen = *((int *) arg);
while (tracklen) {
    tracklen--;
    uv_sleep(1000);
    fprintf(stderr, "Hare ran another step\n");
}
fprintf(stderr, "Hare done running!\n");
}

```

`pthread_join()` 允许目标线程通过第二个参数向调用线程传回一个值，而 `uv_thread_join()` 不支持这一点。要传递值，请使用[线程间通信](#)。

同步原语

这一节刻意写得很简略。本书的主题并不是线程，所以我只把 libuv API 中出人意料之处罗列于此，其余内容请查阅 `pthreads(7)` 手册页。

互斥锁

互斥锁函数到 `pthread` 对应函数是一一对应的直接映射。

libuv mutex functions

```
int uv_mutex_init(uv_mutex_t* handle);
int uv_mutex_init_recursive(uv_mutex_t* handle);
void uv_mutex_destroy(uv_mutex_t* handle);
void uv_mutex_lock(uv_mutex_t* handle);
int uv_mutex_trylock(uv_mutex_t* handle);
void uv_mutex_unlock(uv_mutex_t* handle);
```

`uv_mutex_init()`、`uv_mutex_init_recursive()` 和 `uv_mutex_trylock()` 成功时返回 0，否则返回错误码。

如果 `libuv` 编译时启用了调试，`uv_mutex_destroy()`、`uv_mutex_lock()` 和 `uv_mutex_unlock()` 出错时会调用 `abort()`。类似地，当错误不是 `EAGAIN` 或 `EBUSY` 时，`uv_mutex_trylock()` 也会 `abort`。

`libuv` 支持递归互斥锁，但你不应依赖它，而且递归互斥锁不应与 `uv_cond_t` 变量一起使用。

默认的 BSD 互斥锁实现中，如果一个已持有互斥锁的线程再次尝试加锁，就会报错。例如，下面这样的结构：

```
uv_mutex_init(a_mutex);
uv_mutex_lock(a_mutex);
uv_thread_create(thread_id, entry, (void *)a_mutex);
uv_mutex_lock(a_mutex);
// more things here
```

可以用来等待另一个线程完成某些初始化后再解锁 `a_mutex`；但如果程序运行在调试模式下，这会导致程序崩溃，或者在第二次调用 `uv_mutex_lock()` 时返回错误。

注意：Windows 上的互斥锁总是递归的。

读写锁

读写锁是一种更细粒度的访问控制机制：两个读线程可以同时访问共享内存；当锁被某个读者持有时，写者不能获取锁；当写者持有锁时，读者和写者都不能获取锁。读写锁在数据库中被广泛使用。下面是一个玩具示例。

```
locks/main.c - simple rwlocks
```

```

#include <stdio.h>
#include <uv.h>

uv_barrier_t blocker;
uv_rwlock_t numlock;
int shared_num;

void reader(void *n)
{
    int num = *(int *)n;
    int i;
    for (i = 0; i < 20; i++) {
        uv_rwlock_rdlock(&numlock);
        printf("Reader %d: acquired lock\n", num);
        printf("Reader %d: shared num = %d\n", num, shared_num);
        uv_rwlock_rdunlock(&numlock);
        printf("Reader %d: released lock\n", num);
    }
    uv_barrier_wait(&blocker);
}

void writer(void *n)
{
    int num = *(int *)n;
    int i;
    for (i = 0; i < 20; i++) {
        uv_rwlock_wrlock(&numlock);
        printf("Writer %d: acquired lock\n", num);
        shared_num++;
        printf("Writer %d: incremented shared num = %d\n", num, shared_num);
        uv_rwlock_wrunlock(&numlock);
        printf("Writer %d: released lock\n", num);
    }
    uv_barrier_wait(&blocker);
}

int main()
{
    uv_barrier_init(&blocker, 4);

    shared_num = 0;
    uv_rwlock_init(&numlock);

    uv_thread_t threads[3];

    int thread_nums[] = {1, 2, 1};
    uv_thread_create(&threads[0], reader, &thread_nums[0]);

```

```

    uv_thread_create(&threads[1], reader, &thread_nums[1]);

    uv_thread_create(&threads[2], writer, &thread_nums[2]);

    uv_barrier_wait(&blocker);
    uv_barrier_destroy(&blocker);

    uv_rwlock_destroy(&numlock);
    return 0;
}

```

运行它，你会观察到读者有时会重叠执行。在有多多个写者的情况下，调度器通常会给写者更高的优先级，所以如果你加入两个写者，会看到两个写者往往都先完成，读者才重新有机会运行。

在上面的例子中我们还使用了屏障（barrier），让主线程能够等待所有读者和写者表明自己已经结束。

其他

libuv 还支持信号量、条件变量和屏障，它们的 API 与 pthread 中的对应物非常相似。

此外，libuv 提供了一个便利函数 `uv_once()`。多个线程可以带着同一个 guard（守卫对象）和一个函数指针去调用 `uv_once()`，只有第一个会胜出，函数会被调用一次且仅一次：

```

/* Initialize guard */
static uv_once_t once_only = UV_ONCE_INIT;

int i = 0;

void increment() {
    i++;
}

void thread1() {
    /* ... work */
    uv_once(once_only, increment);
}

void thread2() {
    /* ... work */
    uv_once(once_only, increment);
}

int main() {
    /* ... spawn threads */
}

```

所有线程结束后，`i == 1`。

自 libuv v0.11.11 起，还新增了 `uv_key_t` 结构和相应的 API，用于线程局部存储（thread-local storage）。

libuv 工作队列

`uv_queue_work()` 是一个便利函数，它允许应用程序在单独的线程中运行一个任务，并在任务完成时触发一个回调。这个函数看似简单，它的诱人之处在于：它让几乎所有第三方库都有可能与事件循环范式配合使用。使用事件循环时，务必确保在循环线程中周期性运行的函数不会在执行 I/O 时阻塞，也不是严重的 CPU 消耗大户，否则事件循环的速度就会被拖慢，事件无法被全速处理。

然而，现存的很多代码都包含阻塞式函数（例如底层执行 I/O 的例程），想要保持响应性就得配合线程使用（经典的“每个客户端一个线程”服务器模型），而要让它们与事件循环库协同工作，通常需要自己动手搭一套在独立线程中运行任务的系统。libuv 正是为这一点提供了一个方便的抽象。

下面是一个受 `node.js is cancer` 启发的简单例子。我们要计算斐波那契数，中途还会睡上片刻，但把它放到单独的线程里运行，这样这个阻塞且吃 CPU 的任务就不会妨碍事件循环做其他事情。

queue-work/main.c - lazy fibonacci

```
void fib(uv_work_t *req) {
    int n = *(int *) req->data;
    if (random() % 2)
        sleep(1);
    else
        sleep(3);
    long fib = fib_(n);
    fprintf(stderr, "%dth fibonacci is %lu\n", n, fib);
}

void after_fib(uv_work_t *req, int status) {
    fprintf(stderr, "Done calculating %dth fibonacci\n", *(int *) req->data);
}
```

实际的任务函数很简单，看不出它将要在单独的线程中运行。线索在于 `uv_work_t` 结构：你可以通过它的 `void* data` 字段传递任意数据，并用它与线程双向通信。但请注意，如果两个线程可能同时在运行并且都在修改数据，一定要使用恰当的锁。

触发点是 `uv_queue_work`：

queue-work/main.c

```
int main() {
    loop = uv_default_loop();

    int data[FIB_UNTIL];
    uv_work_t req[FIB_UNTIL];
    int i;
    for (i = 0; i < FIB_UNTIL; i++) {
        data[i] = i;
        req[i].data = (void *) &data[i];
        uv_queue_work(loop, &req[i], fib, after_fib);
    }

    return uv_run(loop, UV_RUN_DEFAULT);
}
```

线程函数会在单独的线程中被启动，并传入 `uv_work_t` 结构；一旦函数返回，`after` 函数就会在事件循环所在线程中被调用，传入的也是同一个结构。

为阻塞式库编写封装时，一种常见的模式是使用 `baton`（接力棒）来交换数据（参见“通用模式”一章中的 `baton` 一节）。

自 `libuv 0.9.4` 版本起，多了一个 `uv_cancel()` 函数，它允许你取消 `libuv` 工作队列中的任务。只有尚未开始的任务才能被取消；如果任务已经开始执行，或者已经执行完毕，`uv_cancel()` **会失败**。

当用户请求终止程序时，`uv_cancel()` 很适合用来清理尚待执行的任务。例如，一个音乐播放器可能把多个目录排入队列以便扫描音频文件。如果用户要终止程序，程序应当尽快退出，而不是等所有待处理的请求都执行完。

让我们修改斐波那契的例子来演示 `uv_cancel()`。我们先为终止信号设置一个信号处理器。

queue-cancel/main.c

```
int main() {
    loop = uv_default_loop();

    int data[FIB_UNTIL];
    int i;
    for (i = 0; i < FIB_UNTIL; i++) {
        data[i] = i;
        fib_reqs[i].data = (void *) &data[i];
        uv_queue_work(loop, &fib_reqs[i], fib, after_fib);
    }

    uv_signal_t sig;
    uv_signal_init(loop, &sig);
    uv_signal_start(&sig, signal_handler, SIGINT);

    return uv_run(loop, UV_RUN_DEFAULT);
}
```

当用户按下 `Ctrl+C` 触发信号时，我们对所有工作线程调用 `uv_cancel()`。对于尚在排队等待执行、可以取消的任务，`uv_cancel()` 会返回 `0`。

queue-cancel/main.c

```
void signal_handler(uv_signal_t *req, int signum)
{
    printf("Signal received!\n");
    int i;
    for (i = 0; i < FIB_UNTIL; i++) {
        uv_cancel((uv_req_t*) &fib_reqs[i]);
    }
    uv_signal_stop(req);
}
```

对于被成功取消的任务，after 函数被调用时 `status` 被设置为 `UV_ECANCELED`。

queue-cancel/main.c

```
void after_fib(uv_work_t *req, int status) {
    if (status == UV_ECANCELED)
        fprintf(stderr, "Calculation of %d cancelled.\n", *(int *) req->data);
}
```

`uv_cancel()` 也可以用于 `uv_fs_t` 和 `uv_getaddrinfo_t` 请求。对文件系统这一族函数来说，`uv_fs_t.errno` 会被设置为 `UV_ECANCELED`。

提示：一个设计良好的程序，应当有办法终止那些已经开始执行的长时运行的工作线程。这样的工作线程可以周期性地检查某个只有主进程才会设置的变量，以感知终止信号。

线程间通信

有时候，你会希望各个线程在运行的同时真的互相发送消息。比如，你可能在单独的线程里运行一个耗时的任务（也许就是通过 `uv_queue_work`），但想向主线程汇报进度。下面这个简单的例子演示了一个下载管理器如何向用户告知正在进行的下载的状态。

progress/main.c

```

uv_loop_t *loop;
}

void print_progress(uv_async_t *handle) {
    double pct = atomic_load_explicit(&percentage, memory_order_acquire);
    fprintf(stderr, "Downloaded %.2f%%\n", pct);
}

int main() {
    loop = uv_default_loop();

    uv_work_t req;
    int size = 10240;
    req.data = (void*) &size;

    uv_async_init(loop, &async, print_progress);
    uv_queue_work(loop, &req, fake_download, after);
    return uv_run(loop, UV_RUN_DEFAULT);
}

```

这种异步线程通信是作用在事件循环上的：任何线程都可以是消息的发送方，但只有运行着 libuv 事件循环的线程才能成为接收方（更准确地说，接收方是事件循环本身）。libuv 每当收到消息时，就会带着 async 监视器调用回调（`print_progress`）。

警告：必须认识到，由于消息的发送是异步的，回调可能在 `uv_async_send` 被另一个线程调用之后立即被触发，也可能过一段时间才被触发。libuv 还可能把多次 `uv_async_send` 合并起来，只调用一次你的回调。libuv 唯一保证的是——在调用 `uv_async_send` 之后，回调函数至少被调用一次。如果没有待处理的 `uv_async_send` 调用，回调不会被调用；如果你调用了两次或更多次，而 libuv 还没来得及运行回调，那么对于这多次 `uv_async_send` 调用，它可能只调用你的回调一次。但你的回调绝不会因为单个事件而被调用两次。

progress/main.c

```

_Atomic double percentage;

void fake_download(uv_work_t *req) {
    int size = *((int*) req->data);
    int downloaded = 0;
    double pct;
    while (downloaded < size) {
        pct = downloaded * 100.0 / size;
        atomic_store_explicit(&percentage, pct, memory_order_release);
        uv_async_send(&async);

        sleep(1);
        downloaded += (200+random())%1000; // can only download max 1000bytes/
sec, // but at least a 200;
    }
}

```

在下载函数中，我们更新进度指示器，并用 `uv_async_send` 把消息排入待发送队列。记住：`uv_async_send` 同样是非阻塞的，会立即返回。

progress/main.c

```

void after(uv_work_t *req, int status) {
    fprintf(stderr, "Download complete\n");
    uv_close((uv_handle_t*) &async, NULL);
}

```

回调就是标准的 libuv 模式——从监视器中取出数据。

最后，记得要清理监视器，这一点很重要。

progress/main.c

```

// Ensure final 100% progress update is sent
pct = 100.0;
atomic_store_explicit(&percentage, pct, memory_order_release);
uv_async_send(&async);

```

在上面这个滥用 `data` 字段的例子之后，bnoordhuis 指出：使用 `data` 字段不是线程安全的，`uv_async_send()` 实际上只是用来唤醒事件循环的。请使用互斥锁或读写锁来保证访问以正确的顺序进行。

注意：互斥锁和读写锁**不能**在信号处理器内部使用，而 `uv_async_send` 可以。

必须使用 `uv_async_send` 的一个场景，是与那些要求线程亲和性（thread affinity）的库互操作时。例如在 `node.js` 中，一个 `v8` 引擎实例、它的上下文及其对象都绑定在启动 `v8` 实例的那个线程上；从另一个线程去操作 `v8` 的数据结构会导致未定义的结果。现在考虑某个绑定了第三方库的 `node.js` 模块，它大致是这样工作的：

1. 在 `node` 中，先为第三方库设置一个 JavaScript 回调，供它在需要更多信息时调用：

```
`` `js
var lib = require('lib');
lib.on_progress(function() {
  console.log("Progress");
});
lib.do();
// do other stuff
`` `
```

1. `lib.do` 本应是非阻塞的，但第三方库是阻塞的，于是绑定层使用 `uv_queue_work`。
2. 在单独线程中执行的实际工作想要触发进度回调，却不能直接调进 `v8` 去操作 JavaScript，于是它使用 `uv_async_send`。
3. `async` 回调在主循环线程（也就是 `v8` 线程）中被调用，随后它与 `v8` 交互，触发那个 JavaScript 回调。

第 8 章 • API：事件循环

本章译自 libuv 官方文档 `api/loop.rst` (libuv 1.52)

`uv_loop_t` — 事件循环

事件循环是 libuv 功能的核心部分。它负责轮询 I/O，并根据不同的事件来源调度回调的执行。

数据类型

`uv_loop_t`

循环数据类型。

`uv_loop_option`

额外的循环选项。参见 `uv_loop_configure()`。

```
typedef enum {
    UV_LOOP_BLOCK_SIGNAL = 0,
    UV_METRICS_IDLE_TIME,
    UV_LOOP_USE_IO_URING_SQPOLL
} uv_loop_option;
```

`uv_run_mode`

与 `uv_run()` 一起使用的运行模式。

```
typedef enum {
    UV_RUN_DEFAULT = 0,
    UV_RUN_ONCE,
    UV_RUN_NOWAIT
} uv_run_mode;
```

```
void (*uv_walk_cb)(uv_handle_t* handle, void* arg)
```

传递给 `uv_walk()` 的回调的类型定义。

公共成员

```
void* uv_loop_t.data
```

用于存放用户自定义任意数据的空间。libuv 不使用也不会触碰该字段。

API

```
int uv_loop_init(uv_loop_t* loop)
```

初始化给定的 `uv_loop_t` 结构。

```
int uv_loop_configure(uv_loop_t* loop, uv_loop_option option, ...)
```

注意：自 1.0.2 版本起新增。

设置额外的循环选项。除非另有说明，通常应在第一次调用 `uv_run()` 之前调用此函数。

成功时返回 0，失败时返回 UV_E* 错误码。请做好处理 UV_ENOSYS 的准备；它表示当前平台不支持该循环选项。

支持的选项：

- UV_LOOP_BLOCK_SIGNAL：在轮询新事件时阻塞某个信号。`uv_loop_configure()` 的第二个参数是信号编号。

该操作目前仅针对 SIGPROF 信号实现，用于在使用采样分析器（sampling profiler）时抑制不必要的唤醒。请求其他信号会以 UV_EINVAL 失败。

- UV_METRICS_IDLE_TIME：累计事件循环在事件提供者（event provider）中花费的空闲时间。

要使用 `uv_metrics_idle_time()`，必须启用此选项。

- UV_LOOP_USE_IO_URING_SQPOLL：启用 SQPOLL 模式的 io_uring 实例来处理异步文件系统操作。

注意：1.39.0 版本变更：新增了 UV_METRICS_IDLE_TIME 选项。

注意：1.49.0 版本变更：新增了 UV_LOOP_USE_IO_URING_SQPOLL 选项。

```
int uv_loop_close(uv_loop_t* loop)
```

释放循环的所有内部资源。只有当循环已结束执行、且所有打开的句柄和请求都已关闭时才调用此函数，否则它将返回 UV_EBUSY。在此函数返回之后，用户即可释放为循环分配的内存。

```
uv_loop_t* uv_default_loop(void)
```

返回已初始化的默认循环。在分配失败时可能返回 NULL。

此函数只是一种在整个应用程序中拥有一个全局循环的便捷方式，默认循环与用

`uv_loop_init()` 初始化的循环没有任何区别。因此，默认循环可以（也应该）用 `uv_loop_close()` 关闭，以释放与之关联的资源。

警告：此函数不是线程安全的。

```
int uv_run(uv_loop_t* loop, uv_run_mode mode)
```

此函数运行事件循环。根据指定的模式，其行为有所不同：

- UV_RUN_DEFAULT：运行事件循环，直到不再有活动和被引用的句柄或请求。如果 `uv_stop()` 被调用且仍有活动的句柄或请求，则返回非零值；其他所有情况下返回零。
- UV_RUN_ONCE：轮询一次 I/O。注意，如果没有待处理的回调，此函数会阻塞。完成时（没有剩余的活动句柄或请求）返回零；如果预期还有更多回调（意味着你将来应在某个时刻再次运行事件循环），则返回非零值。
- UV_RUN_NOWAIT：轮询一次 I/O，但如果没有待处理的回调则不阻塞。完成时（没有剩余的活动句柄或请求）返回零；如果预期还有更多回调（意味着你将来应在某个时刻再次运行事件循环），则返回非零值。

`uv_run()` 不可重入。不得在回调中调用它。

```
int uv_loop_alive(const uv_loop_t* loop)
```

如果循环中存在被引用的活动句柄、活动请求或正在关闭的句柄，则返回非零值。

```
void uv_stop(uv_loop_t* loop)
```

停止事件循环，使 `uv_run()` 尽快结束。这不会早于下一次循环迭代发生。如果此函数在阻塞等待 I/O 之前被调用，循环在本轮迭代中将不会为 I/O 而阻塞。

```
size_t uv_loop_size(void)
```

返回 `uv_loop_t` 结构的大小。对于不想了解结构布局的 FFI 绑定编写者很有用。

```
int uv_backend_fd(const uv_loop_t* loop)
```

获取后端文件描述符。仅支持 kqueue、epoll 和 event ports。

此函数可与 `uv_run(loop, UV_RUN_NOWAIT)` 结合使用，在一个线程中轮询、在另一个线程中运行事件循环的回调，示例参见 `test/test-embed.c`。

注意：将 kqueue 的文件描述符嵌入另一个 kqueue 轮询集在所有平台上并非都可行。添加该文件描述符不算错误，但它永远不会产生事件。

```
int uv_backend_timeout(const uv_loop_t* loop)
```

获取轮询超时值。返回值以毫秒为单位，-1 表示无超时。

```
uint64_t uv_now(const uv_loop_t* loop)
```

返回当前时间戳（毫秒）。时间戳在事件循环 `tick` 开始时被缓存，详情及缘由参见 `uv_update_time()`。

该时间戳从某个任意的时间点起单调递增。不要对起始点做任何假设，否则你只会失望。

注意：如果需要亚毫秒级精度，请使用 `uv_hrtime()`。

```
void uv_update_time(uv_loop_t* loop)
```

更新事件循环所理解的"现在"。libuv 在事件循环 tick 开始时缓存当前时间，以减少与时间相关的系统调用次数。

通常你不需要调用此函数，除非你的回调会长时间阻塞事件循环——"长时间" somewhat 主观，但大概是 1 毫秒或更久的量级。

```
void uv_walk(uv_loop_t* loop, uv_walk_cb walk_cb, void* arg)
```

遍历句柄列表：walk_cb 将以给定的 arg 被执行。

```
int uv_loop_fork(uv_loop_t* loop)
```

注意：自 1.12.0 版本起新增。

在 fork(2) 系统调用之后，在子进程中重新初始化所有必需的内核状态。

之前已启动的监视器（watcher）在子进程中会继续保持启动状态。

对于在父进程中创建、且计划在子进程中继续使用的每一个事件循环，都必须显式调用此函数，包括默认循环（即使你在父进程中不再使用它）。此函数必须在子进程中调用 uv_run() 或任何其他使用该循环的 API 函数之前调用。否则将导致未定义行为，可能包括父进程和子进程都收到重复事件，或导致子进程异常终止。

如果可行，优先选择在子进程中创建新循环，而不是复用父进程中创建的循环。fork 之后在子进程中创建的新循环不应使用此函数。

此函数在 Windows 上未实现，返回 UV_ENOSYS。

警告：此函数是实验性的。它可能包含缺陷，且可能被更改或移除。不保证 API 和 ABI 的稳定性。

注意：在 Mac OS X 上，如果父进程中任意事件循环曾使用目录 FS 事件句柄，子进程将无法再使用最高效的 FSEvent 实现。此后子进程中目录 FS 事件句柄的使用将回退到与文件以及在其它基于 kqueue 的系统上相同的实现。

警告：在 AIX 和 SunOS 上，fork 时父进程中已启动的 FS 事件句柄不会在子进程中投递事件；它们必须被关闭并重新启动。在所有其他平台上，它们无需任何额外干预即可继续正常工作。

警告：之前从 `uv_backend_fd()` 得到的任何返回值现在都已失效。必须重新调用该函数以确定正确的后端文件描述符。

```
void* uv_loop_get_data(const uv_loop_t* loop)
```

返回 `loop->data`。

注意：自 1.19.0 版本起新增。

```
void uv_loop_set_data(uv_loop_t* loop, void* data)
```

将 `loop->data` 设置为 `data`。

注意：自 1.19.0 版本起新增。

第 9 章 • API：句柄与请求

本章译自 libuv 官方文档 `api/handle.rst` 与 `api/request.rst` (libuv 1.52)

第一部分 `uv_handle_t` — 基础句柄

`uv_handle_t` 是所有 libuv 句柄类型的基类型。

各结构经过对齐处理，因此任何 libuv 句柄都可以被强制转换为 `uv_handle_t`。这里定义的所有 API 函数都适用于任何句柄类型。

libuv 的句柄不可移动。传递给函数的句柄结构指针在所请求操作的持续期间必须保持有效。使用在栈上分配的句柄时要格外小心。

数据类型

`uv_handle_t`

libuv 基础句柄类型。

`uv_handle_type`

libuv 句柄的种类。

```
typedef enum {
    UV_UNKNOWN_HANDLE = 0,
    UV_ASYNC,
    UV_CHECK,
    UV_FS_EVENT,
    UV_FS_POLL,
    UV_HANDLE,
    UV_IDLE,
    UV_NAMED_PIPE,
    UV_POLL,
    UV_PREPARE,
    UV_PROCESS,
    UV_STREAM,
    UV_TCP,
    UV_TIMER,
    UV_TTY,
    UV_UDP,
    UV_SIGNAL,
    UV_FILE,
    UV_HANDLE_TYPE_MAX
} uv_handle_type;
```

uv_any_handle

所有句柄类型的联合（union）。

```
void (*uv_alloc_cb)(uv_handle_t* handle, size_t suggested_size, uv_buf_t* buf)
```

传递给 `uv_read_start()` 和 `uv_udp_recv_start()` 的回调的类型定义。用户必须分配内存并填充所提供的 `uv_buf_t` 结构。如果缓冲区的 `base` 被赋为 `NULL` 或长度被赋为 `0`，将在 `uv_udp_recv_cb` 或 `uv_read_cb` 回调中触发 `UV_ENOBUFS` 错误。

每个缓冲区只使用一次，用户有责任在 `uv_udp_recv_cb` 或 `uv_read_cb` 回调中释放它。

会提供一个建议大小（目前大多数情况下为 65536），但它只是一个参考值，与待读取的数据量没有任何关系。用户可以自由分配自己决定的内存量。

例如，采用自定义分配策略（如使用空闲链表、分配池或基于 `slab` 的分配器）的应用程序，可以决定使用与其现有内存块相匹配的不同大小。

示例：

```
static void my_alloc_cb(uv_handle_t* handle, size_t suggested_size, uv_buf_t*
buf) {
    buf->base = malloc(suggested_size);
    buf->len = suggested_size;
}
```

```
void (*uv_close_cb)(uv_handle_t* handle)
```

传递给 `uv_close()` 的回调的类型定义。

公共成员

```
uv_loop_t* uv_handle_t.loop
```

指向句柄所在 `uv_loop_t` 的指针。只读。

```
uv_handle_type uv_handle_t.type
```

`uv_handle_type`，指示底层句柄的类型。只读。

```
void* uv_handle_t.data
```

用于存放用户自定义任意数据的空间。libuv 不使用该字段。

API

```
UV_HANDLE_TYPE_MAP(iter_macro)
```

该宏会对每一种句柄类型展开为一系列对 `iter_macro` 的调用。`iter_macro` 以两个参数被调用：去掉 `UV_` 前缀后的 `uv_handle_type` 枚举元素名，以及去掉 `uv_` 前缀和 `_t` 后缀后的对应结构体类型名。

```
int uv_is_active(const uv_handle_t* handle)
```

如果句柄是活动的则返回非零值，否则返回零。"活动"的含义取决于句柄类型：

- `uv_async_t` 句柄始终是活动的，无法被停用，除非用 `uv_close()` 将其关闭。

- `uv_pipe_t`、`uv_tcp_t`、`uv_udp_t` 等句柄——基本上任何涉及 I/O 的句柄——在其正在执行涉及 I/O 的操作（如读取、写入、连接、接受新连接等）时是活动的。
- `uv_check_t`、`uv_idle_t`、`uv_timer_t` 等句柄在通过调用 `uv_check_start()`、`uv_idle_start()` 等被启动后是活动的。

经验法则：如果类型为 `uv_foo_t` 的句柄有一个 `uv_foo_start()` 函数，那么从该函数被调用的那一刻起它就是活动的；同样，`uv_foo_stop()` 会再次停用该句柄。

```
int uv_is_closing(const uv_handle_t* handle)
```

如果句柄正在关闭或已关闭，返回非零值，否则返回零。

注意：此函数只应在句柄初始化之后、关闭回调到来之前使用。

```
void uv_close(uv_handle_t* handle, uv_close_cb close_cb)
```

请求关闭句柄。`close_cb` 将在此调用之后被异步调用。在释放内存之前，必须对每个句柄调用此函数。此外，内存只能在 `close_cb` 中或其返回之后释放。

包装文件描述符的句柄会被立即关闭，但 `close_cb` 仍会被推迟到事件循环的下一次迭代。这给了你一个释放与句柄关联的资源的机会。

进行中的请求（如 `uv_connect_t` 或 `uv_write_t`）会被取消，其回调将以 `status=UV_ECANCELED` 被异步调用。

在不需要清理或释放资源的情况下，`close_cb` 可以为 `NULL`。

```
void uv_ref(uv_handle_t* handle)
```

引用给定的句柄。引用操作是幂等的，也就是说，如果句柄已被引用，再次调用此函数不会有任何效果。

参见“引用计数”一节。

```
void uv_unref(uv_handle_t* handle)
```

取消对给定句柄的引用。引用操作是幂等的，也就是说，如果句柄未被引用，再次调用此函数不会有任何效果。

参见"引用计数"一节。

```
int uv_has_ref(const uv_handle_t* handle)
```

如果句柄被引用，返回非零值，否则返回零。

参见"引用计数"一节。

```
size_t uv_handle_size(uv_handle_type type)
```

返回给定句柄类型的大小。对于不想了解结构布局的 FFI 绑定编写者很有用。

其他 API 函数

以下 API 函数接受一个 `uv_handle_t` 参数，但只对部分句柄类型有效。

```
int uv_send_buffer_size(uv_handle_t* handle, int* value)
```

获取或设置操作系统为套接字使用的发送缓冲区大小。

如果 `*value == 0`，则会将 `*value` 设置为当前的发送缓冲区大小。如果 `*value > 0`，则会用 `*value` 设置新的发送缓冲区大小。

成功时返回零，出错时返回负数结果。

此函数适用于 Unix 上的 TCP、pipe 和 UDP 句柄，以及 Windows 上的 TCP 和 UDP 句柄。

注意：Linux 会将大小设置为所设值的两倍，并返回所设原始值的两倍。

```
int uv_recv_buffer_size(uv_handle_t* handle, int* value)
```

获取或设置操作系统为套接字使用的接收缓冲区大小。

如果 `*value == 0`，则会将 `*value` 设置为当前的接收缓冲区大小。如果 `*value > 0`，则会用 `*value` 设置新的接收缓冲区大小。

成功时返回零，出错时返回负数结果。

此函数适用于 Unix 上的 TCP、pipe 和 UDP 句柄，以及 Windows 上的 TCP 和 UDP 句柄。

注意：Linux 会将大小设置为所设值的两倍，并返回所设原始值的两倍。

```
int uv_fileno(const uv_handle_t* handle, uv_os_fd_t* fd)
```

获取平台相关的等价文件描述符。

支持以下句柄：TCP、pipes、TTY、UDP 和 poll。传入任何其他句柄类型都会以 `UV_EINVAL` 失败。

如果句柄尚未附加文件描述符，或句柄本身已被关闭，此函数将返回 `UV_EBADF`。

警告：使用此函数时要非常小心。libuv 假定自己掌控着该文件描述符，因此对它的任何更改都可能导致故障。

```
uv_loop_t* uv_handle_get_loop(const uv_handle_t* handle)
```

返回 `handle->loop`。

注意：自 1.19.0 版本起新增。

```
void* uv_handle_get_data(const uv_handle_t* handle)
```

返回 `handle->data`。

注意：自 1.19.0 版本起新增。

```
void uv_handle_set_data(uv_handle_t* handle, void* data)
```

将 `handle->data` 设置为 `data`。

注意：自 1.19.0 版本起新增。

```
uv_handle_type uv_handle_get_type(const uv_handle_t* handle)
```

返回 `handle->type`。

注意：自 1.19.0 版本起新增。

```
const char* uv_handle_type_name(uv_handle_type type)
```

返回给定句柄类型对应结构的名称，例如 `UV_NAMED_PIPE` 对应 `"pipe"`（即 `uv_pipe_t`）。

如果不存在这样的句柄类型，则返回 `NULL`。

注意：自 1.19.0 版本起新增。

引用计数

libuv 事件循环（以默认模式运行时）会一直运行，直到不再有活动且被引用的句柄。用户可以通过取消引用处于活动状态的句柄来强制循环提前退出，例如在调用

```
uv_timer_start() 之后调用 uv_unref()。
```

句柄可以处于被引用或未引用状态，引用计数方案并不使用计数器，因此这两种操作都是幂等的。

默认情况下，所有句柄在活动时都是被引用的，关于"活动"的更多细节参见

```
uv_is_active() 的说明。
```

第二部分 `uv_req_t` — 基础请求

`uv_req_t` 是所有 libuv 请求类型的基类型。

各结构经过对齐处理，因此任何 libuv 请求都可以被强制转换为 `uv_req_t`。这里定义的所有 API 函数都适用于任何请求类型。

数据类型

```
uv_req_t
```

libuv 基础请求结构。

```
uv_any_req
```

所有请求类型的联合（union）。

uv_req_type

libuv 请求的种类。

```
typedef enum {
    UV_UNKNOWN_REQ = 0,
    UV_REQ,
    UV_CONNECT,
    UV_WRITE,
    UV_SHUTDOWN,
    UV_UDP_SEND,
    UV_FS,
    UV_WORK,
    UV_GETADDRINFO,
    UV_GETNAMEINFO,
    UV_REQ_TYPE_MAX,
} uv_req_type;
```

公共成员**void* uv_req_t.data**

用于存放用户自定义任意数据的空间。libuv 不使用该字段。

uv_req_type uv_req_t.type

uv_req_type，指示请求的类型。只读。

API**UV_REQ_TYPE_MAP(iter_macro)**

该宏会对每一种请求类型展开为一系列对 `iter_macro` 的调用。`iter_macro` 以两个参数被调用：去掉 `UV_` 前缀后的 `uv_req_type` 枚举元素名，以及去掉 `uv_` 前缀和 `_t` 后缀后的对应结构体类型名。

```
int uv_cancel(uv_req_t* req)
```

取消一个尚未完成的请求。如果请求正在执行或已执行完毕，则失败。写请求是个例外：取消一个已经在进行中的写操作会尝试中断它（回调可能报告部分写入），而取消一个已完成的写操作则是一次成功的空操作（no-op）。

成功时返回 0，失败时返回小于 0 的错误码。

目前仅支持取消 `uv_write_t`、`uv_fs_t`、`uv_getaddrinfo_t`、`uv_getnameinfo_t`、`uv_random_t` 和 `uv_work_t` 请求。

被取消的请求的回调会在将来某个时刻被调用。在回调被调用之前，释放与请求关联的内存是**不安全的**。

取消通过以下方式报告给回调：

- `uv_fs_t` 请求的 `req->result` 字段被设置为 `UV_ECANCELED`。
- `uv_work_t`、`uv_getaddrinfo_t`、`uv_getnameinfo_t` 或 `uv_random_t` 请求的回调以 `status == UV_ECANCELED` 被调用。
- `uv_write_t` 请求的回调以 `status == UV_ECANCELED` 被调用。可在回调中使用 `uv_write_nwritten()` 来确定在取消之前已写入多少字节。被完全取消的写入（未写入任何字节）的回调，相对于同一流上的其他写操作可能会乱序调用。注意，如果内核在取消生效之前已完成对该写入的处理，被取消的写入仍可能成功或以其他错误失败。

```
size_t uv_req_size(uv_req_type type)
```

返回给定请求类型的大小。对于不想了解结构布局的 FFI 绑定编写者很有用。

```
void* uv_req_get_data(const uv_req_t* req)
```

返回 `req->data`。

注意：自 1.19.0 版本起新增。

```
void uv_req_set_data(uv_req_t* req, void* data)
```

将 `req->data` 设置为 `data`。

注意：自 1.19.0 版本起新增。

```
uv_req_type uv_req_get_type(const uv_req_t* req)
```

返回 req->type。

注意：自 1.19.0 版本起新增。

```
const char* uv_req_type_name(uv_req_type type)
```

返回给定请求类型对应结构的名称，例如 `UV_CONNECT` 对应 `"connect"`（即 `uv_connect_t`）。

如果不存在这样的请求类型，则返回 `NULL`。

注意：自 1.19.0 版本起新增。

第 10 章 · API：定时器与异步唤醒

本章译自 libuv 官方文档 `api/timer.rst`、`api/check.rst`、`api/prepare.rst`、`api/idle.rst` 与 `api/async.rst` (libuv 1.52)

第一部分 `uv_timer_t` — 定时器句柄

定时器句柄用于安排回调在将来被调用。

定时器既可以是一次性的，也可以是重复的。重复定时器不会根据执行开销进行调整，而是相对于事件循环所理解的"现在"重新装载。

libuv 会在执行定时器回调之前、以及从等待 I/O 中唤醒之后，立即更新它对"现在"的理解。另见 `uv_update_time()`。

示例：一个间隔为 50 ms 的重复定时器，若其回调耗时 17 ms 完成，则会在 33 ms 后再次运行。如果其他任务耗时超过 33 ms，定时器回调会尽可能快地运行。

数据类型

`uv_timer_t`

定时器句柄类型。

```
void (*uv_timer_cb)(uv_timer_t* handle)
```

传递给 `uv_timer_start()` 的回调的类型定义。

公共成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

```
int uv_timer_init(uv_loop_t* loop, uv_timer_t* handle)
```

初始化句柄。

```
int uv_timer_start(uv_timer_t* handle, uv_timer_cb cb, uint64_t timeout, uint64_t repeat)
```

启动定时器。 `timeout` 和 `repeat` 的单位是毫秒。

如果 `timeout` 为零，回调将在下一次事件循环迭代时触发。如果 `repeat` 非零，回调先在 `timeout` 毫秒后触发，之后每隔 `repeat` 毫秒重复触发。

注意：不会更新事件循环所理解的"现在"。更多信息参见 `uv_update_time()`。

注意：如果定时器已经处于活动状态，则只是被更新。

```
int uv_timer_stop(uv_timer_t* handle)
```

停止定时器，回调将不再被调用。

```
int uv_timer_again(uv_timer_t* handle)
```

停止定时器，如果它是重复定时器，则以 `repeat` 值作为超时重新启动它。如果定时器从未被启动过，则返回 `UV_EINVAL`。

```
void uv_timer_set_repeat(uv_timer_t* handle, uint64_t repeat)
```

以毫秒为单位设置重复间隔值。定时器将按给定间隔被调度运行，而不受回调执行时长的影响；在发生时间片超限的情况下，将遵循正常的定时器语义。

注意：如果在定时器回调中设置 `repeat` 值，它不会立即生效。如果定时器此前是非重复的，它已经停止；如果它是重复的，下一次超时将使用旧的 `repeat` 值来调度。

```
uint64_t uv_timer_get_repeat(const uv_timer_t* handle)
```

获取定时器的 `repeat` 值。

```
uint64_t uv_timer_get_due_in(const uv_timer_t* handle)
```

获取定时器的到期剩余时间；若已到期则返回 0。该时间相对于 `uv_now()`。

注意：自 1.40.0 版本起新增。

参见：`uv_handle_t` 的 API 函数同样适用。

第二部分 `uv_check_t` — Check 句柄

Check 句柄会在每次循环迭代中运行给定的回调一次，时机紧随 I/O 轮询之后。

数据类型

```
uv_check_t
```

Check 句柄类型。

```
void (*uv_check_cb)(uv_check_t* handle)
```

传递给 `uv_check_start()` 的回调的类型定义。

公共成员

无。

参见：`uv_handle_t` 的成员同样适用。

API

```
int uv_check_init(uv_loop_t* loop, uv_check_t* check)
```

初始化句柄。此函数总是成功。

返回值：0。

```
int uv_check_start(uv_check_t* check, uv_check_cb cb)
```

以给定的回调启动句柄。此函数总是成功，除非 `cb` 为 `NULL`。

返回值：成功时为 0，当 `cb == NULL` 时为 `UV_EINVAL`。

```
int uv_check_stop(uv_check_t* check)
```

停止句柄，回调将不再被调用。此函数总是成功。

返回值：0。

参见： `uv_handle_t` 的 API 函数同样适用。

第三部分 `uv_prepare_t` — Prepare 句柄

Prepare 句柄会在每次循环迭代中运行给定的回调一次，时机紧接在 I/O 轮询之前。

数据类型

```
uv_prepare_t
```

Prepare 句柄类型。

```
void (*uv_prepare_cb)(uv_prepare_t* handle)
```

传递给 `uv_prepare_start()` 的回调的类型定义。

公共成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

```
int uv_prepare_init(uv_loop_t* loop, uv_prepare_t* prepare)
```

初始化句柄。此函数总是成功。

返回值：0。

```
int uv_prepare_start(uv_prepare_t* prepare, uv_prepare_cb cb)
```

以给定的回调启动句柄。此函数总是成功，除非 `cb` 为 `NULL`。

返回值：成功时为 0，当 `cb == NULL` 时为 `UV_EINVAL`。

```
int uv_prepare_stop(uv_prepare_t* prepare)
```

停止句柄，回调将不再被调用。此函数总是成功。

返回值：0。

参见： `uv_handle_t` 的 API 函数同样适用。

第四部分 `uv_idle_t` — Idle 句柄

Idle 句柄会在每次循环迭代中运行给定的回调一次，时机在 `uv_prepare_t` 句柄之前。

注意：与 Prepare 句柄的显著区别在于：当存在活动的 Idle 句柄时，循环将执行零超时轮询，而不是为 I/O 而阻塞。

警告：尽管名字如此，Idle 句柄的回调会在每次循环迭代时都被调用，而不是在循环真正"空闲"时。

数据类型

```
uv_idle_t
```

Idle 句柄类型。

```
void (*uv_idle_cb)(uv_idle_t* handle)
```

传递给 `uv_idle_start()` 的回调的类型定义。

公共成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

```
int uv_idle_init(uv_loop_t* loop, uv_idle_t* idle)
```

初始化句柄。此函数总是成功。

返回值：0。

```
int uv_idle_start(uv_idle_t* idle, uv_idle_cb cb)
```

以给定的回调启动句柄。此函数总是成功，除非 `cb` 为 `NULL`。

返回值：成功时为 0，当 `cb == NULL` 时为 `UV_EINVAL`。

```
int uv_idle_stop(uv_idle_t* idle)
```

停止句柄，回调将不再被调用。此函数总是成功。

返回值：0。

参见： `uv_handle_t` 的 API 函数同样适用。

第五部分 `uv_async_t` — Async 句柄

Async 句柄允许用户"唤醒"事件循环，并让一个回调从另一个线程被调用。

注意：对于给定的 `async` 句柄，`uv_async_send()` 与 `uv_async_cb` 的调用是顺序一致（`seq_cst`）的操作：在 `uv_async_send()` 之前进行的所有内存访问（读和写）对该回调均可见。

警告：libuv 会合并（coalesce）对 `uv_async_send()` 的调用，也就是说，并非每一次调用都会导致回调执行一次。例如：如果在回调被调用之前连续调用了 5 次 `uv_async_send()`，回调将只被调用一次。如果在回调被调用之后再次调用 `uv_async_send()`，回调会再次被调用。不过，由于操作是顺序一致的，该回调中读到的或写入的值将始终与其他线程读到的或写入的值相同（或更新）。

注意：1.53.0 版本变更：`uv_async_send()` 与 `uv_async_cb` 是顺序一致的。在此版本之前，libuv 可能合并调用的任何情形，出于正确性考虑很可能需要在其之前添加一个完整的 `seq_cst` 内存栅栏（fence）。

数据类型

`uv_async_t`

Async 句柄类型。

```
void (*uv_async_cb)(uv_async_t* handle)
```

传递给 `uv_async_init()` 的回调的类型定义。

公共成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

```
int uv_async_init(uv_loop_t* loop, uv_async_t* async, uv_async_cb async_cb)
```

初始化句柄。允许传入 NULL 回调。

返回值：成功时为 0，失败时为小于 0 的错误码。

注意： 与其他句柄初始化函数不同，它会立即启动该句柄。

```
int uv_async_send(uv_async_t* async)
```

唤醒事件循环，并调用 `async` 句柄的回调。

返回值：成功时为 0，失败时为小于 0 的错误码。

注意：可以安全地从任意线程调用此函数。回调将在循环线程中被调用。

注意：`uv_async_send()` 是 异步信号安全的 (`async-signal-safe`)。可以安全地在信号处理函数中调用此函数。

注意：相对于使用同一个 `async` 句柄的其他调用和回调，此调用是一个完整的内存栅栏 (`memory fence`)，因此它会为发送线程和接收线程上围绕此调用及相应回调的所有操作建立顺序。

参见：`uv_handle_t` 的 API 函数同样适用。

第 11 章 • API: 文件系统

文件系统操作 (File system operations)

本章译自 libuv 官方文档 `api/fs.rst` (libuv 1.52)

libuv 提供了丰富的跨平台同步与异步文件系统操作。本文档中定义的所有函数都接受一个回调，且回调允许为 NULL。如果回调为 NULL，则请求以同步方式完成；否则将以异步方式执行。

所有文件操作都在线程池上运行。关于线程池大小的信息请参见线程池一章。

从 libuv v1.45.0 开始，Linux 上的一些文件操作在可能的情况下会被交给 `io_uring` 处理。除了吞吐量的（有时是显著的）提升之外，可观察到的行为不应有任何变化。当所需的内核特性不可用或不适用时，libuv 会回退到使用线程池。从 libuv v1.49.0 开始，该行为被回退，Linux 上的 libuv 默认重新使用线程池。要启用 `io_uring`，必须为 `uv_loop_t` 实例配置 `UV_LOOP_USE_IO_URING_SQPOLL` 选项。

注意：在 Windows 上，`uv_fs_*` 函数使用 utf-8 编码。

数据类型

`uv_fs_t`

文件系统请求类型。

`uv_timespec_t`

用于存储具有纳秒精度的时间的数据类型，不兼容 Y2K38（2038 年问题）。将在 libuv v2.0 中被 `uv_timespec64_t` 取代。

```
typedef struct {
    long tv_sec;
    long tv_nsec;
} uv_timespec_t;
```

uv_stat_t

struct stat 的可移植等价物。

```
typedef struct {
    uint64_t st_dev;
    uint64_t st_mode;
    uint64_t st_nlink;
    uint64_t st_uid;
    uint64_t st_gid;
    uint64_t st_rdev;
    uint64_t st_ino;
    uint64_t st_size;
    uint64_t st_blksize;
    uint64_t st_blocks;
    uint64_t st_flags;
    uint64_t st_gen;
    uv_timespec_t st_atim;
    uv_timespec_t st_mtim;
    uv_timespec_t st_ctim;
    uv_timespec_t st_birthtim;
} uv_stat_t;
```

uv_fs_type

文件系统请求类型。

```
typedef enum {
    UV_FS_UNKNOWN = -1,
    UV_FS_CUSTOM,
    UV_FS_OPEN,
    UV_FS_CLOSE,
    UV_FS_READ,
    UV_FS_WRITE,
    UV_FS_SENDFILE,
    UV_FS_STAT,
    UV_FS_LSTAT,
    UV_FS_FSTAT,
    UV_FS_FTRUNCATE,
    UV_FS_UTIME,
    UV_FS_FUTIME,
    UV_FS_ACCESS,
    UV_FS_CHMOD,
    UV_FS_FCHMOD,
    UV_FS_FSYNC,
    UV_FS_FDATASYNC,
    UV_FS_UNLINK,
    UV_FS_RMDIR,
    UV_FS_MKDIR,
    UV_FS_MKDTEMP,
    UV_FS_RENAME,
    UV_FS_SCANDIR,
    UV_FS_LINK,
    UV_FS_SYMLINK,
    UV_FS_READLINK,
    UV_FS_CHOWN,
    UV_FS_FCHOWN,
    UV_FS_REALPATH,
    UV_FS_COPYFILE,
    UV_FS_LCHOWN,
    UV_FS_OPENDIR,
    UV_FS_READDIR,
    UV_FS_CLOSEDIR,
    UV_FS_MKSTEMP,
    UV_FS_LUTIME
} uv_fs_type;
```

uv_statfs_t

`struct statfs` 的精简跨平台等价物。用于 `uv_fs_statfs`。

```
typedef struct uv_statfs_s {
    uint64_t f_type;
    uint64_t f_bsize;
    uint64_t f_blocks;
    uint64_t f_bfree;
    uint64_t f_bavail;
    uint64_t f_files;
    uint64_t f_ffree;
    uint64_t f_fsize;
    uint64_t f_spare[3];
} uv_statfs_t;
```

uv_dirent_type_t

目录项 (dirent) 的类型。

```
typedef enum {
    UV_DIRENT_UNKNOWN,
    UV_DIRENT_FILE,
    UV_DIRENT_DIR,
    UV_DIRENT_LINK,
    UV_DIRENT_FIFO,
    UV_DIRENT_SOCKET,
    UV_DIRENT_CHAR,
    UV_DIRENT_BLOCK
} uv_dirent_type_t;
```

uv_dirent_t

struct dirent 的跨平台 (精简) 等价物。用于 uv_fs_scandir_next。

```
typedef struct uv_dirent_s {
    const char* name;
    uv_dirent_type_t type;
} uv_dirent_t;
```

uv_dir_t

用于流式目录遍历的数据类型。由 uv_fs_opendir、uv_fs_readdir 和 uv_fs_closedir 使用。dirents 表示用户提供的、用于存放结果的 uv_dirent_t 数组。nentries 是用户提供的 dirents 数组的最大长度。

```
typedef struct uv_dir_s {
    uv_dirent_t* dirents;
    size_t nentries;
} uv_dir_t;
```

```
void (*uv_fs_cb)(uv_fs_t* req)
```

请求异步完成时调用的回调。

公开成员

```
uv_loop_t* uv_fs_t.loop
```

启动该请求并在其中报告完成事件的循环。只读。

```
uv_fs_type uv_fs_t.fs_type
```

文件系统请求类型。

```
const char* uv_fs_t.path
```

请求所作用的路径。

```
ssize_t uv_fs_t.result
```

请求的结果。< 0 表示错误，否则表示成功。对于 `uv_fs_read` 或 `uv_fs_write` 等请求，该值分别表示读取或写入的数据量。

```
uv_stat_t uv_fs_t.statbuf
```

存储 `uv_fs_stat` 及其他 `stat` 请求的结果。

```
void* uv_fs_t.ptr
```

存储 `uv_fs_readlink` 和 `uv_fs_realpath` 的结果，并作为 `statbuf` 的别名。

参见： `uv_req_t` 的成员同样适用。

API

```
void uv_fs_req_cleanup(uv_fs_t* req)
```

```
void uv_fs_req_cleanup(uv_fs_t* req)
```

清理请求。必须在请求结束后调用，以释放 libuv 可能分配的内存。

```
int uv_fs_close(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

```
int uv_fs_close(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

等价于 `close(2)`。

```
int uv_fs_open(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, int mode, uv_fs_cb cb)
```

```
int uv_fs_open(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, int mode, uv_fs_cb cb)
```

等价于 `open(2)`。

注意： 在 Windows 上 libuv 使用 `CreateFileW`，因此文件始终以二进制模式打开。因此不支持 `O_BINARY` 和 `O_TEXT` 标志。

```
int uv_fs_read(uv_loop_t* loop, uv_fs_t* req, uv_file file, const uv_buf_t bufs[], unsigned int nbufs, int64_t offset, uv_fs_cb cb)
```

```
int uv_fs_read(uv_loop_t* loop, uv_fs_t* req, uv_file file, const uv_buf_t bufs[], unsigned int nbufs, int64_t offset, uv_fs_cb cb)
```

等价于 `preadv(2)`。如果 `offset` 参数为 `-1`，则使用并更新当前文件偏移量。

警告：在 Windows 上，在非 MSVC 环境（例如使用 GCC 或 Clang 构建 libuv 时）下，使用 `UV_FS_O_FILEMAP` 打开的文件在内存映射读取操作失败时可能导致致命崩溃。

```
int uv_fs_unlink(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

```
int uv_fs_unlink(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

等价于 `unlink(2)`。

```
int uv_fs_write(uv_loop_t* loop, uv_fs_t* req, uv_file file, const uv_buf_t bufs[],
unsigned int nbufs, int64_t offset, uv_fs_cb cb)
```

```
int uv_fs_write(uv_loop_t* loop, uv_fs_t* req, uv_file file, const uv_buf_t
bufs[], unsigned int nbufs, int64_t offset, uv_fs_cb cb)
```

等价于 `pwritev(2)`。如果 `offset` 参数为 `-1`，则使用并更新当前文件偏移量。

警告：在 Windows 上，在非 MSVC 环境（例如使用 GCC 或 Clang 构建 libuv 时）下，使用 `UV_FS_O_FILEMAP` 打开的文件在内存映射写入操作失败时可能导致致命崩溃。

```
int uv_fs_mkdir(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode, uv_fs_cb
cb)
```

```
int uv_fs_mkdir(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode,
uv_fs_cb cb)
```

等价于 `mkdir(2)`。

注意：`mode` 目前在 Windows 上未实现。

```
int uv_fs_mkdtemp(uv_loop_t* loop, uv_fs_t* req, const char* tpl, uv_fs_cb cb)
```

```
int uv_fs_mkdtemp(uv_loop_t* loop, uv_fs_t* req, const char* tpl, uv_fs_cb cb)
```

等价于 `mkdtemp(3)`。结果可以在 `req->path` 处以 null 结尾字符串的形式找到。

```
int uv_fs_mkstemp(uv_loop_t* loop, uv_fs_t* req, const char* tpl, uv_fs_cb cb)
```

```
int uv_fs_mkstemp(uv_loop_t* loop, uv_fs_t* req, const char* tpl, uv_fs_cb cb)
```

等价于 `mkstemp(3)`。所创建文件的路径可以在 `req->path` 处以 null 结尾字符串的形式找到。文件描述符可以在 `req->result` 处以整数形式找到。

1.34.0 新增。

```
int uv_fs_rmdir(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

```
int uv_fs_rmdir(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

等价于 `rmdir(2)`。

```
int uv_fs_opendir(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

```
int uv_fs_opendir(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

将 `path` 作为目录流打开。成功时，会分配一个 `uv_dir_t` 并通过 `req->ptr` 返回。这块内存不会被 `uv_fs_req_cleanup()` 释放，但 `req->ptr` 会被置为 `NULL`。分配的内存必须通过调用 `uv_fs_closedir()` 来释放。失败时不分配任何内存。

可以通过将得到的 `uv_dir_t` 传给 `uv_fs_readdir()` 来遍历目录内容。

1.28.0 新增。

```
int uv_fs_closedir(uv_loop_t* loop, uv_fs_t* req, uv_dir_t* dir, uv_fs_cb cb)
```

```
int uv_fs_closedir(uv_loop_t* loop, uv_fs_t* req, uv_dir_t* dir, uv_fs_cb cb)
```

关闭 `dir` 所表示的目录流，并释放 `uv_fs_opendir()` 分配的内存。

1.28.0 新增。

```
int uv_fs_readdir(uv_loop_t* loop, uv_fs_t* req, uv_dir_t* dir, uv_fs_cb cb)
```

```
int uv_fs_readdir(uv_loop_t* loop, uv_fs_t* req, uv_dir_t* dir, uv_fs_cb cb)
```

遍历成功调用 `uv_fs_opendir()` 所返回的目录流 `dir`。在调用 `uv_fs_readdir()` 之前，调用者必须设置 `dir->dirents` 和 `dir->nentries`，分别表示用于存放读取到的目录项的 `uv_dirent_t` 元素数组及其大小。

成功时，结果是一个 ≥ 0 的整数，表示从流中读取的目录项数量。

1.28.0 新增。

警告： `uv_fs_readdir()` 不是线程安全的。

注意： 此函数不会返回 "." 和 ".." 目录项。

注意： 成功时此函数会分配内存，必须使用 `uv_fs_req_cleanup()` 释放。
`uv_fs_req_cleanup()` 必须在用 `uv_fs_closedir()` 关闭目录之前调用。

```
int uv_fs_scandir(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, uv_fs_cb cb) / int uv_fs_scandir_next(uv_fs_t* req, uv_dirent_t* ent)
```

```
int uv_fs_scandir(uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, uv_fs_cb cb)
int uv_fs_scandir_next(uv_fs_t* req, uv_dirent_t* ent)
```

等价于 `scandir(3)`，但 API 略有不同。请求的回调被调用后，用户可以使用 `uv_fs_scandir_next` 填充 `ent`，获取下一个目录项数据。当没有更多目录项时，将返回 `UV_EOF`。

注意： 与 `scandir(3)` 不同，此函数不会返回 "." 和 ".." 目录项。

注意： 在 Linux 上，获取目录项类型只有部分文件系统支持（撰写本文时为 `btrfs`、`ext2`、`ext3` 和 `ext4`），请查阅 `getdents(2)` 手册页。

```
int uv_fs_stat(...) / int uv_fs_fstat(...) / int uv_fs_lstat(...)
```

```
int uv_fs_stat(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
int uv_fs_fstat(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
int uv_fs_lstat(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

分别等价于 `stat(2)`、`fstat(2)` 和 `lstat(2)`。

```
int uv_fs_statfs(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

```
int uv_fs_statfs(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

等价于 `statfs(2)`。成功时，会分配一个 `uv_statfs_t` 并通过 `req->ptr` 返回。该内存由 `uv_fs_req_cleanup()` 释放。

注意：结果 `uv_statfs_t` 中底层操作系统不支持的字段将被置为零。

1.31.0 新增。

```
int uv_fs_rename(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, uv_fs_cb cb)
```

```
int uv_fs_rename(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, uv_fs_cb cb)
```

等价于 `rename(2)`。

```
int uv_fs_fsync(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

```
int uv_fs_fsync(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

等价于 `fsync(2)`。

注意：在 AIX 上，`uv_fs_fsync` 对引用非普通文件的文件描述符返回 `UV_EBADF`。

```
int uv_fs_fdatasync(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

```
int uv_fs_fdatasync(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_fs_cb cb)
```

等价于 `fdatasync(2)`。

```
int uv_fs_ftruncate(uv_loop_t* loop, uv_fs_t* req, uv_file file, int64_t offset,
uv_fs_cb cb)
```

```
int uv_fs_ftruncate(uv_loop_t* loop, uv_fs_t* req, uv_file file, int64_t
offset, uv_fs_cb cb)
```

等价于 `ftruncate(2)`。

```
int uv_fs_copyfile(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, int flags, uv_fs_cb cb)
```

```
int uv_fs_copyfile(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, int flags, uv_fs_cb cb)
```

将文件从 `path` 复制到 `new_path`。支持的 `flags` 如下所述。

- `UV_FS_COPYFILE_EXCL`：如果存在此标志，当目标路径已存在时，`uv_fs_copyfile()` 将以 `UV_EEXIST` 失败。默认行为是目标存在时覆盖它。
- `UV_FS_COPYFILE_FICLONE`：如果存在此标志，`uv_fs_copyfile()` 将尝试创建写时复制（copy-on-write）反射链接（reflink）。如果底层平台不支持写时复制，或尝试使用写时复制时发生错误，则使用基于 `uv_fs_sendfile` 的回退复制机制。
- `UV_FS_COPYFILE_FICLONE_FORCE`：如果存在此标志，`uv_fs_copyfile()` 将尝试创建写时复制反射链接。如果底层平台不支持写时复制，或尝试使用写时复制时发生错误，则返回错误。

警告：如果目标路径已创建，但复制数据时发生错误，则目标路径会被删除。在关闭和删除文件之间存在一个短暂的时间窗口，其他进程可能在此期间访问该文件。

1.14.0 新增。

1.20.0 变更：支持 `UV_FS_COPYFILE_FICLONE` 和 `UV_FS_COPYFILE_FICLONE_FORCE`。

1.33.0 变更：使用 `UV_FS_COPYFILE_FICLONE_FORCE` 时如果发生错误，将返回该错误。此前，所有错误都被映射为 `UV_ENOTSUP`。

```
int uv_fs_sendfile(uv_loop_t* loop, uv_fs_t* req, uv_file out_fd, uv_file in_fd,
int64_t in_offset, size_t length, uv_fs_cb cb)
```

```
int uv_fs_sendfile(uv_loop_t* loop, uv_fs_t* req, uv_file out_fd, uv_file
in_fd, int64_t in_offset, size_t length, uv_fs_cb cb)
```

功能受限的 `sendfile(2)` 等价物。

```
int uv_fs_access(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode, uv_fs_cb
cb)
```

```
int uv_fs_access(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode,
uv_fs_cb cb)
```

在 Unix 上等价于 `access(2)`。Windows 使用 `GetFileAttributesW()`。

```
int uv_fs_chmod(...) / int uv_fs_fchmod(...)
```

```
int uv_fs_chmod(uv_loop_t* loop, uv_fs_t* req, const char* path, int mode,
uv_fs_cb cb)
int uv_fs_fchmod(uv_loop_t* loop, uv_fs_t* req, uv_file file, int mode,
uv_fs_cb cb)
```

分别等价于 `chmod(2)` 和 `fchmod(2)`。

```
int uv_fs_utime(...) / int uv_fs_futime(...) / int uv_fs_lutime(...)
```

```
int uv_fs_utime(uv_loop_t* loop, uv_fs_t* req, const char* path, double atime,
double mtime, uv_fs_cb cb)
int uv_fs_futime(uv_loop_t* loop, uv_fs_t* req, uv_file file, double atime, double
le mtime, uv_fs_cb cb)
int uv_fs_lutime(uv_loop_t* loop, uv_fs_t* req, const char* path, double atime,
double mtime, uv_fs_cb cb)
```

分别等价于 `utime(2)`、`futimes(3)` 和 `lutimes(3)`。

将 `UV_FS_UTIME_NOW` 作为 `atime` 或 `mtime` 传入时，时间戳将被设置为当前时间。

将 `UV_FS_UTIME_OMIT` 作为 `atime` 或 `mtime` 传入时，时间戳保持不变。

注意：z/OS: `uv_fs_lutime()` 未在 z/OS 上实现。仍然可以调用它，但会返回 `UV_ENOSYS`。

注意：AIX: `uv_fs_futime()` 和 `uv_fs_lutime()` 函数仅适用于 AIX 7.1 及更新版本。在更旧的版本上仍然可以调用，但会返回 `UV_ENOSYS`。

1.10.0 变更: Windows 支持亚秒精度。

```
int uv_fs_link(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, uv_fs_cb cb)
```

```
int uv_fs_link(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, uv_fs_cb cb)
```

等价于 `link(2)`。

```
int uv_fs_symlink(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, int flags, uv_fs_cb cb)
```

```
int uv_fs_symlink(uv_loop_t* loop, uv_fs_t* req, const char* path, const char*
new_path, int flags, uv_fs_cb cb)
```

等价于 `symlink(2)`。

注意：在 Windows 上，可以指定 `flags` 参数来控制符号链接的创建方式：

- `UV_FS_SYMLINK_DIR`：指示 `path` 指向一个目录。
- `UV_FS_SYMLINK_JUNCTION`：请求使用 junction point（联接点）创建符号链接。

```
int uv_fs_readlink(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

```
int uv_fs_readlink(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

等价于 `readlink(2)`。结果字符串存储在 `req->ptr` 中。

```
int uv_fs_realpath(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

```
int uv_fs_realpath(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_fs_cb cb)
```

在 Unix 上等价于 `realpath(3)`。Windows 使用 `GetFinalPathNameByHandleW`。结果字符串存储在 `req->ptr` 中。

警告：此函数存在一些在 Node 中使用时发现的平台特定注意事项。

- macOS 及其他 BSD：如果在解析给定路径时发现超过 32 个符号链接，此函数将以 `UV_ELOOP` 失败。该限制是硬编码的，无法绕过。
- Windows：虽然此函数在常见情况下可以工作，但存在一些无法处理的边角情况：
 - 无法解析由绕过卷管理器的工具（如 `ImDisk`）创建的 `ramdisk` 卷中的路径。
 - 使用盘符时大小写不一致。
 - 解析出的路径会绕过 `subst` 映射的驱动器。

此函数仍然可以使用，但如果需要支持上述场景，则不推荐使用。

这些问题的背景故事和更多细节可以在[这里](#)查看。

1.8.0 新增。

```
int uv_fs_chown(...) / int uv_fs_fchown(...) / int uv_fs_lchown(...)
```

```
int uv_fs_chown(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_uid_t uid,
uv_gid_t gid, uv_fs_cb cb)
int uv_fs_fchown(uv_loop_t* loop, uv_fs_t* req, uv_file file, uv_uid_t uid, uv_g
id_t gid, uv_fs_cb cb)
int uv_fs_lchown(uv_loop_t* loop, uv_fs_t* req, const char* path, uv_uid_t uid,
uv_gid_t gid, uv_fs_cb cb)
```

分别等价于 `chown(2)`、`fchown(2)` 和 `lchown(2)`。

注意：这些函数在 Windows 上未实现。

1.21.0 变更：实现了 `uv_fs_lchown`。

```
uv_fs_type uv_fs_get_type(const uv_fs_t* req)
```

```
uv_fs_type uv_fs_get_type(const uv_fs_t* req)
```

返回 `req->fs_type`。

1.19.0 新增。

```
ssize_t uv_fs_get_result(const uv_fs_t* req)
```

```
ssize_t uv_fs_get_result(const uv_fs_t* req)
```

返回 `req->result`。

1.19.0 新增。

```
int uv_fs_get_system_error(const uv_fs_t* req)
```

```
int uv_fs_get_system_error(const uv_fs_t* req)
```

返回平台特定的错误代码——Windows 上为 `GetLastError()` 的值，其他平台上为 `-(req->result)`。

1.38.0 新增。

```
void* uv_fs_get_ptr(const uv_fs_t* req)
```

```
void* uv_fs_get_ptr(const uv_fs_t* req)
```

返回 req->ptr。

1.19.0 新增。

```
const char* uv_fs_get_path(const uv_fs_t* req)
```

```
const char* uv_fs_get_path(const uv_fs_t* req)
```

返回 req->path。

1.19.0 新增。

```
uv_stat_t* uv_fs_get_statbuf(uv_fs_t* req)
```

```
uv_stat_t* uv_fs_get_statbuf(uv_fs_t* req)
```

返回 &req->statbuf。

1.19.0 新增。

参见: `uv_req_t` 的 API 函数同样适用。

辅助函数

```
uv_os_fd_t uv_get_osfdhandle(int fd)
```

```
uv_os_fd_t uv_get_osfdhandle(int fd)
```

对于 C 运行时中的文件描述符，获取其依赖操作系统的句柄。在 UNIX 上，原样返回 `fd`。在 Windows 上，此函数调用 `_get_osfhandle`。注意返回值仍归 C 运行时所有，任何关闭它的尝试或在关闭 `fd` 之后使用它都可能导致故障。

1.12.0 新增。

```
int uv_open_osfhandle(uv_os_fd_t os_fd)
```

```
int uv_open_osfhandle(uv_os_fd_t os_fd)
```

对于依赖操作系统的句柄，获取其在 C 运行时中的文件描述符。在 UNIX 上，原样返回 `os_fd`。在 Windows 上，此函数调用 `_open_osfhandle`。注意此函数会消耗该参数，任何关闭它的尝试或在关闭返回值之后使用它都可能导致故障。

1.23.0 新增。

文件打开常量

`UV_FS_O_APPEND`

以追加模式打开文件。每次写入之前，文件偏移量都会定位到文件末尾。

`UV_FS_O_CREAT`

如果文件不存在则创建它。

`UV_FS_O_DIRECT`

文件 I/O 直接在用户空间缓冲区之间进行，缓冲区必须对齐。缓冲区大小和地址应是块设备物理扇区大小的倍数。

注意： `UV_FS_O_DIRECT` 在 Linux 上受支持，在 Windows 上通过 `FILE_FLAG_NO_BUFFERING` 支持。`UV_FS_O_DIRECT` 在 macOS 上不受支持。

UV_FS_O_DIRECTORY

如果路径不是目录，则打开失败。

注意： `UV_FS_O_DIRECTORY` 在 Windows 上不受支持。

UV_FS_O_DSYNC

以同步 I/O 方式打开文件。写入操作将在所有数据和最少量的元数据刷入磁盘后完成。

注意： `UV_FS_O_DSYNC` 在 Windows 上通过 `FILE_FLAG_WRITE_THROUGH` 支持。

UV_FS_O_EXCL

如果设置了 `O_CREAT` 标志且文件已存在，则打开失败。

注意： 一般而言，不与 `O_CREAT` 搭配使用时，`O_EXCL` 的行为是未定义的。有一个例外：在 Linux 2.6 及更高版本上，如果 `pathname` 指向块设备，`O_EXCL` 可以不与 `O_CREAT` 搭配使用。如果该块设备正被系统使用（例如已挂载），打开操作将以错误 `EBUSY` 失败。

UV_FS_O_EXLOCK

以原子方式获取独占锁。

注意： `UV_FS_O_EXLOCK` 仅在 macOS 和 Windows 上受支持。

1.17.0 变更：增加了对 Windows 的支持。

UV_FS_O_FILEMAP

使用内存文件映射来访问文件。使用此标志时，文件不能被并发地多次打开。

注意： `UV_FS_O_FILEMAP` 仅在 Windows 上受支持。

UV_FS_O_NOATIME

读取文件时不更新文件访问时间。

注意： `UV_FS_O_NOATIME` 在 Windows 上不受支持。

`UV_FS_O_NOCTTY`

如果路径标识的是终端设备，打开该路径不会使该终端成为进程的控制终端（如果进程尚无控制终端）。

注意： `UV_FS_O_NOCTTY` 在 Windows 上不受支持。

`UV_FS_O_NOFOLLOW`

如果路径是符号链接，则打开失败。

注意： `UV_FS_O_NOFOLLOW` 在 Windows 上不受支持。

`UV_FS_O_NONBLOCK`

尽可能以非阻塞模式打开文件。

注意： `UV_FS_O_NONBLOCK` 在 Windows 上不受支持。

`UV_FS_O_RANDOM`

访问意图是随机的。系统可以将其作为优化文件缓存的提示。

注意： `UV_FS_O_RANDOM` 仅在 Windows 上通过 `FILE_FLAG_RANDOM_ACCESS` 支持。

`UV_FS_O_RDONLY`

以只读方式打开文件。

`UV_FS_O_RDWR`

以读写方式打开文件。

`UV_FS_O_SEQUENTIAL`

访问意图是从头到尾的顺序读取。系统可以将其作为优化文件缓存的提示。

注意: `UV_FS_O_SEQUENTIAL` 仅在 Windows 上通过 `FILE_FLAG_SEQUENTIAL_SCAN` 支持。

`UV_FS_O_SHORT_LIVED`

文件是临时文件，应尽可能不将其刷入磁盘。

注意: `UV_FS_O_SHORT_LIVED` 仅在 Windows 上通过 `FILE_ATTRIBUTE_TEMPORARY` 支持。

`UV_FS_O_SYMLINK`

打开符号链接本身，而不是它所指向的资源。

`UV_FS_O_SYNC`

以同步 I/O 方式打开文件。写入操作将在所有数据和所有元数据刷入磁盘后完成。

注意: `UV_FS_O_SYNC` 在 Windows 上通过 `FILE_FLAG_WRITE_THROUGH` 支持。

`UV_FS_O_TEMPORARY`

文件是临时文件，应尽可能不将其刷入磁盘。

注意: `UV_FS_O_TEMPORARY` 仅在 Windows 上通过 `FILE_ATTRIBUTE_TEMPORARY` 支持。

`UV_FS_O_TRUNC`

如果文件存在且是普通文件，并且以写权限成功打开，则其长度将被截断为零。

`UV_FS_O_WRONLY`

以只写方式打开文件。

第 12 章 • API：文件监控

文件监控（FS Event 与 FS Poll）

本章译自 libuv 官方文档 `api/fs_event.rst` 与 `api/fs_poll.rst`（libuv 1.52）

FS Event 句柄（`uv_fs_event_t`）

FS Event 句柄允许用户监控给定路径的变更，例如文件被重命名，或文件内容发生了某种变化。该句柄在每个平台上都会使用最适合该任务的底层机制。

注意：对于 AIX，必须安装非默认的 IBM bos.ahafs 软件包。AIX Event Infrastructure 文件系统（ahafs）有一些限制：

- ahafs 按进程跟踪监控，且不是线程安全的。对于同一个事件，必须为每个监控器派生一个单独的进程。
- 如果只监控所在的文件夹，则不会收到文件修改（写入文件）的事件。

更多细节请参阅[文档](#)。

z/OS 的文件系统事件监控基础设施不会通知被监控目录内文件的创建/删除。更多细节请参阅 [IBM Knowledge centre](#)。

数据类型

`uv_fs_event_t`

FS Event 句柄类型。

```
void (*uv_fs_event_cb)(uv_fs_event_t* handle, const char* filename, int events, int status)
```

传给 `uv_fs_event_start` 的回调，句柄启动后将被反复调用。

如果句柄启动时给定的是一个目录，`filename` 参数将是该目录中所含文件的相对路径；如果无法确定文件名，则为 `NULL`。

`events` 参数是 `uv_fs_event` 枚举元素按位或（OR）得到的掩码。

注意：在 FreeBSD 上，由于一个内核缺陷，`filename` 有时可能为 `NULL`。参见 [FreeBSD bug 参考](#)。

`uv_fs_event`

`uv_fs_event_t` 句柄监控的事件类型。

```
enum uv_fs_event {
    UV_RENAME = 1,
    UV_CHANGE = 2
};
```

`uv_fs_event_flags`

可以传给 `uv_fs_event_start` 以控制其行为的标志。

```
enum uv_fs_event_flags {
    /*
     * 默认情况下，如果 fs_event 监视器被给定一个目录名，我们将监视该目录中的
     * 所有事件。此标志会覆盖这一行为，使 fs_event 只报告目录项本身的变更。
     * 此标志不影响被监视的各个单独文件。
     * 此标志目前在任何后端上均未实现。
     */
    UV_FS_EVENT_WATCH_ENTRY = 1,
    /*
     * 默认情况下 uv_fs_event 会尝试使用内核接口（如 inotify 或 kqueue）来检测
     * 事件。这在远程文件系统（如 NFS 挂载）上可能不可用。此标志使 fs_event 回退
     * 到以固定间隔调用 stat()。
     * 此标志目前在任何后端上均未实现。
     */
    UV_FS_EVENT_STAT = 2,
    /*
     * 默认情况下，事件监视器在监视目录时，不会登记（会忽略）其子目录中的变更。
     * 在支持的平台上，此标志会覆盖这一行为。
     */
    UV_FS_EVENT_RECURSIVE = 4
};
```

公开成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

```
int uv_fs_event_init(uv_loop_t* loop, uv_fs_event_t* handle)
```

```
int uv_fs_event_init(uv_loop_t* loop, uv_fs_event_t* handle)
```

初始化句柄。

```
int uv_fs_event_start(uv_fs_event_t* handle, uv_fs_event_cb cb, const char* path,
unsigned int flags)
```

```
int uv_fs_event_start(uv_fs_event_t* handle, uv_fs_event_cb cb, const char*
path, unsigned int flags)
```

以给定回调启动句柄，该回调将监视指定 `path` 的变更。`flags` 可以是 `uv_fs_event_flags` 枚举元素按位或得到的掩码。

注意：目前唯一支持的标志是 `UV_FS_EVENT_RECURSIVE`，且仅在 `macOS (OSX)` 和 `Windows` 上可用。

注意：在 `macOS` 上，在调用 `uv_fs_event_start` 之前紧邻时刻由操作系统收集到的事件可能会报告给 `uv_fs_event_cb` 回调。

```
int uv_fs_event_stop(uv_fs_event_t* handle)
```

```
int uv_fs_event_stop(uv_fs_event_t* handle)
```

停止句柄，回调将不再被调用。

```
int uv_fs_event_getpath(uv_fs_event_t* handle, char* buffer, size_t* size)
```

```
int uv_fs_event_getpath(uv_fs_event_t* handle, char* buffer, size_t* size)
```

获取句柄正在监控的路径。缓冲区必须由用户预先分配。成功返回 0，失败返回错误码 `< 0`。成功时，`buffer` 将包含路径，`size` 为其长度。如果缓冲区不够大，将返回 `UV_ENOBUFS`，且 `size` 会被设置为所需大小（含 null 终止符）。

1.3.0 变更：返回的长度不再包含结尾的 null 字节，且缓冲区不以 null 结尾。

1.9.0 变更：返回 `UV_ENOBUFS` 时返回的长度包含结尾的 null 字节，成功时缓冲区以 null 结尾。

参见：`uv_handle_t` 的 API 函数同样适用。

FS Poll 句柄 (`uv_fs_poll_t`)

FS Poll 句柄允许用户监控给定路径的变更。与 `uv_fs_event_t` 不同，fs poll 句柄使用 `stat` 来检测文件是否发生变化，因此可以在 fs event 句柄无法工作的文件系统上工作。

数据类型

`uv_fs_poll_t`

FS Poll 句柄类型。

```
void (*uv_fs_poll_cb)(uv_fs_poll_t* handle, int status, const uv_stat_t* prev, const uv_stat_t* curr)
```

传给 `uv_fs_poll_start` 的回调，句柄启动后，每当被监控的路径发生任何变化时就会被调用。

如果 `path` 不存在或不可访问，回调将以 `status < 0` 被调用。监视器不会停止，但在情况发生变化之前（例如文件被创建，或错误原因发生改变），回调不会再被调用。

当 `status == 0` 时，回调会收到指向新旧 `uv_stat_t` 结构体的指针。它们仅在回调执行期间有效。

公开成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

```
int uv_fs_poll_init(uv_loop_t* loop, uv_fs_poll_t* handle)
```

```
int uv_fs_poll_init(uv_loop_t* loop, uv_fs_poll_t* handle)
```

初始化句柄。

```
int uv_fs_poll_start(uv_fs_poll_t* handle, uv_fs_poll_cb poll_cb, const char* path,
unsigned int interval)
```

```
int uv_fs_poll_start(uv_fs_poll_t* handle, uv_fs_poll_cb poll_cb, const char*
path, unsigned int interval)
```

每隔 `interval` 毫秒检查 `path` 处的文件是否有变化。

注意：为获得最大可移植性，请使用多秒的间隔。亚秒间隔在许多文件系统上无法检测到所有变更。

```
int uv_fs_poll_stop(uv_fs_poll_t* handle)
```

```
int uv_fs_poll_stop(uv_fs_poll_t* handle)
```

停止句柄，回调将不再被调用。

```
int uv_fs_poll_getpath(uv_fs_poll_t* handle, char* buffer, size_t* size)
```

```
int uv_fs_poll_getpath(uv_fs_poll_t* handle, char* buffer, size_t* size)
```

获取句柄正在监控的路径。缓冲区必须由用户预先分配。成功返回 0，失败返回错误码 < 0。成功时，`buffer` 将包含路径，`size` 为其长度。如果缓冲区不够大，将返回 `UV_ENOBUFS`，且 `size` 会被设置为所需大小。

1.3.0 变更：返回的长度不再包含结尾的 null 字节，且缓冲区不以 null 结尾。

1.9.0 变更：返回 `UV_ENOBUFS` 时返回的长度包含结尾的 null 字节，成功时缓冲区以 null 结尾。

参见：`uv_handle_t` 的 API 函数同样适用。

第 13 章 • API: DNS、线程池与指标

DNS、线程池、指标与版本 (DNS utility functions、Thread pool、Metrics、Version)

本章译自 `libuv` 官方文档 `api/dns.rst`、`api/threadpool.rst`、`api/metrics.rst`、`api/version.rst` (`libuv 1.52`)

DNS 工具函数 (DNS utility functions)

`libuv` 提供了 `getaddrinfo` 和 `getnameinfo` 的异步版本。

数据类型

`uv_getaddrinfo_t`

`getaddrinfo` 请求类型。

```
void (*uv_getaddrinfo_cb)(uv_getaddrinfo_t* req, int status, struct addrinfo* res)
```

域名解析请求完成后携带结果调用的回调。如果请求被取消，`status` 的值将为 `UV_ECANCELED`。

`uv_getnameinfo_t`

`getnameinfo` 请求类型。

```
void (*uv_getnameinfo_cb)(uv_getnameinfo_t* req, int status, const char* hostname, const char* service)
```

名称解析请求完成后携带结果调用的回调。如果请求被取消，`status` 的值将为 `UV_ECANCELED`。

公开成员

`uv_loop_t* uv_getaddrinfo_t.loop`

启动该域名解析请求并在其中报告完成事件的循环。只读。

`struct addrinfo* uv_getaddrinfo_t.addrinfo`

指向包含结果的 `struct addrinfo` 的指针。必须由用户使用 `uv_freeaddrinfo` 释放。

1.3.0 变更：该字段被声明为公开。

`uv_loop_t* uv_getnameinfo_t.loop`

启动该名称解析请求并在其中报告完成事件的循环。只读。

`char[NI_MAXHOST] uv_getnameinfo_t.host`

包含结果主机名的字符数组。以 null 结尾。

1.3.0 变更：该字段被声明为公开。

`char[NI_MAXSERV] uv_getnameinfo_t.service`

包含结果服务名的字符数组。以 null 结尾。

1.3.0 变更：该字段被声明为公开。

参见： `uv_req_t` 的成员同样适用。

API

```
int uv_getaddrinfo(uv_loop_t* loop, uv_getaddrinfo_t* req, uv_getaddrinfo_cb
getaddrinfo_cb, const char* node, const char* service, const struct addrinfo*
hints)
```

```
int uv_getaddrinfo(uv_loop_t* loop, uv_getaddrinfo_t* req, uv_getaddrinfo_cb
getaddrinfo_cb, const char* node, const char* service, const struct addrinfo* hi
nts)
```

异步的 `getaddrinfo(3)`。

`node` 和 `service` 可以有一个为 `NULL`，但不能都为 `NULL`。

`hints` 是指向 `struct addrinfo` 的指针，包含额外的地址类型约束，也可以为 `NULL`。更多细节请查阅 `man -s 3 getaddrinfo`。

成功返回 0，失败返回错误码 < 0 。如果成功，回调将在未来的某个时刻被调用，并携带解析结果，结果为以下二者之一：

- `status == 0`，`res` 参数指向一个有效的 `struct addrinfo`；或
- `status < 0`，`res` 参数为 `NULL`。参见 `UV_EAI_*` 常量。

调用 `uv_freeaddrinfo` 释放 `addrinfo` 结构。

1.3.0 变更：回调参数现在允许为 `NULL`，此时请求将同步运行。

```
void uv_freeaddrinfo(struct addrinfo* ai)
```

```
void uv_freeaddrinfo(struct addrinfo* ai)
```

释放 `struct addrinfo`。允许传入 `NULL`，此时为无操作。

```
int uv_getnameinfo(uv_loop_t* loop, uv_getnameinfo_t* req, uv_getnameinfo_cb
getnameinfo_cb, const struct sockaddr* addr, int flags)
```

```
int uv_getnameinfo(uv_loop_t* loop, uv_getnameinfo_t* req, uv_getnameinfo_cb
getnameinfo_cb, const struct sockaddr* addr, int flags)
```

异步的 `getnameinfo(3)`。

成功返回 0，失败返回错误码 < 0 。如果成功，回调将在未来的某个时刻被调用，并携带解析结果。更多细节请查阅 `man -s 3 getnameinfo`。

1.3.0 变更：回调参数现在允许为 NULL，此时请求将**同步**运行。

参见： `uv_req_t` 的 API 函数同样适用。

线程池工作调度 (Thread pool work scheduling)

libuv 提供了一个线程池，可用于运行用户代码并在循环线程中得到通知。该线程池在内部用于运行所有文件系统操作，以及域名解析 (`getaddrinfo`) 和名称解析 (`getnameinfo`) 请求。

其默认大小为 4，但可以在启动时通过将 `UV_THREADPOOL_SIZE` 环境变量设置为任意值来更改（绝对上限为 1024）。

1.30.0 变更：允许的 `UV_THREADPOOL_SIZE` 最大值从 128 提高到 1024。

1.45.0 变更：线程现在拥有 8 MB 的栈，而不是（有时过低的）平台默认值。

1.50.0 变更：线程现在拥有默认名称 `libuv-worker`。

线程池是全局的，在所有事件循环之间共享。当某个特定函数使用线程池时（例如使用 `uv_queue_work`），libuv 会预分配并初始化 `UV_THREADPOOL_SIZE` 允许的最大线程数。更多的线程通常意味着更高的吞吐量，但也会带来更高的内存占用。不过在大多数平台上，线程栈是惰性增长的。

注意：请注意，尽管使用的是跨所有事件循环共享的全局线程池，这些函数本身并不是线程安全的。

数据类型

`uv_work_t`

工作请求类型。

```
void (*uv_work_cb)(uv_work_t* req)
```

传给 `uv_queue_work` 的回调，将在线程池上运行。

```
void (*uv_after_work_cb)(uv_work_t* req, int status)
```

传给 `uv_queue_work` 的回调，在线程池上的工作完成后于循环线程中被调用。如果工作通过 `uv_cancel` 被取消，`status` 将为 `UV_ECANCELED`。

公开成员

```
uv_loop_t* uv_work_t.loop
```

启动该请求并在其中报告完成事件的循环。只读。

参见： `uv_req_t` 的成员同样适用。

API

```
int uv_queue_work(uv_loop_t* loop, uv_work_t* req, uv_work_cb work_cb,
uv_after_work_cb after_work_cb)
```

```
int uv_queue_work(uv_loop_t* loop, uv_work_t* req, uv_work_cb work_cb,
uv_after_work_cb after_work_cb)
```

初始化一个工作请求，它将在线程池中的某个线程上运行给定的 `work_cb`。 `work_cb` 完成后， `after_work_cb` 将在循环线程中被调用。

该请求可以通过 `uv_cancel` 取消。

参见： `uv_req_t` 的 API 函数同样适用。

指标操作 (Metrics operations)

libuv 提供了一套指标 API，用于跟踪事件循环的各种内部操作。

数据类型

`uv_metrics_t`

包含事件循环指标的结构体。建议在 `uv_prepare_cb` 中获取这些指标，以确保指标计数器之间不存在不一致。

```
typedef struct {
    uint64_t loop_count;
    uint64_t events;
    uint64_t events_waiting;
    /* private */
    uint64_t* reserved[13];
} uv_metrics_t;
```

公开成员

`uint64_t uv_metrics_t.loop_count`

事件循环迭代的次数。

`uint64_t uv_metrics_t.events`

事件处理器已处理的事件数量。

```
uint64_t uv_metrics_t.events_waiting
```

调用事件提供者（event provider）时正等待处理的事件数量。

API

```
uint64_t uv_metrics_idle_time(uv_loop_t* loop)
```

```
uint64_t uv_metrics_idle_time(uv_loop_t* loop)
```

获取事件循环在内核事件提供者（例如 `epoll_wait`）中处于空闲状态的时长。该调用是线程安全的。

返回值是自 `uv_loop_t` 被配置为收集空闲时间以来，在内核事件提供者中累计的空闲时间。

注意：在以 `UV_METRICS_IDLE_TIME` 调用 `uv_loop_configure` 之前，事件循环不会开始累计事件提供者的空闲时间。

1.39.0 新增。

```
int uv_metrics_info(uv_loop_t* loop, uv_metrics_t* metrics)
```

```
int uv_metrics_info(uv_loop_t* loop, uv_metrics_t* metrics)
```

将当前的事件循环指标集合复制到 `metrics` 指针。

1.45.0 新增。

版本检查宏与函数（Version-checking macros and functions）

从 1.0.0 版本开始，libuv 遵循语义化版本（semantic versioning）方案。这意味着在一个主版本的生命周期内可以引入新 API。本节列出了所有可用于条件编译或条件执行代码的宏和函数，以便兼容多个 libuv 版本。

宏

`UV_VERSION_MAJOR`

libuv 版本的主版本号。

`UV_VERSION_MINOR`

libuv 版本的次版本号。

`UV_VERSION_PATCH`

libuv 版本的修订号。

`UV_VERSION_IS_RELEASE`

设为 1 表示 libuv 的正式发布版本，0 表示开发快照。

`UV_VERSION_SUFFIX`

libuv 版本后缀。某些开发版本（如发布候选版本）可能带有诸如 "rc" 的后缀。

`UV_VERSION_HEX`

返回打包为单个整数的 libuv 版本。每个组件占 8 位，修订号存储在最低的 8 位中。例如 libuv 1.2.3 对应 0x010203。

1.7.0 新增。

函数

`unsigned int uv_version(void)`

```
unsigned int uv_version(void)
```

返回 `UV_VERSION_HEX`。

```
const char* uv_version_string(void)
```

```
const char* uv_version_string(void)
```

以字符串形式返回 libuv 版本号。对于非正式发布版本，版本后缀会被包含在内。

第 14 章 • API: TCP 与 UDP

uv_tcp_t — TCP 句柄

本章译自 libuv 官方文档 `api/tcp.rst` (libuv 1.52)

TCP 句柄用于表示 TCP 流和 TCP 服务器。

`uv_tcp_t` 是 `uv_stream_t` 的"子类"。

数据类型

`uv_tcp_t`

TCP 句柄类型。

`uv_tcp_flags`

`uv_tcp_bind` 中使用的标志位。

```
enum uv_tcp_flags {
    /* 用于 uv_tcp_bind, 当使用 IPv6 地址时。 */
    UV_TCP_IPV6ONLY = 1,

    /* 绑定句柄时启用 SO_REUSEPORT 套接字选项。
     * 如果多个进程或线程在绑定端口之前都设置了 SO_REUSEPORT,
     * 则允许它们进行完全重复的绑定。
     * 传入的连接会分布到参与各个监听套接字上。
     *
     * 此标志目前仅在 Linux 3.9+、DragonFlyBSD 3.6+、
     * FreeBSD 12.0+、Solaris 11.4 和 AIX 7.2.5+ 上可用。
     */
    UV_TCP_REUSEPORT = 2,
};
```

公开成员

无。

参见： `uv_stream_t` 的成员同样适用。

API

`int uv_tcp_init(uv_loop_t loop, uv_tcp_t handle)`

初始化句柄。此时还不会创建套接字。

`int uv_tcp_init_ex(uv_loop_t loop, uv_tcp_t handle, unsigned int flags)`

以指定的标志初始化句柄。目前 `flags` 参数只有低 8 位被用作套接字域 (socket domain)。会为给定的域创建一个套接字。如果指定的域是 `AF_UNSPEC`，则不创建套接字，行为与 `uv_tcp_init` 相同。

自 1.7.0 版本新增

`int uv_tcp_open(uv_tcp_t* handle, uv_os_sock_t sock)`

将现有的文件描述符或 SOCKET 作为 TCP 句柄打开。

1.2.1 版本变更：文件描述符会被设置为非阻塞模式。

注意：传入的文件描述符或 SOCKET 不会做类型检查，但要求它必须代表一个有效的流式套接字。

`int uv_tcp_nodelay(uv_tcp_t* handle, int enable)`

启用 `TCP_NODELAY`，即禁用 Nagle 算法。

`int uv_tcp_keepalive(uv_tcp_t* handle, int enable, unsigned int delay)`

启用/禁用 TCP keep-alive。`delay` 是以秒为单位的初始延迟，当 `enable` 为零时被忽略。

在 `delay` 时间到达后，还会进行 10 次连续的探测，每次探测间隔 1 秒。如果在此过程结束时连接仍然丢失，则句柄被销毁，并向相应的回调传入 `UV_ETIMEDOUT` 错误。

如果 `delay` 小于 1，则返回 `UV_EINVAL`。

1.49.0 版本变更：如果 `delay` 小于 1 则返回 `UV_EINVAL`。

int uv_tcp_keepalive_ex(uv_tcp_t* handle, int on, unsigned int idle, unsigned int intvl, unsigned int cnt)

启用/禁用 TCP keep-alive，并可设置全部套接字选项：TCP_KEEPIDLE、TCP_KEEPINTVL 和 TCP_KEEPCNT。idle 是 TCP_KEEPIDLE 的值，intvl 是 TCP_KEEPINTVL 的值，cnt 是 TCP_KEEPCNT 的值，当 on 为零时被忽略。

启用 TCP keep-alive 后，idle 是连接在 TCP 开始发送 keep-alive 探测之前需要保持空闲的时间（秒）。intvl 是各次 keep-alive 探测之间的间隔时间（秒）。TCP 在发送 cnt 次探测仍未收到对端任何回应后会断开连接，随后句柄被销毁，并向相应的回调传入 UV_ETIMEDOUT 错误。

如果 idle、intvl 或 cnt 中任何一个小于 1，则返回 UV_EINVAL。

1.52.0 版本变更：新增对设置 TCP_KEEPINTVL 和 TCP_KEEPCNT 套接字选项的支持。

注意：请确保底层操作系统支持这些套接字选项。当前支持的平台：

- AIX
- DragonFlyBSD
- FreeBSD
- illumos
- Linux
- macOS
- NetBSD
- Solaris
- Windows

int uv_tcp_simultaneous_accepts(uv_tcp_t* handle, int enable)

启用/禁用操作系统在监听新 TCP 连接时排队的同步异步接受请求（simultaneous asynchronous accept requests）。

此设置用于将 TCP 服务器调优到期望的性能。启用同时接受（simultaneous accepts）可以显著提高接受连接的速率（这也是它默认启用的原因），但在多进程部署中可能导致负载分布不均。

int uv_tcp_bind(uv_tcp_t handle, const struct sockaddr addr, unsigned int flags)

将句柄绑定到地址和端口。

当端口已被占用时，你可能会从 `uv_listen` 或 `uv_tcp_connect` 收到 `UV_EADDRINUSE` 错误，除非你为所有绑定的套接字在 `flags` 中指定了 `UV_TCP_REUSEPORT`。也就是说，此函数调用成功并不保证随后对 `uv_listen` 或 `uv_tcp_connect` 的调用也会成功。

参数

- `handle`：TCP 句柄。应已通过 `uv_tcp_init` 初始化。
- `addr`：要绑定到的地址。它应指向一个已初始化的 `struct sockaddr_in` 或 `struct sockaddr_in6`。
- `flags`：控制套接字绑定行为的标志。`flags` 中可以包含 `UV_TCP_IPV6ONLY` 以禁用双栈支持，仅使用 IPv6。`flags` 中可以包含 `UV_TCP_REUSEPORT` 以启用 `SO_REUSEPORT` 套接字选项，使其具备负载均衡能力，将传入连接分布到多个进程或线程的所有监听套接字上。

返回值：成功返回 0，失败返回小于 0 的错误码。

1.49.0 版本变更：新增 `UV_TCP_REUSEPORT` 标志。

注意：`UV_TCP_REUSEPORT` 标志目前仅在 Linux 3.9+、DragonFlyBSD 3.6+、FreeBSD 12.0+、Solaris 11.4 和 AIX 7.2.5+ 上可用。在其他平台上此函数将返回 `UV_ENOTSUP` 错误。

int uv_tcp_getsockname(const uv_tcp_t handle, struct sockaddr name, int* namelen)

获取句柄当前绑定到的地址。`name` 必须指向一段有效且足够大的内存，推荐使用 `struct sockaddr_storage` 以同时支持 IPv4 和 IPv6。

int uv_tcp_getpeername(const uv_tcp_t handle, struct sockaddr name, int* namelen)

获取连接到该句柄的对端地址。`name` 必须指向一段有效且足够大的内存，推荐使用 `struct sockaddr_storage` 以同时支持 IPv4 和 IPv6。

int uv_tcp_connect(uv_connect_t req, uv_tcp_t handle, const struct sockaddr* addr, uv_connect_cb cb)

建立 IPv4 或 IPv6 的 TCP 连接。需提供一个已初始化的 TCP 句柄和一个未初始化的 `uv_connect_t`。 `addr` 应指向一个已初始化的 `struct sockaddr_in` 或 `struct sockaddr_in6`。

在 Windows 上, 如果 `addr` 被初始化为指向未指定地址 (`0.0.0.0` 或 `::`) , 它会被改为指向 `localhost`。这样做是为了与 Linux 系统的行为保持一致。

当连接建立完成或发生连接错误时, 回调被调用。

1.19.0 版本变更: 新增 `0.0.0.0` 和 `::` 到 `localhost` 的映射。

参见: `uv_stream_t` 的 API 函数同样适用。

int uv_tcp_close_reset(uv_tcp_t* handle, uv_close_cb close_cb)

通过发送 RST 包重置 TCP 连接。这是通过将 `SO_LINGER` 套接字选项的 `linger` 间隔设为 0, 然后调用 `uv_close` 来实现的。由于某些平台上的行为不一致, 不允许混用 `uv_shutdown` 和 `uv_tcp_close_reset` 调用。

自 1.32.0 版本新增

int uv_socketpair(int type, int protocol, uv_os_sock_t socket_vector[2], int flags0, int flags1)

创建一对具有指定属性的已连接套接字。得到的句柄可以传给 `uv_tcp_open`、用于 `uv_spawn`, 或用于其他任何用途。

`flags0` 和 `flags1` 的有效值为:

- `UV_NONBLOCK_PIPE`: 以 `OVERLAPPED` 或 `FIONBIO / O_NONBLOCK` I/O 方式打开指定的套接字句柄。对于将被 libuv 使用的句柄, 推荐这样做; 其他情况通常不建议。

等价于域为 `AF_UNIX` 的 `socketpair(2)`。

自 1.41.0 版本新增

uv_udp_t — UDP 句柄

本章译自 libuv 官方文档 `api/udp.rst` (libuv 1.52)

UDP 句柄为客户端和服务端封装 UDP 通信。

数据类型

uv_udp_t

UDP 句柄类型。

uv_udp_send_t

UDP 发送请求类型。

uv_udp_flags

`uv_udp_bind` 和 `uv_udp_recv_cb` 中使用的标志位。

```

enum uv_udp_flags {
    /* 禁用双栈模式。 */
    UV_UDP_IPV6ONLY = 1,
    /*
     * 表示消息因读取缓冲区太小而被截断。剩余部分已被操作系统丢弃。
     * 在 uv_udp_recv_cb 中使用。
     */
    UV_UDP_PARTIAL = 2,
    /*
     * 表示绑定句柄时是否设置 SO_REUSEADDR。
     * 在 BSD 系统 (DragonFlyBSD 除外)、OS X 以及其他 SO_REUSEPORT
     * 不具备负载均衡能力的平台上, 这会设置 SO_REUSEPORT 套接字标志,
     * 与 UV_UDP_REUSEPORT 的作用相反。在其他 Unix 平台上, 它设置
     * SO_REUSEADDR 标志。这意味着多个线程或进程可以绑定到同一地址
     * 而不出错 (前提是它们都设置了该标志), 但只有最后绑定的那个会
     * 收到流量, 实际上是从之前的监听者那里 "偷走" 了端口。
     */
    UV_UDP_REUSEADDR = 4,
    /*
     * 表示消息是通过 recvmsg 接收的, 因此提供的缓冲区
     * 不得由 recv_cb 回调释放。
     */
    UV_UDP_MMSG_CHUNK = 8,
    /*
     * 表示提供的缓冲区已被 recvmsg 完全使用, 现在应由 recv_cb
     * 回调释放。当在 uv_udp_recv_cb 中设置此标志时, nread 总是 0,
     * addr 总是 NULL。
     */
    UV_UDP_MMSG_FREE = 16,
    /*
     * 表示绑定句柄时是否设置 IP_RECVERR/IPV6_RECVERR。
     * 在 Linux 上, 这会为 IPv4 UDP 套接字设置 IP_RECVERR, 为 IPv6 UDP
     * 套接字设置 IPV6_RECVERR。这可以阻止 Linux 内核抑制某些 ICMP 错误
     * 消息, 并启用完整的 ICMP 错误报告以加快故障转移。
     * 此标志在 Linux 以外的平台上是空操作。
     */
    UV_UDP_LINUX_RECVERR = 32,
    /*
     * 表示绑定句柄时是否设置 SO_REUSEPORT。
     * 这会在支持的平台上设置 SO_REUSEPORT 套接字选项。
     * 与 UV_UDP_REUSEADDR 不同, 此标志会让绑定到相同地址和端口的
     * 多个线程或进程 "共享" 该端口, 也就是说传入的数据报会分布到
     * 各线程或进程的接收套接字上。
     *
     * 此标志目前仅在 Linux 3.9+、DragonFlyBSD 3.6+、
     * FreeBSD 12.0+、Solaris 11.4 和 AIX 7.2.5+ 上可用。
     */
}

```

```

    */
    UV_UDP_REUSEPORT = 64,
    /*
     * 表示应使用 recvmsg (如果可用)。此句柄的 uv_alloc_cb
     * 应创建以 64 KiB 为倍数大小的缓冲区。
     */
    UV_UDP_RECVMSG = 256
};

```

void (uv_udp_send_cb)(uv_udp_send_t req, int status)

传给 `uv_udp_send` 的回调的类型定义，在数据发送完成后被调用。

void (uv_udp_recv_cb)(uv_udp_t handle, ssize_t nread, const uv_buf_t buf, const struct sockaddr addr, unsigned flags)

传给 `uv_udp_recv_start` 的回调的类型定义，当端点收到数据时被调用。

- `handle`：UDP 句柄
- `nread`：已接收的字节数。如果没有更多数据可读则为 0。注意 0 也可能表示收到了一个空数据报（此时 `addr` 不为 NULL）。如果检测到传输错误则 < 0 ；如果使用 `recvmsg(2)`，则不会再收到更多分块，缓冲区可以被安全释放。
- `buf`：包含已接收数据的 `uv_buf_t`。
- `addr`：包含发送方地址的 `struct sockaddr*`。可以为 NULL。仅在回调执行期间有效。
- `flags`：一个或多个按位或组合的 `UV_UDP_*` 常量。

被调用方负责释放缓冲区，`libuv` 不会复用它。出错时缓冲区可能是一个空缓冲区（即 `buf->base == NULL` 且 `buf->len == 0`）。

使用 `recvmsg(2)` 时，分块会带有 `UV_UDP_MMSG_CHUNK` 标志，这些分块不得被释放。如果没有发生错误，最后会以 `nread` 为 0、`addr` 为 NULL 的形式再调用一次回调，缓冲区指向最初分配的数据，`UV_UDP_MMSG_CHUNK` 标志被清除而 `UV_UDP_MMSG_FREE` 标志被设置。如果发生 UDP 套接字错误，`nread` 将 < 0 。无论哪种情况，被调用方此时都可以安全地释放提供的缓冲区。

1.40.0 版本变更：新增 `UV_UDP_MMSG_FREE` 标志。

注意：当没有内容可读时，接收回调会以 `nread == 0` 且 `addr == NULL` 被调用；当收到一个空的 UDP 包时，则以 `nread == 0` 且 `addr != NULL` 被调用。

uv_membership

多播地址的成员关系类型。

```
typedef enum {
    UV_LEAVE_GROUP = 0,
    UV_JOIN_GROUP
} uv_membership;
```

公开成员

size_t uv_udp_t.send_queue_size

已排队等待发送的字节数。此字段严格反映当前排队的信息量。

size_t uv_udp_t.send_queue_count

当前在队列中等待处理的发送请求数量。

uv_udp_t* uv_udp_send_t.handle

此发送请求所在的 UDP 句柄。

参见： `uv_handle_t` 的成员同样适用。

API

int uv_udp_init(uv_loop_t loop, uv_udp_t handle)

初始化一个新的 UDP 句柄。实际的套接字是惰性创建的。成功返回 0。

int uv_udp_init_ex(uv_loop_t loop, uv_udp_t handle, unsigned int flags)

以指定的标志初始化句柄。`flags` 参数的低 8 位用作套接字域。会为给定的域创建套接字。如果指定的域是 `AF_UNSPEC`，则不创建套接字，行为与 `uv_udp_init` 相同。

其余位可用于设置以下标志之一：

- `UV_UDP_RECVMSG`：如果设置且平台支持，将使用 `recvmsg(2)`。此句柄的 `uv_alloc_cb` 应创建以 64 KiB 为倍数大小的缓冲区。

自 1.7.0 版本新增

1.37.0 版本变更：新增 `UV_UDP_RECVMSG` 标志。

int uv_udp_open_ex(uv_udp_t* handle, uv_os_sock_t sock, unsigned int flags)

将现有的文件描述符或 Windows SOCKET 作为 UDP 句柄打开。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `sock`：要与句柄关联的现有套接字。
- `flags`：控制套接字行为的标志，支持 `UV_UDP_REUSEADDR` 和 `UV_UDP_REUSEPORT`。

返回值：成功返回 0，失败返回小于 0 的错误码。

自 1.52.0 版本新增

int uv_udp_open(uv_udp_t* handle, uv_os_sock_t sock)

将现有的文件描述符或 Windows SOCKET 作为 UDP 句柄打开。

仅限 Unix：对 `sock` 参数的唯一要求是它遵循数据报契约（可在未连接模式下工作、支持 `sendmsg()/recvmsg()` 等）。换句话说，其他数据报类型的套接字如原始套接字（raw socket）或 netlink 套接字也可以传给此函数。

1.2.1 版本变更：文件描述符会被设置为非阻塞模式。

注意：传入的文件描述符或 SOCKET 不会做类型检查，但要求它必须代表一个有效的数据报套接字。

内部会无条件设置 `SO_REUSEADDR` 套接字选项。这意味着复用标志总是被启用，无论用户意图如何。如需更多控制，请使用 `uv_udp_open_ex`。

int uv_udp_bind(uv_udp_t handle, const struct sockaddr addr, unsigned int flags)

将 UDP 句柄绑定到 IP 地址和端口。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `addr`：包含要绑定的地址和端口的 `struct sockaddr_in` 或 `struct sockaddr_in6`。
- `flags`：指示套接字的绑定方式，支持 `UV_UDP_IPV6ONLY`、`UV_UDP_REUSEADDR`、`UV_UDP_REUSEPORT` 和 `UV_UDP_RECVERR`。

返回值：成功返回 0，失败返回小于 0 的错误码。

1.49.0 版本变更：新增 `UV_UDP_REUSEPORT` 标志。

注意：`UV_UDP_REUSEPORT` 标志目前仅在 Linux 3.9+、DragonFlyBSD 3.6+、FreeBSD 12.0+、Solaris 11.4 和 AIX 7.2.5+ 上可用。在其他平台上此函数将返回 `UV_ENOTSUP` 错误。对于 `SO_REUSEPORT` 具备负载均衡能力的平台，允许在 `flags` 中同时指定 `UV_UDP_REUSEADDR` 和 `UV_UDP_REUSEPORT`，且 `SO_REUSEPORT` 总是覆盖 `SO_REUSEADDR` 的行为。对于 `SO_REUSEPORT` 不具备负载均衡能力的平台，在 `flags` 中同时指定 `UV_UDP_REUSEADDR` 和 `UV_UDP_REUSEPORT` 会失败，返回 `UV_ENOTSUP` 错误。

`int uv_udp_connect(uv_udp_t handle, const struct sockaddr addr)`

将 UDP 句柄关联到远程地址和端口，此后该句柄发出的每条消息都会自动发送到该目的地。以 `NULL` 的 `addr` 调用此函数会断开句柄的连接。对已连接的句柄再次调用 `uv_udp_connect()` 会导致 `UV_EISCONN` 错误。尝试断开一个未连接的句柄会返回 `UV_ENOTCONN` 错误。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `addr`：包含要关联的地址和端口的 `struct sockaddr_in` 或 `struct sockaddr_in6`。

返回值：成功返回 0，失败返回小于 0 的错误码。

自 1.27.0 版本新增

int uv_udp_getpeername(const uv_udp_t handle, struct sockaddr name, int* namelen)

在已连接的 UDP 句柄上获取其远程 IP 和端口。在未连接的句柄上，返回 `UV_ENOTCONN`。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化并绑定。
- `name`：指向将被填入地址数据的结构的指针。为支持 IPv4 和 IPv6，应使用 `struct sockaddr_storage`。
- `namelen`：输入时表示 `name` 字段的容量；输出时表示实际填充了多少。

返回值：成功返回 0，失败返回小于 0 的错误码。

自 1.27.0 版本新增

int uv_udp_getsockname(const uv_udp_t handle, struct sockaddr name, int* namelen)

获取 UDP 句柄的本地 IP 和端口。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化并绑定。
- `name`：指向将被填入地址数据的结构的指针。为支持 IPv4 和 IPv6，应使用 `struct sockaddr_storage`。
- `namelen`：输入时表示 `name` 字段的容量；输出时表示实际填充了多少。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_set_membership(uv_udp_t handle, const char multicast_addr, const char* interface_addr, uv_membership membership)

设置多播地址的成员关系。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `multicast_addr`：要设置成员关系的多播地址。
- `interface_addr`：接口地址。

- `membership`：应为 `UV_JOIN_GROUP` 或 `UV_LEAVE_GROUP`。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_set_source_membership(uv_udp_t handle, const char multicast_addr, const char interface_addr, const char source_addr, uv_membership membership)

设置特定源多播组（source-specific multicast group）的成员关系。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `multicast_addr`：要设置成员关系的多播地址。
- `interface_addr`：接口地址。
- `source_addr`：源地址。
- `membership`：应为 `UV_JOIN_GROUP` 或 `UV_LEAVE_GROUP`。

返回值：成功返回 0，失败返回小于 0 的错误码。

自 1.32.0 版本新增

int uv_udp_set_multicast_loop(uv_udp_t* handle, int on)

设置 IP 多播回环标志。使多播数据包回环到本地套接字。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init_ex` 以 `AF_INET` 或 `AF_INET6` 初始化，或已通过 `uv_udp_bind` 显式绑定到地址，或已通过 `uv_udp_send` 或 `uv_udp_recv_start` 隐式绑定。
- `on`：1 表示开启，0 表示关闭。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_set_multicast_ttl(uv_udp_t* handle, int ttl)

设置多播 TTL。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init_ex` 以 `AF_INET` 或 `AF_INET6` 初始化，或已通过 `uv_udp_bind` 显式绑定到地址，或已通过 `uv_udp_send` 或 `uv_udp_recv_start` 隐式绑定。
- `ttl`：1 到 255。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_set_multicast_interface(uv_udp_t handle, const char interface_addr)

设置用于发送或接收数据的多播接口。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init_ex` 以 `AF_INET` 或 `AF_INET6` 初始化，或已通过 `uv_udp_bind` 显式绑定到地址，或已通过 `uv_udp_send` 或 `uv_udp_recv_start` 隐式绑定。
- `interface_addr`：接口地址。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_set_broadcast(uv_udp_t* handle, int on)

开启或关闭广播。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init_ex` 以 `AF_INET` 或 `AF_INET6` 初始化，或已通过 `uv_udp_bind` 显式绑定到地址，或已通过 `uv_udp_send` 或 `uv_udp_recv_start` 隐式绑定。
- `on`：1 表示开启，0 表示关闭。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_set_ttl(uv_udp_t* handle, int ttl)

设置生存时间 (time to live) 。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init_ex` 以 `AF_INET` 或 `AF_INET6` 初始化，或已通过 `uv_udp_bind` 显式绑定到地址，或已通过 `uv_udp_send` 或 `uv_udp_recv_start` 隐式绑定。
- `ttl`：1 到 255。

返回值：成功返回 0，失败返回小于 0 的错误码。

int uv_udp_send(uv_udp_send_t req, uv_udp_t handle, const uv_buf_t bufs[], unsigned int nbufs, const struct sockaddr* addr, uv_udp_send_cb send_cb)

通过 UDP 套接字发送数据。如果套接字之前未通过 `uv_udp_bind` 绑定，它将被绑定到 0.0.0.0（即“所有接口”的 IPv4 地址）和一个随机端口。

在 Windows 上，如果 `addr` 被初始化为指向未指定地址（0.0.0.0 或 `::`），它会被改为指向 `localhost`。这样做是为了与 Linux 系统的行为保持一致。

对于已连接的 UDP 句柄，`addr` 必须设为 `NULL`，否则将返回 `UV_EISCONN` 错误。

对于未连接的 UDP 句柄，`addr` 不能为 `NULL`，否则将返回 `UV_EDESTADDRREQ` 错误。

参数

- `req`：UDP 请求句柄。无需初始化。
- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `bufs`：要发送的缓冲区列表。
- `nbufs`：`bufs` 中缓冲区的数量。
- `addr`：包含远程对端地址和端口的 `struct sockaddr_in` 或 `struct sockaddr_in6`。
- `send_cb`：数据发出后要调用的回调。

返回值：成功返回 0，失败返回小于 0 的错误码。

1.19.0 版本变更：新增 0.0.0.0 和 `::` 到 `localhost` 的映射。

1.27.0 版本变更：新增对已连接套接字的支持。

```
int uv_udp_try_send(uv_udp_t handle, const uv_buf_t bufs[], unsigned int nbufs,
const struct sockaddr addr)
```

与 `uv_udp_send` 相同，但如果发送请求无法立即完成，则不会将其加入队列。

对于已连接的 UDP 句柄，`addr` 必须设为 `NULL`，否则将返回 `UV_EISCONN` 错误。

对于未连接的 UDP 句柄，`addr` 不能为 `NULL`，否则将返回 `UV_EDESTADDRREQ` 错误。

返回值：`>= 0`：已发送的字节数（与给定的缓冲区大小一致）。`< 0`：负数错误码（当消息无法立即发送时返回 `UV_EAGAIN`）。

1.27.0 版本变更：新增对已连接套接字的支持。

```
int uv_udp_try_send2(uv_udp_t handle, unsigned int count, uv_buf_t bufs[/  
count/], unsigned int nbufs[/count/], struct sockaddr addrs[/count*/], unsigned int  
flags)
```

与 `uv_udp_try_send` 类似，但可以一次发送多个数据报。是对 `sendmsg(2)` 的轻量抽象，对不支持该系统调用的平台使用 `sendmsg(2)` 回退循环。句柄必须已完全初始化；请先调用 `uv_udp_bind`。

返回值：`>= 0`：已发送的数据报数。仅当 `count` 为零时才为 0。`< 0`：负数错误码。仅当发送第一个数据报失败时才返回负数，否则返回正的发送数量。当数据报当前无法发送时返回 `UV_EAGAIN`；此时应回退到 `uv_udp_send`。

自 1.50.0 版本新增

```
int uv_udp_recv_start(uv_udp_t* handle, uv_alloc_cb alloc_cb, uv_udp_recv_cb  
recv_cb)
```

准备接收数据。如果套接字之前未通过 `uv_udp_bind` 绑定，它会被绑定到 `0.0.0.0`（即“所有接口”的 IPv4 地址）和一个随机端口。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。
- `alloc_cb`：需要临时存储时调用的回调。
- `recv_cb`：收到数据时调用的回调。

返回值：成功返回 0，失败返回小于 0 的错误码。

注意：使用 `recvmsg(2)` 时，一次接收的消息数量受限于能在 `alloc_cb` 分配的缓冲区中容纳的最大尺寸数据报（dgram）的数量，且 `udp_recv` 的 `alloc_cb` 中的 `suggested_size` 总是被设置为 1 个最大尺寸数据报的大小。

1.35.0 版本变更：在支持的平台上新增对 `recvmsg(2)` 的支持。此句柄的 `uv_alloc_cb` 应创建以 64 KiB 为倍数大小的缓冲区。

1.37.0 版本变更：`recvmsg(2)` 支持不再被隐式启用，必须通过向 `uv_udp_init_ex` 传入 `UV_UDP_RECVMSG` 标志显式请求。

1.39.0 版本变更：`uv_udp_using_recvmsg` 可在 `alloc_cb` 中用于判断是否应为当前句柄/平台分配一个适用于 `recvmsg(2)` 大小的缓冲区。

int uv_udp_using_recvmsg(uv_udp_t* handle)

如果 UDP 句柄是以 `UV_UDP_RECVMSG` 标志创建且平台支持 `recvmsg(2)`，则返回 1；否则返回 0。

自 1.39.0 版本新增

int uv_udp_recv_stop(uv_udp_t* handle)

停止监听传入的数据报。

参数

- `handle`：UDP 句柄。应已通过 `uv_udp_init` 初始化。

返回值：成功返回 0，失败返回小于 0 的错误码。

size_t uv_udp_get_send_queue_size(const uv_udp_t* handle)

返回 `handle->send_queue_size`。

自 1.19.0 版本新增

size_t uv_udp_get_send_queue_count(const uv_udp_t* handle)

返回 `handle->send_queue_count`。

自 1.19.0 版本新增

参见： `uv_handle_t` 的 API 函数同样适用。

第 15 章 • API：流、管道、轮询与 TTY

uv_pipe_t — 管道句柄 • uv_stream_t — 流句柄 • uv_poll_t — 轮询句柄 • uv_tty_t — TTY 句柄

本章译自 libuv 官方文档 `api/pipe.rst`、`api/stream.rst`、`api/poll.rst`、`api/tty.rst` (libuv 1.52)

uv_pipe_t — 管道句柄

管道句柄在 Unix 上（包括本地域套接字、管道和 FIFO）以及 Windows 上的命名管道之上提供了一层抽象。

`uv_pipe_t` 是 `uv_stream_t` 的"子类"。

数据类型

uv_pipe_t

管道句柄类型。

公开成员

int uv_pipe_t.ipc

此管道是否适合用于进程间传递句柄。只有即将传递句柄的已连接管道才应设置此标志，而不是调用 `uv_accept` 的监听管道。

参见： `uv_stream_t` 的成员同样适用。

API

int uv_pipe_init(uv_loop_t loop, uv_pipe_t handle, int ipc)

初始化一个管道句柄。 `ipc` 参数是一个布尔值，指示此管道是否将用于进程间传递句柄（这可能会改变传输的字节内容）。只有即将传递句柄的已连接管道才应设置此标志，而不是调用 `uv_accept` 的监听管道。

int uv_pipe_open(uv_pipe_t* handle, uv_file file)

将现有的文件描述符或 HANDLE 作为管道打开。

1.2.1 版本变更：文件描述符会被设置为非阻塞模式。

注意：传入的文件描述符或 HANDLE 不会做类型检查，但要求它必须代表一个有效的管道。

int uv_pipe_bind(uv_pipe_t handle, const char name)

将管道绑定到一个文件路径（Unix）或名称（Windows）。

与 `uv_pipe_bind2` 不同，不支持 Linux 抽象命名空间套接字。

是 `uv_pipe_bind2(handle, name, strlen(name), 0)` 的别名。

注意：Unix 上的路径会被截断到 `sizeof(sockaddr_un.sun_path)` 字节，通常在 92 到 108 字节之间。

int uv_pipe_bind2(uv_pipe_t handle, const char name, size_t namelen, unsigned int flags)

将管道绑定到一个文件路径（Unix）或名称（Windows）。

`flags` 必须为 0 或 `UV_PIPE_NO_TRUNCATE`。遇到不支持的标志时返回 `UV_EINVAL`，不执行绑定操作。

支持 Linux 抽象命名空间套接字。 `namelen` 必须包含前导的 `nul` 字节，但不包含结尾的 `nul` 字节。

自 1.46.0 版本新增

注意：Unix 上的路径会被截断到 `sizeof(sockaddr_un.sun_path)` 字节，通常在 92 到 108 字节之间；除非指定了 `UV_PIPE_NO_TRUNCATE` 标志，此时会返回 `UV_EINVAL` 错误。

void uv_pipe_connect(uv_connect_t req, uv_pipe_t handle, const char* name, uv_connect_cb cb)

连接到 Unix 域套接字或 Windows 命名管道。

与 `uv_pipe_connect2` 不同，不支持 Linux 抽象命名空间套接字。

是 `uv_pipe_connect2(req, handle, name, strlen(name), 0, cb)` 的别名。

注意：Unix 上的路径会被截断到 `sizeof(sockaddr_un.sun_path)` 字节，通常在 92 到 108 字节之间。

int uv_pipe_connect2(uv_connect_t req, uv_pipe_t handle, const char* name, size_t namelen, unsigned int flags, uv_connect_cb cb)

连接到 Unix 域套接字或 Windows 命名管道。

`flags` 必须为 0 或 `UV_PIPE_NO_TRUNCATE`。遇到不支持的标志时返回 `UV_EINVAL`，不执行连接操作。

支持 Linux 抽象命名空间套接字。`namelen` 必须包含前导的 `nul` 字节，但不包含结尾的 `nul` 字节。

自 1.46.0 版本新增

注意：Unix 上的路径会被截断到 `sizeof(sockaddr_un.sun_path)` 字节，通常在 92 到 108 字节之间；除非指定了 `UV_PIPE_NO_TRUNCATE` 标志，此时会返回 `UV_EINVAL` 错误。

int uv_pipe_getsockname(const uv_pipe_t handle, char buffer, size_t* size)

获取 Unix 域套接字或命名管道的名称。

必须提供一个预先分配好的缓冲区。`size` 参数在输入时保存缓冲区的长度，在输出时被设置为写入缓冲区的字节数。如果缓冲区不够大，将返回 `UV_ENOBUFS`，且 `len` 将包含所需的大小。

FreeBSD 上 UNIX 域套接字的路径不能超过 `sockaddr_un.sun_path` 的大小，尽管该平台定义了更大的 `SOCK_MAXADDRLLEN`。

1.3.0 版本变更：返回的长度不再包含结尾的 null 字节，且缓冲区不以 null 结尾。

int uv_pipe_getpeername(const uv_pipe_t handle, char buffer, size_t* size)

获取该句柄所连接的 Unix 域套接字或命名管道的名称。

必须提供一个预先分配好的缓冲区。`size` 参数在输入时保存缓冲区的长度，在输出时被设置为写入缓冲区的字节数。如果缓冲区不够大，将返回 `UV_ENOBUFS`，且 `len` 将包含所需的大小。

自 1.3.0 版本新增

void uv_pipe_pending_instances(uv_pipe_t* handle, int count)

设置管道服务器等待连接时的待处理管道实例数。

注意：此设置仅适用于 Windows。

int uv_pipe_pending_count(uv_pipe_t* handle)

uv_handle_type uv_pipe_pending_type(uv_pipe_t* handle)

用于通过 IPC 管道接收句柄。

步骤：先调用 `uv_pipe_pending_count`，如果结果 > 0 ，则按 `uv_pipe_pending_type` 返回的给定 `type` 初始化一个句柄，然后调用 `uv_accept(pipe, handle)`。

参见：`uv_stream_t` 的 API 函数同样适用。

int uv_pipe_chmod(uv_pipe_t* handle, int flags)

修改管道权限，使其可以被不同用户运行的进程访问。使管道对所有用户可写或可读。

模式可以是 `UV_WRITABLE`、`UV_READABLE` 或 `UV_WRITABLE | UV_READABLE`。此函数是阻塞的。

自 1.16.0 版本新增

`int uv_pipe(uv_file fds[2], int read_flags, int write_flags)`

创建一对已连接的管道句柄。数据可以写入 `fds[1]` 并从 `fds[0]` 读取。得到的句柄可以传给 `uv_pipe_open`、用于 `uv_spawn`，或用于其他任何用途。

`flags` 的有效值为：

- `UV_NONBLOCK_PIPE`：以 `OVERLAPPED` 或 `FIONBIO / O_NONBLOCK` I/O 方式打开指定的句柄。对于将被 libuv 使用的句柄，推荐这样做；其他情况通常不建议。

等价于设置了 `O_CLOEXEC` 标志的 `pipe(2)`。

自 1.41.0 版本新增

uv_stream_t — 流句柄

流句柄提供了对双工通信信道的抽象。`uv_stream_t` 是一个抽象类型，libuv 以 `uv_tcp_t`、`uv_pipe_t` 和 `uv_tty_t` 的形式提供了 3 种流的实现。

数据类型

`uv_stream_t`

流句柄类型。

`uv_connect_t`

连接请求类型。

`uv_shutdown_t`

关闭 (shutdown) 请求类型。

`uv_write_t`

写请求类型。复用此类型的对象时必须格外小心。当流处于非阻塞模式时，通过

`uv_write` 发出的写请求会被排队。此时复用对象是未定义行为。只有在传给 `uv_write` 的回调被触发之后，才可安全地复用 `uv_write_t` 对象。

void (uv_read_cb)(uv_stream_t stream, ssize_t nread, const uv_buf_t* buf)

在流上读到数据时调用的回调。

当有数据可用时 `nread > 0`，出错时 `< 0`。当到达 EOF 时，`nread` 会被设为 `UV_EOF`。当 `nread < 0` 时，`buf` 参数可能不指向有效的缓冲区；此时 `buf.len` 和 `buf.base` 都会被置为 0。

注意： `nread` 可能为 0，这不表示错误或 EOF。这等价于 `read(2)` 下的 `EAGAIN` 或 `EWOULDBLOCK`。

发生错误时，被调用方负责通过调用 `uv_read_stop` 或 `uv_close` 来停止/关闭流。再次尝试从流中读取是未定义的。

被调用方负责释放缓冲区，`libuv` 不会复用它。出错时缓冲区可能是一个空缓冲区（即 `buf->base == NULL` 且 `buf->len == 0`）。

void (uv_write_cb)(uv_write_t req, int status)

在流上写入数据后调用的回调。成功时 `status` 为 0，否则 `< 0`。

void (uv_connect_cb)(uv_connect_t req, int status)

由 `uv_connect` 发起的连接完成后调用的回调。成功时 `status` 为 0，否则 `< 0`。

void (uv_shutdown_cb)(uv_shutdown_t req, int status)

关闭请求完成后调用的回调。成功时 `status` 为 0，否则 `< 0`。

void (uv_connection_cb)(uv_stream_t server, int status)

流服务器收到传入连接时调用的回调。用户可以通过调用 `uv_accept` 来接受该连接。成功时 `status` 为 0，否则 `< 0`。

公开成员

size_t uv_stream_t.write_queue_size

包含正在排队等待发送的字节数。只读。

uv_stream_t* uv_connect_t.handle

此连接请求在其上运行的流的指针。

uv_stream_t* uv_shutdown_t.handle

此关闭请求在其上运行的流的指针。

uv_stream_t* uv_write_t.handle

此写请求在其上运行的流的指针。

uv_stream_t* uv_write_t.send_handle

正通过此写请求发送的流的指针。

参见： `uv_handle_t` 的成员同样适用。

API**int uv_shutdown(uv_shutdown_t req, uv_stream_t handle, uv_shutdown_cb cb)**

关闭双工流的输出（写）一侧。它会等待待处理的写请求完成。`handle` 应指向一个已初始化的流。`req` 应是一个未初始化的关闭请求结构体。关闭完成后会调用 `cb`。

int uv_listen(uv_stream_t* stream, int backlog, uv_connection_cb cb)

开始监听传入连接。`backlog` 表示内核可以排队的连接数，与 `listen(2)` 相同。当收到新的传入连接时，会调用 `uv_connection_cb` 回调。

int uv_accept(uv_stream_t server, uv_stream_t client)

此调用与 `uv_listen` 配合使用，用于接受传入连接。在收到 `uv_connection_cb` 后调用此函数来接受连接。调用此函数之前，客户端句柄必须已初始化。返回值 `< 0` 表示错误。

当 `uv_connection_cb` 回调被调用时，可以保证此函数第一次会成功完成。如果尝试多次使用，则可能失败。建议每次 `uv_connection_cb` 调用只调用此函数一次。

注意： `server` 和 `client` 必须是运行在同一个事件循环上的句柄。

int uv_read_start(uv_stream_t* stream, uv_alloc_cb alloc_cb, uv_read_cb read_cb)

从传入的流读取数据。 `uv_read_cb` 回调会被多次调用，直到没有更多数据可读或调用了 `uv_read_stop`。

1.38.0 版本变更： `uv_read_start` 现在在被调用两次时一致地返回 `UV_EALREADY`，在流正在关闭时返回 `UV_EINVAL`。在更早的 libuv 版本中，它在 Windows 上返回 `UV_EALREADY` 而在 UNIX 上不返回，在 UNIX 上返回 `UV_EINVAL` 而在 Windows 上不返回。

int uv_read_stop(uv_stream_t*)

停止从流读取数据。 `uv_read_cb` 回调将不再被调用。

此函数是幂等的，可以安全地对已停止的流调用。

此函数总是成功，因此无需检查其返回值。非零的返回值仅表示在 Windows 上可能还有资源释放工作待该 TTY 的下一个输入事件完成，并不表示失败。

int uv_write(uv_write_t req, uv_stream_t handle, const uv_buf_t bufs[], unsigned int nbufs, uv_write_cb cb)

向流写入数据。缓冲区按顺序写入。示例：

```

void cb(uv_write_t* req, int status) {
    /* 处理写入结果的逻辑 */
}

uv_buf_t a[] = {
    { .base = "1", .len = 1 },
    { .base = "2", .len = 1 }
};

uv_buf_t b[] = {
    { .base = "3", .len = 1 },
    { .base = "4", .len = 1 }
};

uv_write_t req1;
uv_write_t req2;

/* 写出 "1234" */
uv_write(&req1, stream, a, 2, cb);
uv_write(&req2, stream, b, 2, cb);

```

注意：缓冲区所指向的内存必须在回调被调用之前保持有效。 `uv_write2` 同样如此。

int uv_write2(uv_write_t req, uv_stream_t handle, const uv_buf_t bufs[], unsigned int nbufs, uv_stream_t* send_handle, uv_write_cb cb)

用于通过管道发送句柄的扩展写函数。管道必须以 `ipc == 1` 初始化。

注意：在 Unix 上 `send_handle` 必须是 TCP、管道或 UDP 句柄，在 Windows 上必须是 TCP 句柄，且它必须是服务器或已建立的连接（处于监听或已连接状态）。已绑定（bound）的套接字或管道将被视为服务器。

int uv_try_write(uv_stream_t* handle, const uv_buf_t bufs[], unsigned int nbufs)

与 `uv_write` 相同，但如果写请求无法立即完成，则不会将其加入队列。

将返回以下之一：

- `> 0`：已写入的字节数（可能小于提供的缓冲区大小）。
- `< 0`：负数错误码（如果没有任何数据能被立即发送，返回 `UV_EAGAIN`）。

int uv_try_write2(uv_stream_t handle, const uv_buf_t bufs[], unsigned int nbufs, uv_stream_t send_handle)

与 `uv_try_write` 相同，同时是像 `uv_write2` 那样通过管道发送句柄的扩展写函数。

Windows 不支持尝试发送句柄，此时返回 `UV_EAGAIN`。

自 1.42.0 版本新增

size_t uv_write_nwritten(const uv_write_t* req)

返回写请求已写入的字节数。仅在写回调（`uv_write_cb`）内部调用时有效。

这主要用于写请求通过 `uv_cancel` 被取消、回调收到 `UV_ECANCELED` 状态时，判断取消之前实际写入了多少字节。注意，如果内核在取消生效之前已完成写处理，被取消的写操作仍可能成功或以其他错误失败。

自 1.53.0 版本新增

int uv_is_readable(const uv_stream_t* handle)

如果流可读返回 1，否则返回 0。

int uv_is_writable(const uv_stream_t* handle)

如果流可写返回 1，否则返回 0。

int uv_stream_set_blocking(uv_stream_t* handle, int blocking)

启用或禁用流的阻塞模式。

启用阻塞模式后，所有写操作都会同步完成。除此之外接口保持不变，例如操作的完成或失败仍会通过异步触发的回调来报告。

警告：不建议过度依赖此 API。它在将来很可能发生重大变化。

目前在 Windows 上仅对 `uv_pipe_t` 句柄有效。在 UNIX 平台上，所有 `uv_stream_t` 句柄均受支持。

此外，libuv 目前在已提交写请求之后才改变阻塞模式时不做任何顺序保证。因此建议在打开或创建流之后立即设置阻塞模式。

1.4.0 版本变更：新增 UNIX 实现。

`size_t uv_stream_get_write_queue_size(const uv_stream_t* stream)`

返回 `stream->write_queue_size`。

自 1.19.0 版本新增

参见： `uv_handle_t` 的 API 函数同样适用。

uv_poll_t — 轮询句柄

轮询（poll）句柄用于监视文件描述符的可读、可写和断开状态，其用途类似于

`poll(2)`。

轮询句柄的用途是集成那些依赖事件循环并需要就套接字状态变化通知它的外部库，比如 `c-ares` 或 `libssh2`。不建议将 `uv_poll_t` 用于任何其他用途；`uv_tcp_t`、`uv_udp_t` 等提供的实现比 `uv_poll_t` 能达到的更快、更具可扩展性，尤其是在 Windows 上。

轮询句柄偶尔会在文件描述符实际上不可读或不可写时仍然发出可读或可写信号。因此用户在尝试对该 fd 进行读或写时，应始终准备好处理 `EAGAIN` 或等价情况。

同一个套接字上不允许有多个处于活动状态的轮询句柄，这可能导致 `libuv` 忙循环（`busyloop`）或出现其他故障。

在被活动中的轮询句柄轮询期间，用户不应关闭文件描述符。这可能导致句柄报告错误，但也可能使它开始轮询另一个套接字。不过，在调用 `uv_poll_stop` 或 `uv_close` 之后可以立即安全地关闭该 fd。

注意： 在 Windows 上，只有套接字可以被轮询句柄轮询。在 Unix 上，任何能被 `poll(2)` 接受的文件描述符都可以使用。

注意： 在 AIX 上不支持监视断开事件。

数据类型

`uv_poll_t`

轮询句柄类型。

void (uv_poll_cb)(uv_poll_t handle, int status, int events)

传给 `uv_poll_start` 的回调的类型定义。

uv_poll_event

轮询事件类型

```
enum uv_poll_event {
    UV_READABLE = 1,
    UV_WRITABLE = 2,
    UV_DISCONNECT = 4,
    UV_PRIORITIZED = 8
};
```

公开成员

无。

参见： `uv_handle_t` 的成员同样适用。

API

int uv_poll_init(uv_loop_t loop, uv_poll_t handle, int fd)

使用文件描述符初始化句柄。

1.2.2 版本变更：文件描述符会被设置为非阻塞模式。

int uv_poll_init_socket(uv_loop_t loop, uv_poll_t handle, uv_os_sock_t socket)

使用套接字描述符初始化句柄。在 Unix 上这与 `uv_poll_init` 相同。在 Windows 上它接受一个 SOCKET 句柄。

1.2.2 版本变更：套接字会被设置为非阻塞模式。

int uv_poll_start(uv_poll_t* handle, int events, uv_poll_cb cb)

开始轮询文件描述符。`events` 是由 `UV_READABLE`、`UV_WRITABLE`、`UV_PRIORITIZED` 和 `UV_DISCONNECT` 组成的位掩码。一旦检测到事件，就会调用回调，此时 `status` 被设为 0，检测到的事件被设置在 `events` 字段上。

`UV_PRIORITIZED` 事件用于监视 `sysfs` 中断或 TCP 带外 (out-of-band) 消息。

`UV_DISCONNECT` 事件是可选的，即它可能不会被上报，用户也可以忽略它，但它有助于优化关闭路径，因为可以省去一次额外的读或写调用。

如果在轮询期间发生错误，`status` 将 < 0 ，并对应某个 `UV_E*` 错误码（参见错误处理一章）。用户不应在句柄活动期间关闭套接字。如果用户还是那样做了，回调可能会被调用并报告错误状态，但这不被保证。如果 `status == UV_EBADF`，则对该文件句柄的轮询会停止，不再上报任何事件。此时用户应在该句柄上调用 `uv_close`。

注意：对已经处于活动状态的句柄调用 `uv_poll_start` 是可以的。这样做会更新正在被监视的事件掩码。

注意：尽管可以设置 `UV_DISCONNECT`，但它在 AIX 上不受支持，因此在回调的 `events` 字段中不会被设置。

注意：只要设置了 `UV_READABLE` 或 `UV_WRITABLE` 中的某个事件，并且给定的 fd/套接字相应地保持可读或可写，回调就会被再次调用。特别是在以下各种场景中：

- 回调被调用是因为套接字变为可读/可写，而回调根本没有对该套接字进行读/写操作。
- 回调对套接字执行了读操作，但没有读完所有可用数据（设置了 `UV_READABLE` 时）。
- 回调对套接字执行了写操作，但套接字之后仍保持可写（设置了 `UV_WRITABLE` 时）。
- 在对与此套接字关联的轮询句柄调用 `uv_poll_start` 之前，套接字就已经变为可读/可写，且此后套接字的状态没有变化。

在上述所有场景中，套接字保持可读或可写，因此回调会被再次调用（取决于位掩码中设置的事件）。这种行为称为水平触发 (level triggering)。

1.9.0 版本变更：新增 `UV_DISCONNECT` 事件。

1.14.0 版本变更：新增 `UV_PRIORITIZED` 事件。

int uv_poll_stop(uv_poll_t* poll)

停止轮询文件描述符，回调将不再被调用。

注意：调用 `uv_poll_stop` 立即生效：任何待处理的回调也会被取消，即使套接字状态变化通知已经处于待处理状态。

参见：`uv_handle_t` 的 API 函数同样适用。

uv_tty_t — TTY 句柄

TTY 句柄表示控制台的一个流。

`uv_tty_t` 是 `uv_stream_t` 的"子类"。

数据类型

`uv_tty_t`

TTY 句柄类型。

`uv_tty_mode_t`

自 1.2.0 版本新增

TTY 模式类型：

```
typedef enum {
    /* 初始/普通终端模式 */
    UV_TTY_MODE_NORMAL,
    /*
     * 原始输入模式（在 Windows 上还会启用 ENABLE_WINDOW_INPUT）。
     * 在未来的 libuv 版本中可能变得等价于 UV_TTY_MODE_RAW_VT。
     */
    UV_TTY_MODE_RAW,
    /* 用于 IPC 的二进制安全 I/O 模式（仅限 Unix） */
    UV_TTY_MODE_IO,
    /* 原始输入模式。在 Windows 上还会设置 ENABLE_VIRTUAL_TERMINAL_INPUT。 */
    UV_TTY_MODE_RAW_VT
} uv_tty_mode_t;
```

`uv_tty_vtermstate_t`

控制台虚拟终端模式类型：

```
typedef enum {
    /*
     * 控制台支持处理虚拟终端序列
     * (Windows 10 新控制台、ConEmu)
     */
    UV_TTY_SUPPORTED,
    /* 控制台无法处理虚拟终端序列。(传统控制台) */
    UV_TTY_UNSUPPORTED
} uv_tty_vtermstate_t
```

公开成员

无。

参见： `uv_stream_t` 的成员同样适用。

API

int uv_tty_init(uv_loop_t loop, uv_tty_t handle, uv_file fd, int unused)

用给定的文件描述符初始化一个新的 TTY 流。文件描述符通常为：

- 0 = stdin
- 1 = stdout
- 2 = stderr

在 Unix 上，此函数会使用 `ttyname_r(3)` 确定 fd 所对应终端的路径并重新打开它，仅当传入的文件描述符确实指向一个 TTY 时才使用它。这使得 libuv 可以把 tty 置于非阻塞模式，而不影响共享该 tty 的其他进程。

在不支持 `ioctl TIOCGPTN` 或 `TIOCPTYGNAME` 的系统上（例如 OpenBSD 和 Solaris），此函数不是线程安全的。

注意： 如果重新打开 TTY 失败，libuv 会回退到阻塞式写入。

1.23.1 版本变更： `readable` 参数现已不再使用、被忽略。正确的值现在会由内核自动检测。

1.9.0 版本变更：TTY 的路径由 `ttyname_r(3)` 确定。在更早的版本中，libuv 会打开 `/dev/tty`。

1.5.0 版本变更：在 UNIX 上，若尝试用一个指向普通文件的文件描述符初始化 TTY 流，则返回 `UV_EINVAL`。

int uv_tty_set_mode(uv_tty_t* handle, uv_tty_mode_t mode)

1.2.0 版本变更：模式现在以 `uv_tty_mode_t` 值指定。

将 TTY 设置为指定的终端模式。

int uv_tty_reset_mode(void)

应在程序退出时调用。将 TTY 设置重置为默认值，以交给下一个进程接管。

此函数在 Unix 平台上是异步信号安全的（async signal-safe），但如果在执行处于 `uv_tty_set_mode` 内部时调用它，可能会失败并返回错误码 `UV_EBUSY`。

int uv_tty_get_winsize(uv_tty_t handle, int width, int* height)

获取当前的窗口大小。成功时返回 0。

参见： `uv_stream_t` 的 API 函数同样适用。

void uv_tty_set_vterm_state(uv_tty_vtermstate_t state)

控制虚拟终端序列由 libuv 还是控制台来处理。对启用 ConEmu 对 ANSI X3.64 和 Xterm 256 色的支持特别有用。否则，Windows 10 控制台通常会被自动检测。

此函数仅在 Windows 系统上有意义。在 Unix 上会被静默忽略。

自 1.33.0 版本新增

int uv_tty_get_vterm_state(uv_tty_vtermstate_t* state)

获取当前虚拟终端序列由 libuv 还是控制台处理的状态。

此函数在 Unix 上未实现，会返回 `UV_ENOTSUP`。

自 1.33.0 版本新增

第 16 章 · API：信号、进程与线程

uv_signal_t — 信号句柄 · uv_process_t — 进程句柄 · 线程与同步工具

本章译自 libuv 官方文档 `api/signal.rst`、`api/process.rst`、`api/threading.rst` (libuv 1.52)

uv_signal_t — 信号句柄

信号句柄以每个事件循环为单位实现 Unix 风格的信号处理。

Windows 说明

部分信号的接收是模拟出来的：

- `SIGINT` 通常在用户按下 `CTRL+C` 时送达。不过与 Unix 一样，当终端处于原始 (raw) 模式时它不会被生成。
- `SIGBREAK` 在用户按下 `CTRL + BREAK` 时送达。
- `SIGHUP` 在用户关闭控制台窗口时生成。收到 `SIGHUP` 后，程序会有大约 10 秒的时间执行清理。之后 Windows 将无条件终止它。
- `SIGWINCH` 会在 libuv 检测到控制台大小被改变时触发。当 libuv 应用运行在控制台模拟器下，或当 32 位 libuv 应用运行在 64 位系统上时，`SIGWINCH` 会被模拟。在这些情况下，`SIGWINCH` 信号可能无法总是及时送达。对于一个可写的 `uv_tty_t` 句柄，libuv 只会在光标移动时才检测尺寸变化。当使用可读的 `uv_tty_t` 句柄时，只有当句柄处于原始模式且正在被读取时，才会检测到控制台缓冲区大小的改变。
- 其他信号的监视器可以成功创建，但这些信号永远不会被收到。这些信号是：`SIGILL`、`SIGABRT`、`SIGFPE`、`SIGSEGV`、`SIGTERM` 和 `SIGKILL`。
- 以编程方式调用 `raise()` 或 `abort()` 来触发的信号不会被 libuv 检测到；它们不会触发信号监视器。

1.15.0 版本变更: 改进了 Windows 上的 SIGWINCH 支持。

1.31.0 版本变更: 64 位 Windows 上 32 位 libuv 的 SIGWINCH 支持回退到旧实现。

Unix 说明

- SIGKILL 和 SIGSTOP 无法被捕获。
- 通过 libuv 处理 SIGBUS、SIGFPE、SIGILL 或 SIGSEGV 会导致未定义行为。
- 由 `abort()` (例如通过 `assert()`) 生成的 SIGABRT 不会被 libuv 捕获。
- 在 Linux 上, SIGRT0 和 SIGRT1 (信号 32 和 33) 被 NPTL pthreads 库用于管理线程。为这些信号安装监视器会导致不可预测的行为, 强烈不建议这样做。libuv 的未来版本可能会直接拒绝它们。

数据类型

`uv_signal_t`

信号句柄类型。

`void (uv_signal_cb)(uv_signal_t handle, int signum)`

传给 `uv_signal_start` 的回调的类型定义。

公开成员

`int uv_signal_t.signum`

此句柄正在监视的信号。只读。

参见: `uv_handle_t` 的成员同样适用。

API

`int uv_signal_init(uv_loop_t loop, uv_signal_t signal)`

初始化句柄。

int uv_signal_start(uv_signal_t* signal, uv_signal_cb cb, int signum)

以给定的回调启动句柄，监视给定的信号。

int uv_signal_start_oneshot(uv_signal_t* signal, uv_signal_cb cb, int signum)

自 1.12.0 版本新增

功能与 `uv_signal_start` 相同，但信号在收到的瞬间监视器即被重置（仅触发一次）。

int uv_signal_stop(uv_signal_t* signal)

停止句柄，回调将不再被调用。

参见： `uv_handle_t` 的 API 函数同样适用。

uv_process_t — 进程句柄

进程句柄用于派生新进程，并允许用户控制它、通过流与它建立通信信道。

数据类型

uv_process_t

进程句柄类型。

uv_process_options_t

派生进程的选项（传给 `uv_spawn`）。

```
typedef struct uv_process_options_s {
    uv_exit_cb exit_cb;
    const char* file;
    char** args;
    char** env;
    const char* cwd;
    unsigned int flags;
    int stdio_count;
    uv_stdio_container_t* stdio;
    uv_uid_t uid;
    uv_gid_t gid;
} uv_process_options_t;
```

void (uv_exit_cb)(uv_process_t, int64_t exit_status, int term_signal)

`uv_process_options_t` 中传入的回调的类型定义，它会给出退出状态以及导致进程终止的信号（如果有）。

uv_process_flags

要设置在 `uv_process_options_t` 的 `flags` 字段上的标志。

```

enum uv_process_flags {
    /*
     * 设置子进程的用户 id。
     */
    UV_PROCESS_SETUID = (1 << 0),
    /*
     * 设置子进程的组 id。
     */
    UV_PROCESS_SETGID = (1 << 1),
    /*
     * 在将参数列表转换为命令行字符串时，不给任何参数加引号，
     * 也不做任何其他转义。此选项仅在 Windows 系统上有意义。
     * 在 Unix 上会被静默忽略。
     */
    UV_PROCESS_WINDOWS_VERBATIM_ARGUMENTS = (1 << 2),
    /*
     * 以分离 (detached) 状态派生子进程—这会使其成为进程组组长，
     * 并实际允许子进程在父进程退出后继续运行。注意，除非父进程对
     * 子进程句柄调用 uv_unref()，否则子进程仍会保持父进程的
     * 事件循环存活。
     */
    UV_PROCESS_DETACHED = (1 << 3),
    /*
     * 隐藏通常会被创建的子进程窗口。此选项仅在 Windows 系统上有意义。
     * 在 Unix 上会被静默忽略。
     */
    UV_PROCESS_WINDOWS_HIDE = (1 << 4),
    /*
     * 隐藏通常会被创建的子进程控制台窗口。此选项仅在 Windows 系统上
     * 有意义。在 Unix 上会被静默忽略。
     */
    UV_PROCESS_WINDOWS_HIDE_CONSOLE = (1 << 5),
    /*
     * 隐藏通常会被创建的子进程 GUI 窗口。此选项仅在 Windows 系统上
     * 有意义。在 Unix 上会被静默忽略。
     */
    UV_PROCESS_WINDOWS_HIDE_GUI = (1 << 6),
    /*
     * 在 Windows 上，如果 uv_process_options_t 的 file 字段指定的
     * 待执行程序路径带有目录成分，则在尝试带 '.exe' 或 '.cmd' 等
     * 扩展名的变体之前，先搜索确切的文件名。
     */
    UV_PROCESS_WINDOWS_FILE_PATH_EXACT_NAME = (1 << 7)
};

```

uv_stdio_container_t

传给子进程的每个 stdio 句柄或 fd 的容器。

```
typedef struct uv_stdio_container_s {
    uv_stdio_flags flags;
    union {
        uv_stream_t* stream;
        int fd;
    } data;
} uv_stdio_container_t;
```

uv_stdio_flags

指定如何将 stdio 传递给子进程的标志。

```

typedef enum {
    /*
     * 以下四个选项互斥，用于定义对子进程中相应
     * 文件描述符执行的操作：
     */

    /*
     * 不提供文件描述符（如果是 fd 0、1 或 2，则重定向到 /dev/null）。
     */
    UV_IGNORE = 0x00,

    /*
     * 按照下面的标志打开一个新管道到 data.stream。`data.stream`
     * 字段必须指向一个已用 uv_pipe_init(loop, data.stream, ipc);`
     * 初始化、但尚未打开或连接的 uv_pipe_t 对象。
     */
    UV_CREATE_PIPE = 0x01,

    /*
     * 子进程将获得父进程 data.fd 所指文件描述符的一个副本。
     */
    UV_INHERIT_FD = 0x02,

    /*
     * 子进程将获得父进程的 data.stream 句柄正在使用的文件描述符
     * 的一个副本。
     */
    UV_INHERIT_STREAM = 0x04,

    /*
     * 当指定 UV_CREATE_PIPE 时，UV_READABLE_PIPE 和 UV_WRITABLE_PIPE
     * 从子进程的视角决定数据流的方向。两个标志可以同时指定，
     * 以创建双工数据流。
     */
    UV_READABLE_PIPE = 0x10,
    UV_WRITABLE_PIPE = 0x20,

    /*
     * 当指定 UV_CREATE_PIPE 时，再指定 UV_NONBLOCK_PIPE 会在子进程中
     * 以非阻塞模式打开该句柄。如果子进程的设计没有考虑遇到这种模式，
     * 可能会导致数据丢失，但也可以显著提高效率。
     */
    UV_NONBLOCK_PIPE = 0x40
} uv_stdio_flags;

```

公开成员

int uv_process_t.pid

被派生进程的 PID。它在调用 `uv_spawn` 之后被设置，并且在进程退出后仍保留该值。该值仅在进程存活期间唯一；进程退出后，另一个进程可能被重新分配相同的 PID。

注意： `uv_handle_t` 的成员同样适用。

uv_exit_cb uv_process_options_t.exit_cb

进程退出后调用的回调。

const char* uv_process_options_t.file

指向要执行的程序的路径。

char uv_process_options_t.args**

命令行参数。args[0] 应为程序的路径。在 Windows 上这会使用 `CreateProcess`，它把参数拼接成一个字符串，这可能引发一些奇怪的错误。参见 `uv_process_flags` 中的 `UV_PROCESS_WINDOWS_VERBATIM_ARGUMENTS` 标志。

char uv_process_options_t.env**

新进程的环境变量。如果为 NULL，则使用父进程的环境变量。

const char* uv_process_options_t.cwd

子进程的当前工作目录。

unsigned int uv_process_options_t.flags

控制 `uv_spawn` 行为的各种标志。参见 `uv_process_flags`。

int uv_process_options_t.stdio_count

uv_stdio_container_t* uv_process_options_t.stdio

`stdio` 字段指向一个 `uv_stdio_container_t` 结构体数组，描述将提供给子进程使用的文件描述符。约定是 `stdio[0]` 指向 `stdin`，fd 1 用于 `stdout`，fd 2 为 `stderr`。

注意：在 Windows 上，只有当子进程使用 MSVCRT 运行时，大于 2 的文件描述符才对子进程可用。

uv_uid_t uv_process_options_t.uid

uv_gid_t uv_process_options_t.gid

libuv 可以改变子进程的用户/组 id。只有当 flags 字段中设置了相应的位时才会发生。

注意：Windows 不支持此功能，`uv_spawn` 会失败并把错误设置为 `UV_ENOTSUP`。

uv_stdio_flags uv_stdio_container_t.flags

指定 stdio 容器如何传递给子进程的标志。

union @0 uv_stdio_container_t.data

包含要传给子进程的 `stream` 或 `fd` 的联合体。

API

void uv_disable_stdio_inheritance(void)

禁用本进程从其父进程继承的文件描述符/句柄的继承性。其效果是由本进程派生的子进程不会意外继承这些句柄。

建议尽早在程序中调用此函数，最好在继承来的文件描述符可能被关闭或复制之前。

注意：此函数尽力而为：不保证 `libuv` 能发现所有被继承的文件描述符。一般而言它在 Windows 上做得比在 Unix 上好。

int uv_spawn(uv_loop_t loop, uv_process_t handle, const uv_process_options_t* options)

初始化进程句柄并启动进程。如果进程成功派生，此函数返回 0。否则返回对应失败原因的负数错误码。注意无论成功还是失败——你最终都必须调用 `uv_close` 关闭句柄，然后才能释放句柄的内存，这与 `libuv` 中其他的 `init` 函数不同。

派生失败的可能原因包括（但不限于）：要执行的文件不存在、没有权限使用指定的 `setuid` 或 `setgid`，或者没有足够的内存分配给新进程。

警告：在 Unix 上，如果调用 `uv_close` 时进程尚未退出，你会制造一个 libuv 无法回收的僵尸进程。之后你需要负责调用 `waitpid`。这在 Windows 上不适用。

1.24.0 版本变更：新增 `UV_PROCESS_WINDOWS_HIDE_CONSOLE` 和 `UV_PROCESS_WINDOWS_HIDE_GUI` 标志。

1.48.0 版本变更：新增 `UV_PROCESS_WINDOWS_FILE_PATH_EXACT_NAME` 标志。

int uv_process_kill(uv_process_t* handle, int signum)

向给定的进程句柄发送指定的信号。关于信号支持情况（特别是在 Windows 上）请查阅信号一章的文档。

如果指定的进程已经死亡，这不会杀掉恰好复用了同一 `pid` 的另一个进程。相比之下，如果你使用缓存的 `uv_process_get_pid` 值，`uv_kill` 可能会杀掉任意的其他进程。

int uv_kill(int pid, int signum)

向给定的 PID 发送指定的信号。关于信号支持情况（特别是在 Windows 上）请查阅信号一章的文档。

uv_pid_t uv_process_get_pid(const uv_process_t* handle)

返回 `handle->pid`。

自 1.19.0 版本新增

参见：`uv_handle_t` 的 API 函数同样适用。

线程与同步工具

libuv 为多种线程和同步原语提供了跨平台实现。该 API 在很大程度上遵循 pthreads API。

数据类型

uv_thread_t

线程数据类型。

void (uv_thread_cb)(void arg)

用于启动线程执行的回调。arg 与传给 uv_thread_create 的值相同。

uv_key_t

线程局部存储键（thread-local key）数据类型。

uv_once_t

一次性初始化器数据类型。

uv_mutex_t

互斥锁数据类型。

uv_rwlock_t

读写锁数据类型。

uv_sem_t

信号量数据类型。

uv_cond_t

条件变量数据类型。

uv_barrier_t

屏障（barrier）数据类型。

API

线程

`uv_thread_options_t`

派生新线程的选项（传给 `uv_thread_create_ex`）。

```
typedef struct uv_thread_options_s {
    enum {
        UV_THREAD_NO_FLAGS = 0x00,
        UV_THREAD_HAS_STACK_SIZE = 0x01
    } flags;
    size_t stack_size;
} uv_thread_options_t;
```

此结构体随时可能添加更多字段，因此不应依赖其确切的布局 and 大小。

自 1.26.0 版本新增

`int uv_thread_create(uv_thread_t tid, uv_thread_cb entry, void arg)`

1.4.1 版本变更：失败时返回 `UV_E` 错误码。*

`int uv_thread_detach(uv_thread_t* tid)`

分离一个线程。被分离的线程在终止时会自动释放自己的资源，应用无需再调用

`uv_thread_join`。

自 1.50.0 版本新增

`int uv_thread_create_ex(uv_thread_t tid, const uv_thread_options_t params, uv_thread_cb entry, void* arg)`

与 `uv_thread_create` 类似，但额外指定创建新线程的选项。

如果设置了 `UV_THREAD_HAS_STACK_SIZE`，`stack_size` 为新线程指定栈大小。`0` 表示应使用默认值，即行为如同未设置该标志。其他值会被向上取整到最近的页边界。

自 1.26.0 版本新增

```
int uv_thread_setaffinity(uv_thread_t tid, char cpumask, char* oldmask, size_t mask_size)
```

将指定线程的亲亲和性设置为 cpumask，它以字节为单位指定。可以选择在 oldmask 中返回之前的亲和性设置。在 Unix 上，使用 `pthread_getaffinity_np(3)` 获取亲和性设置，并把 `cpu_set_t` 映射到 oldmask 中的字节。然后把 cpumask 中的字节映射到 `cpu_set_t` 并使用 `pthread_setaffinity_np(3)`。在 Windows 上，把 cpumask 中的字节映射到位掩码并使用 `SetThreadAffinityMask()`，它会返回之前的亲和性设置。

`mask_size` 指定 cpumask/oldmask 中条目（字节）的数量，必须大于或等于 `uv_cpumask_size`。

注意：在 Windows 上线程亲和性设置不是原子操作。macOS 上不支持。

自 1.45.0 版本新增

```
int uv_thread_getaffinity(uv_thread_t tid, char cpumask, size_t mask_size)
```

获取指定线程的亲亲和性设置。在 Unix 上，这会把 `pthread_getaffinity_np(3)` 返回的 `cpu_set_t` 映射到 cpumask 中的字节。

`mask_size` 指定 cpumask 中条目（字节）的数量，必须大于或等于 `uv_cpumask_size`。

注意：在 Windows 上线程亲和性获取不是原子操作。macOS 上不支持。

自 1.45.0 版本新增

```
int uv_thread_getcpu(void)
```

获取调用线程当前运行所在的 CPU 编号。

注意：目前仅在 Windows、Linux 和 FreeBSD 上实现。

自 1.45.0 版本新增

```
uv_thread_t uv_thread_self(void)
```

```
int uv_thread_join(uv_thread_t *tid)
```

```
int uv_thread_equal(const uv_thread_t t1, const uv_thread_t t2)
```

```
int uv_thread_setname(const char* name)
```

设置当前线程的名称。不同平台对线程名称的最大字符数定义了不同的限制：Linux、IBM i (16)、macOS (64)、Windows (32767) 和 NetBSD (32) 等。如果 `name` 超出平台的限制，`uv_thread_setname()` 会将其截断。

Windows Server 2016 上不支持，返回 `UV_ENOSYS`。

自 1.50.0 版本新增

```
int uv_thread_getname(uv_thread_t tid, char name, size_t* size)
```

获取 `tid` 指定线程的名称。线程名称（连同结尾的 NUL）会被复制到 `name` 指向的缓冲区。`size` 参数指定 `name` 所指缓冲区的大小。缓冲区应足够容纳线程名称加上结尾的 NUL，否则将被截断以适应（并带结尾 NUL）。

在线程已终止之后调用此函数是未定义行为。

Windows Server 2016 上不支持，返回 `UV_ENOSYS`。

自 1.50.0 版本新增

```
int uv_thread_setpriority(uv_thread_t tid, int priority)
```

函数成功时返回值为 0。函数失败时返回值小于 0。设置 `tid` 指定线程的调度优先级。在某些平台上设置特定优先级需要提升的权限。优先级可以设置为以下常量：

`UV_THREAD_PRIORITY_HIGHEST`、`UV_THREAD_PRIORITY_ABOVE_NORMAL`、`UV_THREAD_PRIORITY_NORMAL`、`UV_THREAD_PRIORITY_BELOW_NORMAL`、`UV_THREAD_PRIORITY_LOWEST`。

在线程已终止之后调用此函数是未定义行为。

自 1.48.0 版本新增

```
int uv_thread_getpriority(uv_thread_t tid, int* priority)
```

函数成功时返回值为 0。函数失败时返回值小于 0。获取 tid 指定线程的调度优先级。输出参数 priority 中的值与平台相关。在 Linux 上，当调度策略为 SCHED_OTHER（默认）时，priority 为 0。

在线程已终止之后调用此函数是未定义行为。

自 1.48.0 版本新增

线程局部存储

注意：线程局部存储的总大小可能是有限的。也就是说，可能无法创建很多 TLS 键。

```
int uv_key_create(uv_key_t* key)
```

```
void uv_key_delete(uv_key_t* key)
```

```
void uv_key_get(uv_key_t key)
```

```
void uv_key_set(uv_key_t key, void value)
```

一次性初始化

只运行一次某函数。以同一个 guard 并发调用 uv_once 时，除一个调用者外（未指定是哪一个），所有调用者都会被阻塞。guard 应使用 UV_ONCE_INIT 宏进行静态初始化。

```
void uv_once(uv_once_t guard, void (callback)(void))
```

互斥锁

这些函数成功时返回 0，失败时返回小于 0 的错误码（当然，返回类型为 void 的除外）。

```
int uv_mutex_init(uv_mutex_t* handle)
```

```
int uv_mutex_init_recursive(uv_mutex_t* handle)
```

```
void uv_mutex_destroy(uv_mutex_t* handle)
```

```
void uv_mutex_lock(uv_mutex_t* handle)
```

```
int uv_mutex_trylock(uv_mutex_t* handle)
```

```
void uv_mutex_unlock(uv_mutex_t* handle)
```

读写锁

这些函数成功时返回 0，失败时返回小于 0 的错误码（当然，返回类型为 void 的除外）。

```
int uv_rwlock_init(uv_rwlock_t* rwlock)
```

```
void uv_rwlock_destroy(uv_rwlock_t* rwlock)
```

```
void uv_rwlock_rdlock(uv_rwlock_t* rwlock)
```

```
int uv_rwlock_tryrdlock(uv_rwlock_t* rwlock)
```

```
void uv_rwlock_rdunlock(uv_rwlock_t* rwlock)
```

```
void uv_rwlock_wrlock(uv_rwlock_t* rwlock)
```

```
int uv_rwlock_trywrlock(uv_rwlock_t* rwlock)
```

```
void uv_rwlock_wrunlock(uv_rwlock_t* rwlock)
```

信号量

这些函数成功时返回 0，失败时返回小于 0 的错误码（当然，返回类型为 void 的除外）。

```
int uv_sem_init(uv_sem_t* sem, unsigned int value)
```

```
void uv_sem_destroy(uv_sem_t* sem)
```

```
void uv_sem_post(uv_sem_t* sem)
```

```
void uv_sem_wait(uv_sem_t* sem)
```

```
int uv_sem_trywait(uv_sem_t* sem)
```

条件变量

这些函数成功时返回 0，失败时返回小于 0 的错误码（当然，返回类型为 void 的除外）。

注意：

1. 调用者应准备好应对 `uv_cond_wait` 和 `uv_cond_timedwait` 上的虚假唤醒（spurious wakeups）。
2. `uv_cond_timedwait` 的超时参数是相对于函数被调用的时刻的。
3. 在 z/OS 上，`uv_cond_timedwait` 的超时参数会被转换为等待过期的绝对系统时间。如果当前系统时钟时间在条件被通知之前就超过了计算出的绝对时间，则产生 `ETIMEDOUT` 错误。等待开始后，等待时间不受系统时钟变更的影响。

```
int uv_cond_init(uv_cond_t* cond)
```

```
void uv_cond_destroy(uv_cond_t* cond)
```

```
void uv_cond_signal(uv_cond_t* cond)
```

```
void uv_cond_broadcast(uv_cond_t* cond)
```

```
void uv_cond_wait(uv_cond_t cond, uv_mutex_t mutex)
```

```
int uv_cond_timedwait(uv_cond_t cond, uv_mutex_t mutex, uint64_t timeout)
```

屏障

这些函数成功时返回 0，失败时返回小于 0 的错误码（当然，返回类型为 void 的除外）。

注意： `uv_barrier_wait` 会向一个任意选定的"序列化者"（serializer）线程返回大于 0 的值，以便于进行清理，即：

```
if (uv_barrier_wait(&barrier) > 0)
    uv_barrier_destroy(&barrier);
```

```
int uv_barrier_init(uv_barrier_t* barrier, unsigned int count)
```

```
void uv_barrier_destroy(uv_barrier_t* barrier)
```

```
int uv_barrier_wait(uv_barrier_t* barrier)
```

第 17 章 · API：杂项、错误码与设计概览

杂项工具 · 错误处理 · 设计概述 · 共享库处理

本章译自 libuv 官方文档 `api/misc.rst`、`api/errors.rst`、`api/design.rst`、`api/dll.rst` (libuv 1.52)

杂项工具

本节包含不适合归入其他任何小节的杂项函数。

数据类型

`uv_buf_t`

缓冲区数据类型。

`char* uv_buf_t.base`

指向缓冲区基地址的指针。

`size_t uv_buf_t.len`

缓冲区中的总字节数。

注意：在 Windows 上此字段为 `ULONG`。

`void (uv_malloc_func)(size_t size)`

`malloc(3)` 的替换函数。参见 `uv_replace_allocator`。

`void (uv_realloc_func)(void* ptr, size_t size)`

`realloc(3)` 的替换函数。参见 `uv_replace_allocator`。

void (uv_malloc_func)(size_t count, size_t size)

`calloc(3)` 的替换函数。参见 `uv_replace_allocator`。

void (uv_free_func)(void ptr)

`free(3)` 的替换函数。参见 `uv_replace_allocator`。

void (uv_random_cb)(uv_random_t req, int status, void* buf, size_t buflen)

传给 `uv_random` 的回调。出错时 `status` 非零。`buf` 指针与传给 `uv_random` 的指针相同。

uv_file

文件句柄的跨平台表示。

uv_os_sock_t

套接字句柄的跨平台表示。

uv_os_fd_t

文件描述符的抽象表示。在 Unix 系统上它是 `int` 的 `typedef`，在 Windows 上是 `HANDLE`。

uv_pid_t

`pid_t` 的跨平台表示。

自 1.16.0 版本新增

uv_timeval_t

以微秒分辨率存储时间的非 Y2K38 安全数据类型。将在 libuv v2.0 中被 `uv_timeval64_t` 取代。

```
typedef struct {
    long tv_sec;
    long tv_usec;
} uv_timeval_t;
```

uv_timeval64_t

以微秒分辨率存储时间的 Y2K38 安全数据类型。

```
typedef struct {  
    int64_t tv_sec;  
    int32_t tv_usec;  
} uv_timeval64_t;
```

uv_timespec64_t

以纳秒分辨率存储时间的 Y2K38 安全数据类型。

```
typedef struct {  
    int64_t tv_sec;  
    int32_t tv_nsec;  
} uv_timespec64_t;
```

uv_clock_id

`uv_clock_gettime` 使用的时钟源。

```
typedef enum {  
    UV_CLOCK_MONOTONIC,  
    UV_CLOCK_REALTIME  
} uv_clock_id;
```

uv_rusage_t

资源占用结果的数据类型。

```
typedef struct {
    uv_timeval_t ru_utime; /* 使用的用户 CPU 时间 */
    uv_timeval_t ru_stime; /* 使用的系统 CPU 时间 */
    uint64_t ru_maxrss; /* 最大常驻内存集大小 */
    uint64_t ru_ixrss; /* 共享内存总量 (X) */
    uint64_t ru_idrss; /* 非共享数据总量 (X) */
    uint64_t ru_isrss; /* 非共享栈总量 (X) */
    uint64_t ru_minflt; /* 页回收 (软缺页) (X) */
    uint64_t ru_majflt; /* 缺页 (硬缺页) */
    uint64_t ru_nswap; /* 换出次数 (X) */
    uint64_t ru_inblock; /* 块输入操作 */
    uint64_t ru_oublock; /* 块输出操作 */
    uint64_t ru_msgsnd; /* 发送的 IPC 消息 (X) */
    uint64_t ru_msgrcv; /* 接收的 IPC 消息 (X) */
    uint64_t ru_nsignals; /* 收到的信号 (X) */
    uint64_t ru_nvcsw; /* 自愿上下文切换 (X) */
    uint64_t ru_nivcsw; /* 非自愿上下文切换 (X) */
} uv_rusage_t;
```

标记 (X) 的成员在 Windows 上不受支持。类 UNIX 平台上支持的字段参见 `getrusage(2)`。

最大常驻内存集大小以千字节 (KB) 为单位报告，这是大多数平台原生使用的单位。

uv_cpu_info_t

CPU 信息的数据类型。

```
typedef struct uv_cpu_info_s {
    char* model;
    int speed;
    struct uv_cpu_times_s {
        uint64_t user; /* 毫秒 */
        uint64_t nice; /* 毫秒 */
        uint64_t sys; /* 毫秒 */
        uint64_t idle; /* 毫秒 */
        uint64_t irq; /* 毫秒 */
    } cpu_times;
} uv_cpu_info_t;
```

uv_interface_address_t

接口地址的数据类型。

```
typedef struct uv_interface_address_s {
    char* name;
    char phys_addr[6];
    int is_internal;
    union {
        struct sockaddr_in address4;
        struct sockaddr_in6 address6;
    } address;
    union {
        struct sockaddr_in netmask4;
        struct sockaddr_in6 netmask6;
    } netmask;
} uv_interface_address_t;
```

uv_passwd_t

密码文件（password file）信息的数据类型。

```
typedef struct uv_passwd_s {
    char* username;
    long uid;
    long gid;
    char* shell;
    char* homedir;
} uv_passwd_t;
```

uv_group_t

组文件（group file）信息的数据类型。

```
typedef struct uv_group_s {
    char* groupname;
    unsigned long gid;
    char** members;
} uv_group_t;
```

uv_utsname_t

操作系统名称和版本信息的数据类型。

```
typedef struct uv_utsname_s {
    char sysname[256];
    char release[256];
    char version[256];
    char machine[256];
} uv_utsname_t;
```

uv_env_item_t

环境变量存储的数据类型。

```
typedef struct uv_env_item_s {
    char* name;
    char* value;
} uv_env_item_t;
```

uv_random_t

随机数据请求类型。

API

uv_handle_type uv_guess_handle(uv_file file)

用于检测给定的文件描述符应使用哪种类型的流。通常会在初始化阶段用它来猜测 `stdio` 流的类型。

要获得 `isatty(3)` 等价的功能，可以使用此函数并检测是否为 `UV_TTY`。

int uv_replace_allocator(uv_malloc_func malloc_func, uv_realloc_func realloc_func, uv_calloc_func calloc_func, uv_free_func free_func)

自 1.6.0 版本新增

覆盖标准库的 `malloc(3)`、`calloc(3)`、`realloc(3)`、`free(3)` 内存分配函数。

此函数必须在任何其他 `libuv` 函数被调用之前调用，或者在所有资源已被释放、`libuv` 不再引用任何已分配内存块之后调用。

成功时返回 0；如果任何一个函数指针为 `NULL`，返回 `UV_EINVAL`。

警告： 没有针对多次更换分配器的保护。如果用户更换了分配器，他们有责任确保更换时没有用旧分配器分配过内存，或新旧分配器是兼容的。

警告： 分配器必须是线程安全的。

`void uv_library_shutdown(void);`

自 1.38.0 版本新增

释放 libuv 持有的所有全局状态。libuv 通常会在自身被卸载时自动完成这一操作，但也可以指示它手动执行清理。

警告： `uv_library_shutdown` 只能调用一次。

警告： 当仍有事件循环或 I/O 请求处于活动状态时，不要调用 `uv_library_shutdown`。

警告： 调用 `uv_library_shutdown` 之后不要再调用任何 libuv 函数。

`uv_buf_t uv_buf_init(char* base, unsigned int len)`

`uv_buf_t` 的构造函数。

由于平台差异，用户不能依赖 `uv_buf_t` 结构体中 `base` 和 `len` 成员的顺序。`uv_buf_t` 用完后由用户负责释放 `base`。结构体按值返回。

警告： 不建议把 `len` 设置为很大的值，那可能导致虚假的失败。具体来说，Windows 上超过约 511 MB 的写入可能失败，各种类 Unix 系统上超过约 2 GB (0x7fff000 字节) 的 I/O 可能失败。通常更好的做法是把数据拆分成多次 `uv_write` 调用（把 `write_cb` 附加到最后一次调用上）。

`char uv_setup_args(int argc, char argv)`

保存程序参数。获取/设置进程标题或可执行文件路径时需要它。libuv 可能会接管 `argv` 所指向的内存。此函数应在程序启动时恰好调用一次。

示例：

```
argv = uv_setup_args(argc, argv); /* 可能返回 argv 的副本。 */
```

int uv_get_process_title(char* buffer, size_t size)

获取当前进程的标题。在 Unix 和 AIX 系统上调用此函数之前你必须先调用 `uv_setup_args`。如果在需要它的系统上尚未调用过 `uv_setup_args`，则返回 `UV_ENOBUFS`。如果 `buffer` 为 `NULL` 或 `size` 为零，返回 `UV_EINVAL`。如果 `size` 无法容纳进程标题和结尾的 `\nul` 字符，函数返回 `UV_ENOBUFS`。

注意：在 BSD 系统上，获取初始进程标题需要 `uv_setup_args`。在调用 `uv_setup_args` 或 `uv_set_process_title` 之前，返回的进程标题将是空字符串。

1.18.1 版本变更：现在在所有支持的平台上都是线程安全的。

1.39.0 版本变更：现在如果需要 `uv_setup_args` 但尚未调用，会返回错误。

int uv_set_process_title(const char* title)

设置当前进程的标题。在 Unix 和 AIX 系统上调用此函数之前你必须先调用 `uv_setup_args`。如果在需要它的系统上尚未调用过 `uv_setup_args`，则返回 `UV_ENOBUFS`。在为进程标题提供固定大小缓冲区的平台上，`title` 的内容会被复制到缓冲区中，如果超出可用空间则被截断。其他平台如果无法分配足够的空间来复制 `title` 的内容，将返回 `UV_ENOMEM`。

1.18.1 版本变更：现在在所有支持的平台上都是线程安全的。

1.39.0 版本变更：现在如果需要 `uv_setup_args` 但尚未调用，会返回错误。

int uv_resident_set_memory(size_t* rss)

获取当前进程的常驻内存集大小（RSS）。

int uv_uptime(double* uptime)

获取当前系统的运行时间（uptime）。根据系统的不同，返回完整的或带小数的秒数。

int uv_getrusage(uv_rusage_t* rusage)

获取当前进程的资源占用度量。

注意：在 Windows 上并非所有字段都会被设置，不支持的字段以零填充。更多细节参见 `uv_rusage_t`。

`int uv_getrusage_thread(uv_rusage_t* rusage)`

获取调用线程的资源占用度量。

自 1.50.0 版本新增

注意：并非所有平台都支持。可能返回 `UV_ENOTSUP`。在 macOS 和 Windows 上并非所有字段都会被设置，不支持的字段以零填充。更多细节参见 `uv_rusage_t`。

`uv_pid_t uv_os_getpid(void)`

返回当前进程的进程 ID。

自 1.18.0 版本新增

`uv_pid_t uv_os_getppid(void)`

返回父进程的进程 ID。

自 1.16.0 版本新增

`unsigned int uv_available_parallelism(void)`

返回程序应使用的默认并行度估计值。总是返回非零值。

在 Linux 上，会检查调用线程的 CPU 亲和性掩码，以判断它是否被固定到了特定的 CPU。

在 Windows 上，当系统拥有超过 64 个逻辑 CPU 时，可用并行度可能被低估。

在其他平台上，报告操作系统认为处于在线状态的 CPU 数量。

自 1.44.0 版本新增

`int uv_cpu_info(uv_cpu_info_t* cpu_infos, int count)`

获取系统上 CPU 的信息。`cpu_infos` 数组将包含 `count` 个元素，需要用 `uv_free_cpu_info` 释放。

如果你需要知道有多少 CPU 可供线程或子进程使用，请使用 `uv_available_parallelism`。

void uv_free_cpu_info(uv_cpu_info_t* cpu_infos, int count)

释放之前由 `uv_cpu_info` 分配的 `cpu_infos` 数组。

int uv_cpumask_size(void)

返回用于进程/线程亲和性的掩码的最大大小；如果当前平台不支持亲和性，则返回

`UV_ENOTSUP`。

自 1.45.0 版本新增

int uv_interface_addresses(uv_interface_address_t* addresses, int count)

获取系统上网络接口的地址信息。会分配一个包含 `count` 个元素的数组并在 `addresses` 中返回。用户必须调用 `uv_free_interface_addresses` 释放它。

void uv_free_interface_addresses(uv_interface_address_t* addresses, int count)

释放由 `uv_interface_addresses` 返回的 `uv_interface_address_t` 数组。

void uv_loadavg(double avg[3])

获取平均负载。参见：[Load \(computing\) - Wikipedia](#)

注意：在 Windows 上返回 [0,0,0]（即未实现）。

int uv_ip4_addr(const char ip, int port, struct sockaddr_in addr)

将包含 IPv4 地址的字符串转换为二进制结构。

int uv_ip6_addr(const char ip, int port, struct sockaddr_in6 addr)

将包含 IPv6 地址的字符串转换为二进制结构。

int uv_ip4_name(const struct sockaddr_in src, char dst, size_t size)

将包含 IPv4 地址的二进制结构转换为字符串。

int uv_ip6_name(const struct sockaddr_in6 src, char dst, size_t size)

将包含 IPv6 地址的二进制结构转换为字符串。

int uv_ip_name(const struct sockaddr src, char dst, size_t size)

将包含 IPv4 地址或 IPv6 地址的二进制结构转换为字符串。

int uv_inet_ntop(int af, const void src, char dst, size_t size)

int uv_inet_pton(int af, const char src, void dst)

支持 IPv6 的 `inet_ntop(3)` 和 `inet_pton(3)` 跨平台实现。成功时返回 0。出错时目标 `dst` 指针保持不变。

UV_IF_NAMESIZE

IPv6 接口标识符名称的最大长度。在 Unix 上定义为 `IFNAMSIZ`，在 Linux 和 Windows 上定义为 `IF_NAMESIZE`。

自 1.16.0 版本新增

int uv_if_indextoname(unsigned int ifindex, char buffer, size_t size)

支持 IPv6 的 `if_indextoname(3)` 实现。调用时 `*size` 表示 `buffer` 的长度，用于存放结果。成功时返回零，`buffer` 包含接口名，`*size` 表示 `buffer` 的字符串长度（不含 NUL 结尾字节，NUL 结尾字节不计入 `*size`）。出错时返回负数。如果 `buffer` 不足以容纳结果，返回 `UV_ENOBUFS`，且 `*size` 表示所需的字节大小（包含 NUL 结尾字节，计入 `*size`）。

在 Unix 上，返回的接口名可以直接用作带作用域的 IPv6 地址中的接口标识符，例如 `fe80::abc:def1:2345%en0`。

在 Windows 上，返回的接口名不能用作接口标识符，因为 Windows 使用数字形式的接口标识符，例如 `fe80::abc:def1:2345%5`。

要以跨平台兼容的方式获取接口标识符，请使用 `uv_if_indextooid()`。

示例：

```
char ifname[UV_IF_NAMESIZE];
size_t size = sizeof(ifname);
uv_if_indextoname(sin6->sin6_scope_id, ifname, &size);
```

自 1.16.0 版本新增

int uv_if_indextoid(unsigned int ifindex, char buffer, size_t size)

获取适用于 IPv6 带作用域地址的网络接口标识符。在 Windows 上，把数字形式的 `ifindex` 作为字符串返回。在所有其他平台上，会调用 `uv_if_indextoname()`。结果写入 `buffer`，`*size` 表示 `buffer` 的长度。如果 `buffer` 不足以容纳结果，返回 `UV_ENOBUFS`，且 `*size` 表示容纳结果所需的大小（包含 NUL 字节）。

更多细节参见 `uv_if_indextoname`。

自 1.16.0 版本新增

int uv_exepath(char buffer, size_t size)

获取可执行文件路径。调用此函数之前你必须先调用 `uv_setup_args`。

在 `setuid` 可执行文件中要小心。在某些平台上，可执行文件路径是一个由用户控制的任意字符串。在另一些平台上，会查询可能处于用户控制之下的环境变量。

int uv_cwd(char buffer, size_t size)

获取当前工作目录，并将其存入 `buffer`。如果当前工作目录太长而放不进 `buffer`，此函数返回 `UV_ENOBUFS`，并把 `size` 设置为所需的长度（包含 null 结尾符）。

1.1.0 版本变更：在 Unix 上路径不再以斜杠结尾。

1.9.0 版本变更：返回 `UV_ENOBUFS` 时返回的长度包含结尾的 null 字节；成功时缓冲区以 null 结尾。

int uv_chdir(const char* dir)

改变当前工作目录。

int uv_os_homedir(char buffer, size_t size)

获取当前用户的主目录。在 Windows 上，`uv_os_homedir()` 首先使用 `GetEnvironmentVariableW()` 检查 `USERPROFILE` 环境变量。如果 `USERPROFILE` 未设置，则调用 `GetUserProfileDirectoryW()`。在所有其他操作系统上，`uv_os_homedir()` 首先使用 `getenv(3)` 检查 `HOME` 环境变量。如果 `HOME` 未设置，则调用 `getpwuid_r(3)`。用户的主目录存入 `buffer`。调用 `uv_os_homedir()` 时，`size` 表示 `buffer` 的最大大小。成功时 `size` 被设为 `buffer` 的字符串长度。失败返回 `UV_ENOBUFS` 时，`size` 被设为 `buffer` 所需的长度（包含 null 字节）。

警告： `uv_os_homedir()` 不是线程安全的。

自 1.6.0 版本新增

int uv_os_tmpdir(char buffer, size_t size)

获取临时目录。在 Windows 上，`uv_os_tmpdir()` 使用 `GetTempPathW()`。在所有其他操作系统上，`uv_os_tmpdir()` 使用按 `TMPDIR`、`TMP`、`TEMP`、`TEMPDIR` 顺序找到的第一个环境变量。如果都找不到，则使用路径 `"/tmp"`，或在 Android 上使用 `"/data/local/tmp"`。临时目录存入 `buffer`。调用 `uv_os_tmpdir()` 时，`size` 表示 `buffer` 的最大大小。成功时 `size` 被设为 `buffer` 的字符串长度（不含结尾的 null）。失败返回 `UV_ENOBUFS` 时，`size` 被设为 `buffer` 所需的长度（包含 null 字节）。

警告： `uv_os_tmpdir()` 不是线程安全的。

自 1.9.0 版本新增

int uv_os_get_passwd(uv_passwd_t* pwd)

获取当前有效 uid（而非真实 uid）的密码文件条目的一个子集。填充的数据包括用户名、`uid`、`gid`、`shell` 和主目录。在非 Windows 系统上，所有数据来自 `getpwuid_r(3)`。在 Windows 上，`uid` 和 `gid` 被设为 -1 而没有意义，`shell` 为 `NULL`。成功调用此函数后，分配给 `pwd` 的内存需要用 `uv_os_free_passwd` 释放。

自 1.9.0 版本新增

int uv_os_get_passwd2(uv_passwd_t* pwd, uv_uid_t uid)

获取所提供 uid 的密码文件条目的一个子集。填充的数据包括用户名、euid、gid、shell 和主目录。在非 Windows 系统上，所有数据来自 `getpwuid_r(3)`。在 Windows 上，uid 和 gid 被设为 -1 而没有意义，shell 为 `NULL`。成功调用此函数后，分配给 `pwd` 的内存需要用 `uv_os_free_passwd` 释放。

自 1.45.0 版本新增

int uv_os_get_group(uv_group_t* group, uv_uid_t gid)

获取所提供 gid 的组文件条目的一个子集。填充的数据包括组名、gid 和成员。在非 Windows 系统上，所有数据来自 `getgrgid_r(3)`。在 Windows 上，uid 和 gid 被设为 -1 而没有意义。成功调用此函数后，分配给 `group` 的内存需要用 `uv_os_free_group` 释放。

自 1.45.0 版本新增

void uv_os_free_group(uv_passwd_t* pwd)

释放之前由 `uv_os_get_group` 分配的内存。

自 1.45.0 版本新增

void uv_os_free_passwd(uv_passwd_t* pwd)

释放之前由 `uv_os_get_passwd` 分配的 `pwd` 内存。

自 1.9.0 版本新增

uint64_t uv_get_free_memory(void)

获取系统中可用的空闲内存量，以内核报告为准（字节）。未知时返回 0。

uint64_t uv_get_total_memory(void)

获取系统中的物理内存总量（字节）。未知时返回 0。

uint64_t uv_get_constrained_memory(void)

根据操作系统施加的限制，获取进程可用的内存总量（字节）。如果没有这样的限制，或限制未知，返回 0。如果存在限制机制但未设置限制，返回 `UINT64_MAX`。注意该值小于或大于 `uv_get_total_memory` 都不罕见。

注意：此函数目前在 Linux（如有 `cgroups` 则基于 `cgroups`）和 z/OS（基于 `RLIMIT_MEMLIMIT`）上才会返回非零值。

自 1.29.0 版本新增

`uint64_t uv_get_available_memory(void)`

获取进程仍然可用的空闲内存量（字节）。它与 `uv_get_free_memory` 的区别在于它考虑了操作系统施加的任何限制。如果没有这样的限制，或限制未知，返回的量与 `uv_get_free_memory` 相同。

注意：此函数目前在 Linux（如有 `cgroups` 则基于 `cgroups`）上才会返回与 `uv_get_free_memory` 不同的值。

自 1.45.0 版本新增

`uint64_t uv_hrttime(void)`

返回当前的高精度时间戳。以纳秒表示。它相对于过去某个任意时刻。它与一天中的时间无关，因此不受时钟漂移影响。主要用途是测量区间之间的性能。

注意：并非每个平台都能支持纳秒分辨率；但此值将始终以纳秒为单位。

`int uv_clock_gettime(uv_clock_id clock_id, uv_timespec64_t* ts)`

从高精度的实时（real-time）或单调（monotonic）时钟源获取当前系统时间。

实时时钟从 UNIX 纪元（1970-01-01）起计数，且会受时间调整影响；它可能向后跳变。

单调时钟从过去某个任意时刻起计数，永远不会向后跳变。

自 1.45.0 版本新增

`void uv_print_all_handles(uv_loop_t loop, FILE stream)`

将与给定 `loop` 关联的所有句柄打印到给定的 `stream`。

示例：

```
uv_print_all_handles(uv_default_loop(), stderr);
/*
[ -I] signal    0x1a25ea8
[ -AI] async    0x1a25cf0
[ R--] idle     0x1a7a8c8
*/
```

格式为 `[flags] handle-type handle-address`。对于 `flags`：

- 句柄被引用（referenced）时打印 `R`
- 句柄处于活动状态（active）时打印 `A`
- 句柄是内部句柄（internal）时打印 `I`

警告：此函数用于临时调试，没有 API/ABI 稳定性保证。

自 1.8.0 版本新增

void uv_print_active_handles(uv_loop_t loop, FILE stream)

与 `uv_print_all_handles` 相同，只是只打印活动句柄。

警告：此函数用于临时调试，没有 API/ABI 稳定性保证。

自 1.8.0 版本新增

int uv_os_environ(uv_env_item_t* envitems, int count)

检索所有环境变量。此函数会分配内存，必须调用 `uv_os_free_environ` 释放。

警告：此函数不是线程安全的。

自 1.31.0 版本新增

void uv_os_free_environ(uv_env_item_t* envitems, int count);

释放 `uv_os_environ` 为环境变量分配的内存。

自 1.31.0 版本新增

int uv_os_getenv(const char name, char buffer, size_t* size)

检索 `name` 指定的环境变量，将其值复制到 `buffer` 中，并把 `size` 设为该值的字符串长度。调用此函数时，`size` 必须被设置为 `buffer` 中可用的存储量（包含 null 结尾符）。如果环境变量的值超出 `buffer` 的可用存储量，返回 `UV_ENOBUFS`，并把 `size` 设为容纳该值所需的存储量。如果不存在匹配的环境变量，返回 `UV_ENOENT`。

警告：此函数不是线程安全的。

自 1.12.0 版本新增

int uv_os_setenv(const char name, const char value)

以 `value` 创建或更新 `name` 指定的环境变量。

警告：此函数不是线程安全的。

自 1.12.0 版本新增

int uv_os_unsetenv(const char* name)

删除 `name` 指定的环境变量。如果不存在这样的环境变量，此函数也会成功返回。

警告：此函数不是线程安全的。

自 1.12.0 版本新增

int uv_os_gethostname(char buffer, size_t size)

以 null 结尾字符串的形式在 `buffer` 中返回主机名，并把 `size` 设为主机名的字符串长度。调用此函数时，`size` 必须被设置为 `buffer` 中可用的存储量（包含 null 结尾符）。如果主机名超出 `buffer` 的可用存储量，返回 `UV_ENOBUFS`，并把 `size` 设为容纳该值所需的存储量。

自 1.12.0 版本新增

1.26.0 版本变更：`UV_MAXHOSTNAMESIZE` 可用，它表示存储主机名和结尾 `nul` 字符所需的最大 `buffer` 大小。

int uv_os_getpriority(uv_pid_t pid, int* priority)

检索 `pid` 指定进程的调度优先级。返回的 `priority` 值介于 -20（高优先级）和 19（低优先级）之间。

注意：在 Windows 上，返回的优先级将等于某个 `UV_PRIORITY` 常量。

自 1.23.0 版本新增

int uv_os_setpriority(uv_pid_t pid, int priority)

设置 `pid` 指定进程的调度优先级。`priority` 值的范围介于 -20（高优先级）和 19（低优先级）之间。为方便起见还提供了常量

`UV_PRIORITY_LOW`、
`UV_PRIORITY_BELOW_NORMAL`、`UV_PRIORITY_NORMAL`、`UV_PRIORITY_ABOVE_NORMAL`、
`UV_PRIORITY_HIGH` 和 `UV_PRIORITY_HIGHEST`。

注意：在 Windows 上，此函数使用 `SetPriorityClass()`。`priority` 参数被映射到一个 Windows 优先级类。检索进程优先级时，结果将等于某个 `UV_PRIORITY` 常量，而不一定是 `priority` 的精确值。

注意：在 Windows 上，设置 `PRIORITY_HIGHEST` 只对提升过权限的用户有效，对其他用户会被静默降为 `PRIORITY_HIGH`。

注意：在 IBM i PASE 上，最高进程优先级是 -10。常量 `UV_PRIORITY_HIGHEST` 为 -10，`UV_PRIORITY_HIGH` 为 -7，`UV_PRIORITY_ABOVE_NORMAL` 为 -4，`UV_PRIORITY_NORMAL` 为 0，`UV_PRIORITY_BELOW_NORMAL` 为 15，`UV_PRIORITY_LOW` 为 39。

注意：在 IBM i PASE 上，除非你拥有 `*JOBCTL` 特殊权限（special authority），否则不允许更改自己的优先级（即便调低也不行）。

自 1.23.0 版本新增

int uv_os_uname(uv_utsname_t* buffer)

在 `buffer` 中检索系统信息。填充的数据包括操作系统名称、发行版、版本和机器类型。在非 Windows 系统上，`uv_os_uname()` 是 `uname(2)` 的一个薄封装。成功返回零，否则返回非零的错误值。

自 1.25.0 版本新增

int uv_gettimeofday(uv_timeval64_t* tv)

`gettimeofday(2)` 的跨平台实现。`gettimeofday()` 的时区参数不受支持，因为它已被视为过时。

自 1.28.0 版本新增

int uv_random(uv_loop_t loop, uv_random_t req, void* buf, size_t buflen, unsigned int flags, uv_random_cb cb)

用从系统 CSPRNG 获取的恰好 `buflen` 个密码学强随机字节填充 `buf`。`flags` 保留用于未来扩展，当前必须为 0。

不可能出现短读 (short read)。当可用的随机字节少于 `buflen` 时，会返回非零的错误值或将其传给回调。

同步版本在熵不足时可能无限期阻塞。异步版本在系统熵低时可能永远不会完成。

熵的来源：

- Windows: `RtlGenRandom` <<https://docs.microsoft.com/en-us/windows/desktop/api/ntsecapi/nf-ntsecapi-rtlgenrandom>> _。
- Linux、Android: 如有 `getrandom(2)` 则使用之，否则在从 `/dev/random` 读取一次后使用 `urandom(4)`，或使用 `KERN_RANDOM` `sysctl(2)`。
- FreeBSD: `getrandom(2)` <<https://www.freebsd.org/cgi/man.cgi?query=getrandom&sektion=2>> _，或在从 `/dev/random` 读取一次后使用 `/dev/urandom`。
- NetBSD: `KERN_ARND` `sysctl(7)` <<https://man.netbsd.org/sysctl.7>> _
- macOS、OpenBSD: 如有 `getentropy(2)` <<https://man.openbsd.org/getentropy.2>> _ 则使用之，或在从 `/dev/random` 读取一次后使用 `/dev/urandom`。
- AIX: `/dev/random`。
- IBM i: `/dev/urandom`。
- 其他 UNIX: 在从 `/dev/random` 读取一次后使用 `/dev/urandom`。

返回值：成功返回 0，失败返回小于 0 的错误码。出错后 `buf` 的内容是未定义的。

注意：使用同步版本时，`loop` 和 `req` 两个参数都不使用，可以设为 `NULL`。

自 1.33.0 版本新增

void uv_sleep(unsigned int msec)

使调用线程休眠 `msec` 毫秒。

自 1.34.0 版本新增

```
#include <uv.h>
#include <stdio.h>
int main() {
    printf("Sleeping for 1 second...\n");
    uv_sleep(1000);
    printf("Awake!\n");
    return 0;
}
```

字符串操作函数

libuv 内部需要这些字符串工具来处理 Windows，并将其导出，以便在 libuv API 尚不完整时让客户端能够统一地处理这些数据。

size_t uv_utf16_length_as_wtf8(const uint16_t* utf16, ssize_t utf16_len)

获取 `utf16` 值（UTF-16 或 UCS-2）转换为 WTF-8 之后的长度。如果 `utf16` 以 NUL 结尾，`utf16_len` 可以设为 -1；否则必须指定。

自 1.47.0 版本新增

int uv_utf16_to_wtf8(const uint16_t utf16, ssize_t utf16_len, char wtf8_ptr, size_t wtf8_len_ptr)

把 `utf16` 中的 UTF-16（或 UCS-2）数据转换为 `*wtf8_ptr` 中的 WTF-8 数据。`utf16_len` 计数（以字符为单位）给出 `utf16` 的长度。如果 `utf16` 以 NUL 结尾，`utf16_len` 可以设为 -1；否则必须指定。如果 `wtf8_ptr` 为 `NULL`，则不计算结果，但长度（等于 `uv_utf16_length_as_wtf8`）会存入 `wtf8_len_ptr`。如果 `*wtf8_ptr` 为 `NULL`，会为转换分配空间并经 `wtf8_ptr` 返回，长度经 `wtf8_len_ptr` 返回。否则，必须把 `*wtf8_ptr` 的长度经 `wtf8_len_ptr` 传入。`wtf8_ptr` 必须多留出一个空间用于结果之后

的额外 NUL。如果结果被截断，将返回 `UV_ENOBUFS`，且 `wtf8_len_ptr` 为容纳完整结果所需的 `wtf8_ptr` 长度。

自 1.47.0 版本新增

`ssize_t uv_wtf8_length_as_utf16(const char* wtf8)`

获取以 NUL 结尾的 UTF-8 `wtf8` 值转换为 UTF-16（或 UCS-2）之后的字符长度，包括 NUL 结尾符。

自 1.47.0 版本新增

`void uv_wtf8_to_utf16(const char utf8, uint16_t utf16, size_t utf16_len)`

把 `wtf8` 中以 NUL 结尾的 UTF-8 数据转换为 `utf16` 中的 UTF-16（或 UCS-2）数据。
`utf16_len` 计数（以字符为单位）必须包含 NUL 结尾符的空间。

自 1.47.0 版本新增

错误处理

在 `libuv` 中，错误是否为负数的常量。根据经验法则，只要出现状态参数，或某个 API 函数返回整数，负数就意味着出错。

当接受回调的函数返回错误时，回调将永远不会被调用。

注意：实现细节：在 Unix 上错误码是对 `errno` 取负（即 `-errno`），而在 Windows 上它们由 `libuv` 定义为任意的负数。

错误常量

`UV_E2BIG`

参数列表太长

`UV_EACCES`

权限被拒绝

UV_EADDRINUSE

地址已被占用

UV_EADDRNOTAVAIL

地址不可用

UV_EAFNOSUPPORT

不支持该地址族

UV_EAGAIN

资源暂时不可用

UV_EAI_ADDRFAMILY

不支持该地址族

UV_EAI_AGAIN

临时失败

UV_EAI_BADFLAGS

无效的 ai_flags 值

UV_EAI_BADHINTS

hints 值无效

UV_EAI_CANCELED

请求已取消

UV_EAI_FAIL

永久性失败

UV_EAI_FAMILY

不支持 ai_family

UV_EAI_MEMORY

内存不足

UV_EAI_NODATA

没有地址

UV_EAI_NONAME

未知的节点或服务

UV_EAI_OVERFLOW

参数缓冲区溢出

UV_EAI_PROTOCOL

解析出的协议未知

UV_EAI_SERVICE

该套接字类型没有可用的服务

UV_EAI_SOCKTYPE

不支持该套接字类型

UV_EALREADY

连接已在进行中

UV_EBADF

无效的文件描述符

UV_EBUSY

资源繁忙或被锁定

UV_ECANCELED

操作已取消

UV_ECHARSET

无效的 Unicode 字符

UV_ECONNABORTED

软件导致的连接中止

UV_ECONNREFUSED

连接被拒绝

UV_ECONNRESET

连接被对端重置

UV_EDESTADDRREQ

需要目标地址

UV_EEXIST

文件已存在

UV_EFAULT

系统调用参数中的地址错误

UV_EFBIG

文件过大

UV_EHOSTUNREACH

主机不可达

UV_EINTR

系统调用被中断

UV_EINVAL

无效的参数

UV_EIO

I/O 错误

UV_EISCONN

套接字已连接

UV_EISDIR

对目录的非法操作

UV_ELOOP

遇到过多符号链接

UV_EMFILE

打开的文件过多

UV EMSGSIZE

消息过长

UV_ENAMETOOLONG

名称过长

UV_ENETDOWN

网络已断开

UV_ENETUNREACH

网络不可达

UV_ENFILE

文件表溢出

UV_ENOBUFS

没有可用的缓冲区空间

UV_ENODEV

没有这样的设备

UV_ENOENT

没有这样的文件或目录

UV_ENOMEM

内存不足

UV_ENONET

机器不在网络上

UV_ENOPROTOOPT

协议不可用

UV_ENOSPC

设备上没有剩余空间

UV_ENOSYS

函数未实现

UV_ENOTCONN

套接字未连接

UV_ENOTDIR

不是目录

UV_ENOTEMPTY

目录非空

UV_ENOTSOCK

对非套接字进行套接字操作

UV_ENOTSUP

套接字不支持该操作

UV_EOVERFLOW

值对于定义的数据类型而言过大

UV_EPERM

操作不被允许

UV_EPIPE

管道破裂

UV_EPROTO

协议错误

UV_EPROTONOSUPPORT

不支持该协议

UV_EPROTOTYPE

套接字的协议类型错误

UV_ERANGE

结果过大

UV_EROFS

只读文件系统

UV_ESHUTDOWN

传输端点关闭后无法发送

UV_ESPIPE

无效的定位 (seek)

UV_ESRCH

没有这样的进程

UV_ETIMEDOUT

连接超时

UV_ETXTBSY

文本文件繁忙

UV_EXDEV

不允许跨设备链接

UV_UNKNOWN

未知错误

UV_EOF

文件结束

UV_ENXIO

没有这样的设备或地址

UV_EMLINK

链接过多

UV_ENOTTY

对该设备不适当的 ioctl 操作

UV_EFTYPE

不适当的文件类型或格式

UV_EILSEQ

非法的字节序列

UV_ESOCKTNOSUPPORT

不支持该套接字类型

UV_EUNATCH

协议驱动未附加

API

UV_ERRNO_MAP(iter_macro)

一个宏，它会对上述每个错误常量展开为一系列对 `iter_macro` 的调用。`iter_macro` 以两个参数被调用：不带 `uv_` 前缀的错误常量名，以及错误消息字符串字面量。

const char* uv_strerror(int err)

返回给定错误码的错误消息。当用未知错误码调用它时会泄漏少量内存。

char uv_strerror_r(int err, char buf, size_t buflen)

返回给定错误码的错误消息。以零结尾的消息被存入用户提供的、至多 `buflen` 字节的缓冲区 `buf` 中。

自 1.22.0 版本新增

const char* uv_err_name(int err)

返回给定错误码的错误名称。当用未知错误码调用它时会泄漏少量内存。

char uv_err_name_r(int err, char buf, size_t buflen)

返回给定错误码的错误名称。以零结尾的名称被存入用户提供的、至多 `buflen` 字节的缓冲区 `buf` 中。

自 1.22.0 版本新增

int uv_translate_sys_error(int sys_errno)

返回与给定平台相关错误码等价的 libuv 错误码：在 Unix 上是 POSIX 错误码（存储在 `errno` 中的那些），在 Windows 上是 Win32 错误码（由 `GetLastError()` 或 `WSAGetLastError()` 返回的那些）。

如果 `sys_errno` 已经是 libuv 错误码，则原样返回。

1.10.0 版本变更：函数被声明为公开。

设计概述

libuv 是一个跨平台支持库，最初为 Node.js 编写。它围绕事件驱动的异步 I/O 模型设计。

这个库提供的远不止是对不同 I/O 轮询机制的简单抽象：'句柄' (handles) 和 '流' (streams) 为套接字和其他实体提供了高层抽象；此外还提供跨平台的文件 I/O 和线程功能等。

下面的图解说明了组成 libuv 的各个部分以及它们与各子系统的关系：

(图形版参见官方文档 [static/architecture.png](#)，文本版参见 [static/architecture.txt](#)。)

句柄与请求

libuv 为用户提供了两种与事件循环配合使用的抽象：句柄 (handle) 和请求 (request)。

句柄表示长生命周期的对象，在其活动期间能够执行某些操作。一些例子：

- prepare (准备) 句柄在活动期间每轮循环迭代都会触发一次回调。
- TCP 服务器句柄每有新连接时都会触发一次连接回调。

请求表示 (通常) 短生命周期的操作。这些操作可以在句柄之上执行：写请求用于在句柄上写数据；也可以独立存在：getaddrinfo 请求不需要句柄，它们直接在事件循环上运行。

处理句柄和请求的准则：

1. 如果 `uv_foo_init()` 成功初始化了句柄，就必须调用 `uv_close`。如果句柄的 `init` 函数报错，则无需做任何事情。
2. 只有句柄会被关闭，请求不会。例如，`uv_tcp_t` 是句柄，`uv_write_t` 是请求。
3. 句柄的内存只能在 `uv_close_cb` 内部或之后被回收或复用，不能更早。

4. 大多数句柄都有 `init + start/stop` 函数；有些句柄没有。例如 `uv_tcp_t` 与 `uv_process_t`；`uv_spawn` 把句柄初始化和进程启动合并成了一个。
5. 请求在完成时、或被 `uv_cancel` 取消时自动关闭。
6. 除 `uv_fs_t` 和 `uv_getaddrinfo_t` 请求外，无需额外的清理。对于 `uv_fs_t`，用完后调用 `uv_fs_req_cleanup`；对于 `uv_getaddrinfo_t`，则是 `uv_freeaddrinfo`。
7. 从那时起，请求的内存才能被回收或复用。

I/O 循环

I/O（事件）循环是 libuv 的核心部分。它为所有 I/O 操作建立上下文，且被设计为绑定到单个线程。可以运行多个事件循环，只要每个运行在不同的线程中。libuv 的事件循环（以及任何其他涉及循环或句柄的 API）**不是线程安全的**，除非另有说明。

事件循环遵循相当常见的单线程异步 I/O 方式：所有（网络）I/O 都在非阻塞套接字上执行，并使用给定平台上可用的最佳机制进行轮询：Linux 上是 `epoll`，OS X 和其他 BSD 上是 `kqueue`，SunOS 上是 `event ports`，Windows 上是 `IOCP`。作为循环迭代的一部分，循环会阻塞等待已加入轮询器的套接字上的 I/O 活动，并触发回调来指示套接字状态（可读、可写、挂断），以便句柄能够读取、写入或执行所需的 I/O 操作。

为了更好地理解事件循环是如何运作的，下图说明了一轮循环迭代的全部阶段：

（图形版参见官方文档 [static/loop_iteration.png](#)，文本版参见 [static/loop_iteration.txt](#)。）

1. 循环的 'now' 概念被初始设定。
2. 如果循环以 `UV_RUN_DEFAULT` 模式运行，到期的定时器会被执行。所有计划时间早于循环 now 的活动定时器都会触发各自的回调。
3. 如果循环是存活的，则开始一轮迭代，否则循环立即退出。那么，何时认为循环是存活的呢？如果循环有活动的且被引用（ref'd）的句柄、活动请求或正在关闭的句柄，它就被认为是存活的。
4. 待处理的回调被调用。多数情况下，所有 I/O 回调都会在轮询 I/O 之后立即被调用。不过在某些情况下，此类回调的调用会被推迟到下一轮循环迭代。如果上一轮迭代推迟了任何 I/O 回调，它会在此刻运行。

5. idle（空闲）句柄回调被调用。尽管名字容易让人误会，只要处于活动状态，idle 句柄在每轮循环迭代中都会运行。
6. prepare（准备）句柄回调被调用。prepare 句柄的回调恰在循环因 I/O 而阻塞之前被触发。
7. 计算轮询超时。在因 I/O 阻塞之前，循环会计算它应该阻塞多久。计算超时的规则如下：
 8. 如果循环以 `UV_RUN_NOWAIT` 标志运行，超时为 0。
 9. 如果循环即将停止（已调用 `uv_stop`），超时为 0。
 10. 如果没有活动的句柄或请求，超时为 0。
 11. 如果有任何活动的 idle 句柄，超时为 0。
 12. 如果有句柄正待关闭，超时为 0。
 13. 如果以上情况都不匹配，则取最近的定时器的超时；如果没有活动定时器，则为无穷大。
14. 循环因 I/O 而阻塞。此时循环会按上一步计算出的时长阻塞等待 I/O。所有正在监视给定文件描述符读写操作的 I/O 相关句柄都会在此刻触发各自的回调。
15. check（检查）句柄回调被调用。check 句柄的回调恰在循环因 I/O 阻塞之后被触发。check 句柄本质上是 prepare 句柄的对应物。
16. 关闭回调被调用。如果句柄是通过调用 `uv_close` 关闭的，它的关闭回调会被调用。
17. 循环的 'now' 概念被更新。
18. 到期的定时器被执行。注意，'now' 在下一轮循环迭代之前不会再次更新。因此，如果某个定时器在其他定时器处理期间到期，它要等到下一轮事件循环迭代才会被执行。
19. 迭代结束。如果循环以 `UV_RUN_NOWAIT` 或 `UV_RUN_ONCE` 模式运行，迭代结束，`uv_run` 将返回。如果循环以 `UV_RUN_DEFAULT` 运行，只要它仍然存活就会从头继续，否则也将结束。

重要：libuv 使用线程池来实现异步文件 I/O 操作，但网络 I/O **总是在**单线程（即每个循环各自的线程）中执行。

注意：虽然轮询机制各不相同，libuv 使 Unix 系统和 Windows 之间的执行模型保持一致。

文件 I/O

与网络 I/O 不同，没有 libuv 可以依赖的平台特定的文件 I/O 原语，因此目前的方式是在线程池中运行阻塞式文件 I/O 操作。

关于跨平台文件 I/O 格局的详细说明，请参阅[这篇文章](#)。

libuv 目前使用一个全局线程池，所有循环都可以向它排队工作。目前在这个线程池上运行 3 类操作：

- 文件系统操作
- DNS 函数（getaddrinfo 和 getnameinfo）
- 通过 `uv_queue_work` 提交的用户指定代码

警告：更多细节参见线程池一节，但请记住线程池的大小相当有限。

共享库处理

libuv 提供跨平台工具用于加载共享库并从中检索符号，使用以下 API。

数据类型

`uv_lib_t`

共享库数据类型。

公开成员

无。

API

int uv_dlopen(const char filename, uv_lib_t lib)

打开一个共享库。文件名采用 utf-8。成功返回 0，出错返回 -1。调用 `uv_dLError` 获取错误消息。

void uv_dlclose(uv_lib_t* lib)

关闭共享库。

int uv_dlsym(uv_lib_t lib, const char name, void ptr)**

从动态库中检索数据指针。符号映射到 NULL 是合法的。成功返回 0，未找到符号返回 -1。

const char uv_dLError(const uv_lib_t lib)

返回上一次 uv_dlopen() 或 uv_dlsym() 的错误消息。